

Содержание

1. Задание 1. Линейные фильтры	2
1.1. Фильтр первого порядка	2
1.2. Режекторный полосовой фильтр	16
2. Задание 2. Сглаживание биржевых данных	51
3. Приложение	53

1. Задание 1. Линейные фильтры

Зададимся числами a , t_1 , t_2 такими, что $t_1 < t_2$, и построим функцию

$$g(t) = \begin{cases} a, & t \in [t_1, t_2], \\ 0, & t \notin [t_1, t_2]; \end{cases}$$

и её зашумлённую версию

$$u(t) = g(t) + b\xi(t) + c \sin(dt),$$

где $\xi(t) \sim \mathcal{U}[-1, 1]$ - равномерное распределение, представляющее белый шум, а значения b , c , d - параметры возмущения.

1.1. Фильтр первого порядка

Применим $c = 0$. Зададим постоянную времени $T > 0$ и пропустим сигнал $u(t)$ через линейный фильтр первого порядка

$$W_1(p) = \frac{1}{Tp + 1}.$$

Построим сравнительные графики исходного и фильтрованного сигналов, графики модулей их Фурье-образов, а так же АЧХ фильтра. Исследуем влияние постоянной времени T и значение параметра a на эффективность фильтрации.

Графики сигналов представлены на рис. 1.1-1.3, графики Фурье-образов на рис. 1.4-1.6, АЧХ фильтра на рис. 1.7.

Анализ

Из графиков видим, что параметр $T = \frac{1}{\omega_c}$ влияет на сглаживание и запаздывание сигнала. При увеличении T остаётся меньше шумов, но при этом увеличивается запаздывание сигнала и он теряет свою форму. По сути, T - это время отклика, насколько быстро фильтр реагирует на изменение сигнала. Поэтому при увеличении T увеличивается запаздывание, но фильтр накапливает больше информации о сигнале и фильтрация шумов получается более эффективной.

Наиболее оптимальным в нашем варианте является $T \approx 0.1$ при $a = 2$ и $T = 0.025$ при остальных a , при таких комбинациях остаётся мало шума, и форма сигнала близка к оригиналу.

Параметр a определяет амплитуду сигнала $g(t)$. При большем a зашумленный сигнал эффективнее фильтруется, так как мы рассматриваем фиксированные параметры генерации шума, и при большей амплитуде исходного сигнала, отношение сигнал/шум увеличивается, делая шум менее заметным относительно полезного сигнала.

Посмотрим на графики Фурье-образа: подавление верхних частот происходит постепенно - чем больше частота, тем сильнее уменьшается её амплитуда. Для более точного объяснения обратимся к графикам АЧХ, на которых видим, что в данном фильтре подавление частот происходит на всём спектре (нет усиления выбранных частот), и при увеличении T фильтр начинает подавлять высокочастотные помехи быстрее, фильтр становится более "жёстким".

На графиках 1.8 - 1.13 представлена проверка теоремы о свёртке для временной и частотной области. Эксперимент подтвердил теоретические ожидания.

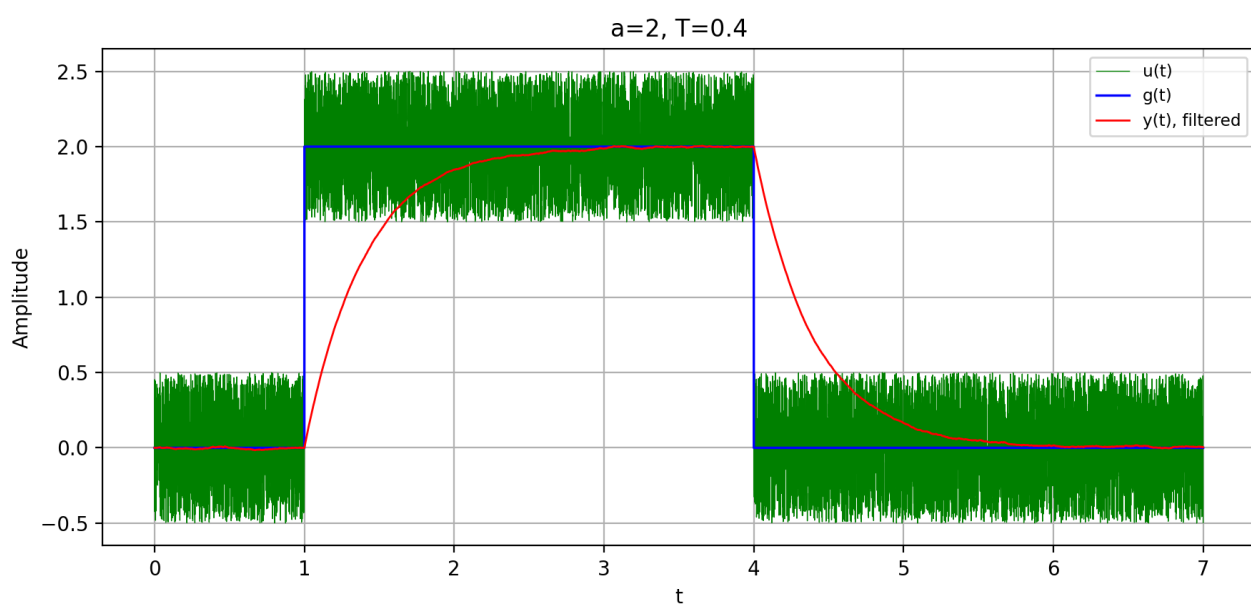
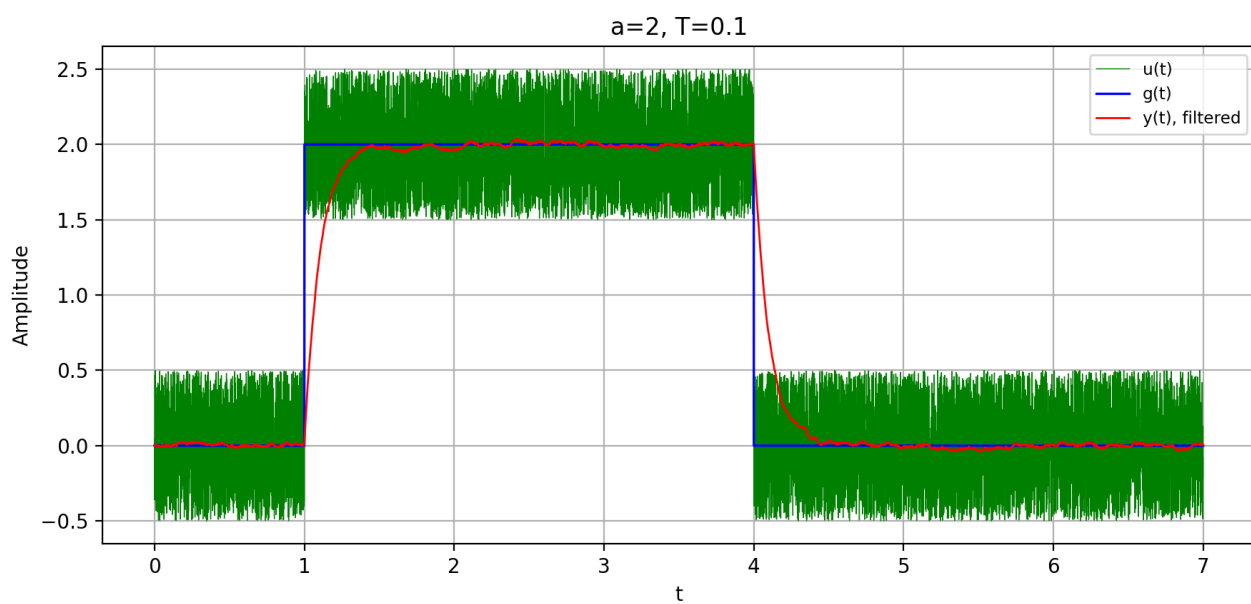
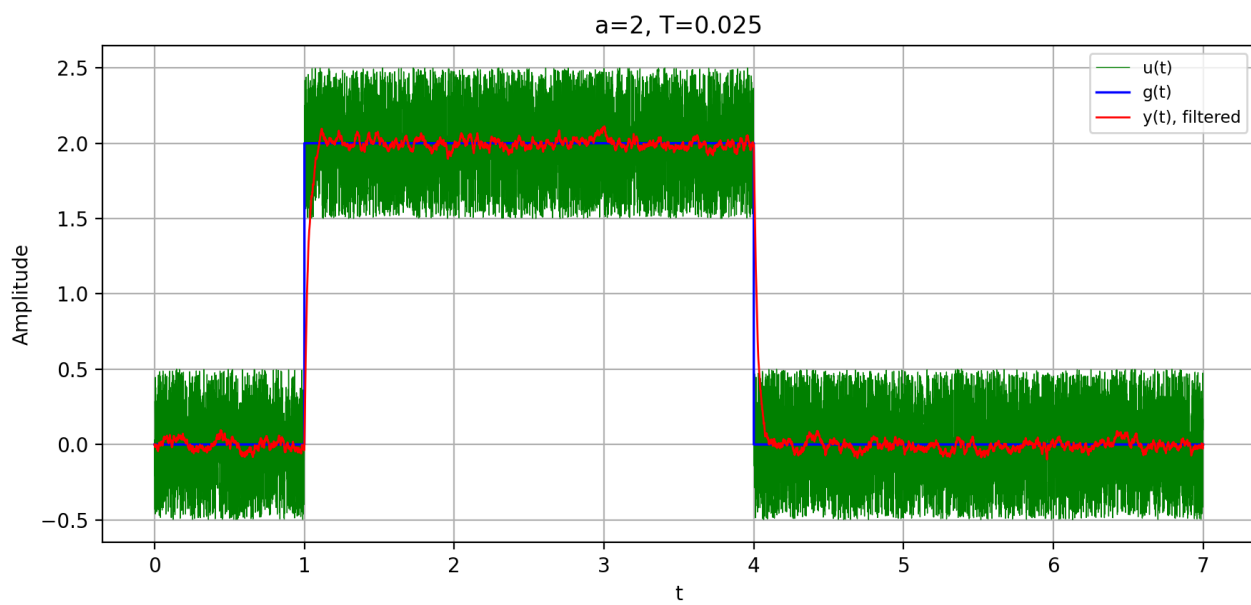


Рисунок 1.1. Сигнал

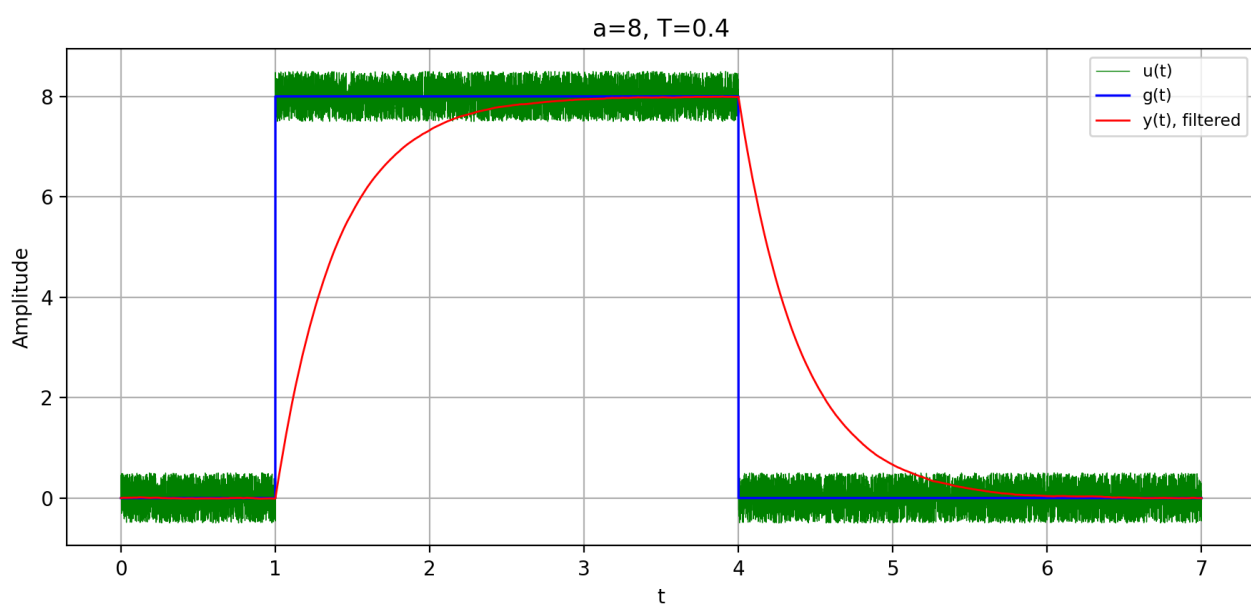
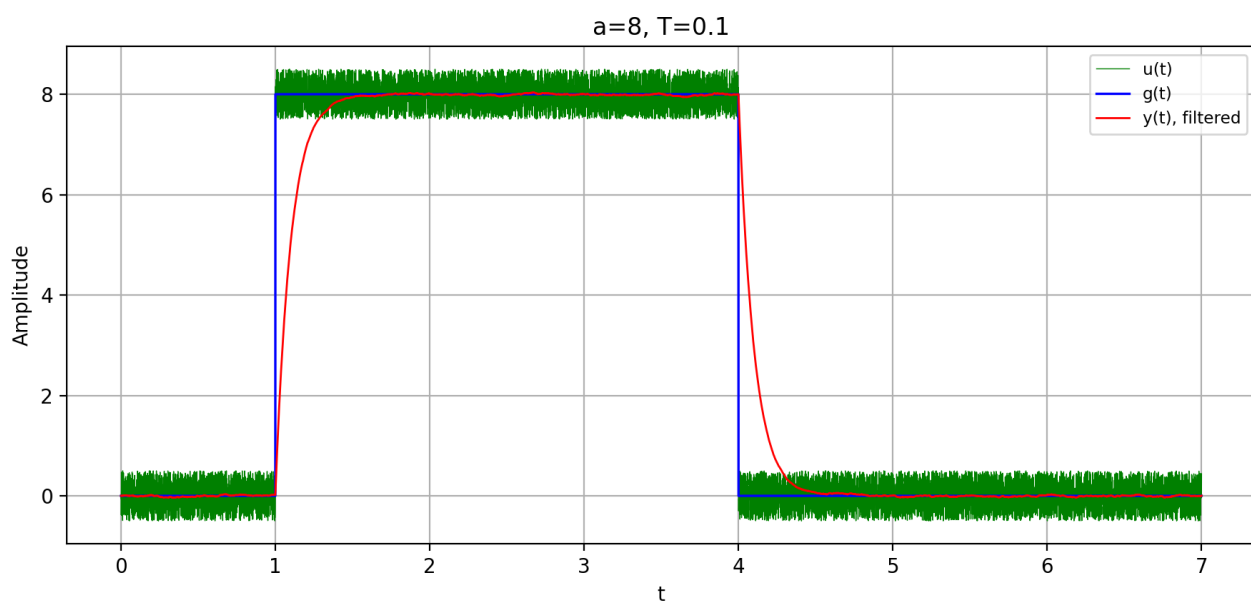
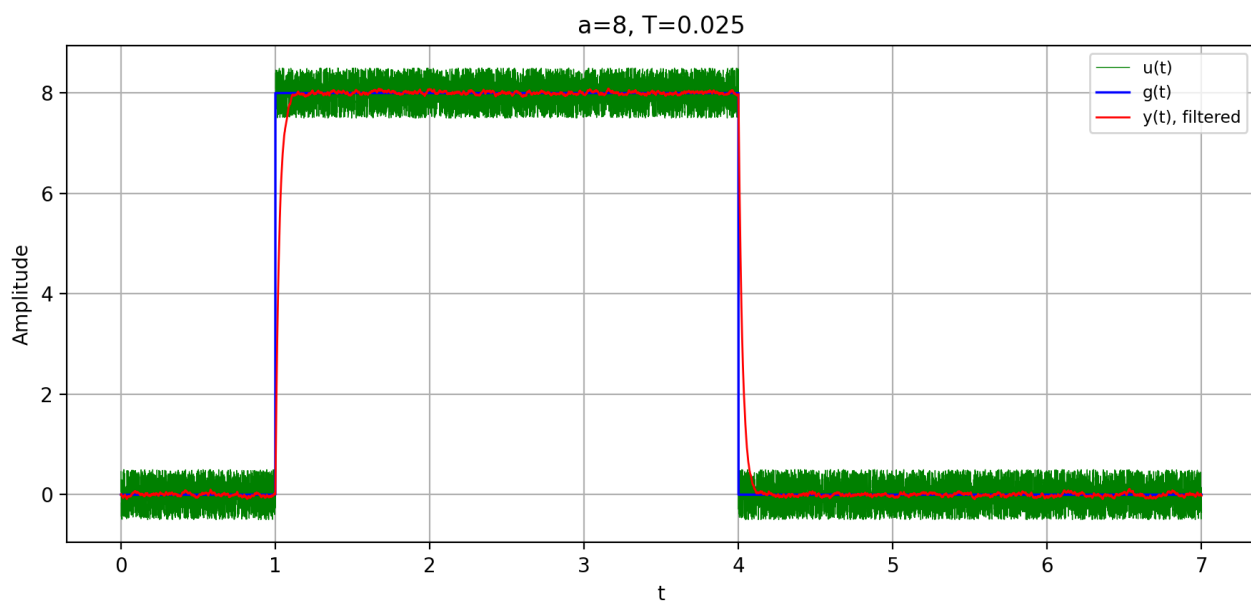


Рисунок 1.2. Сигнал

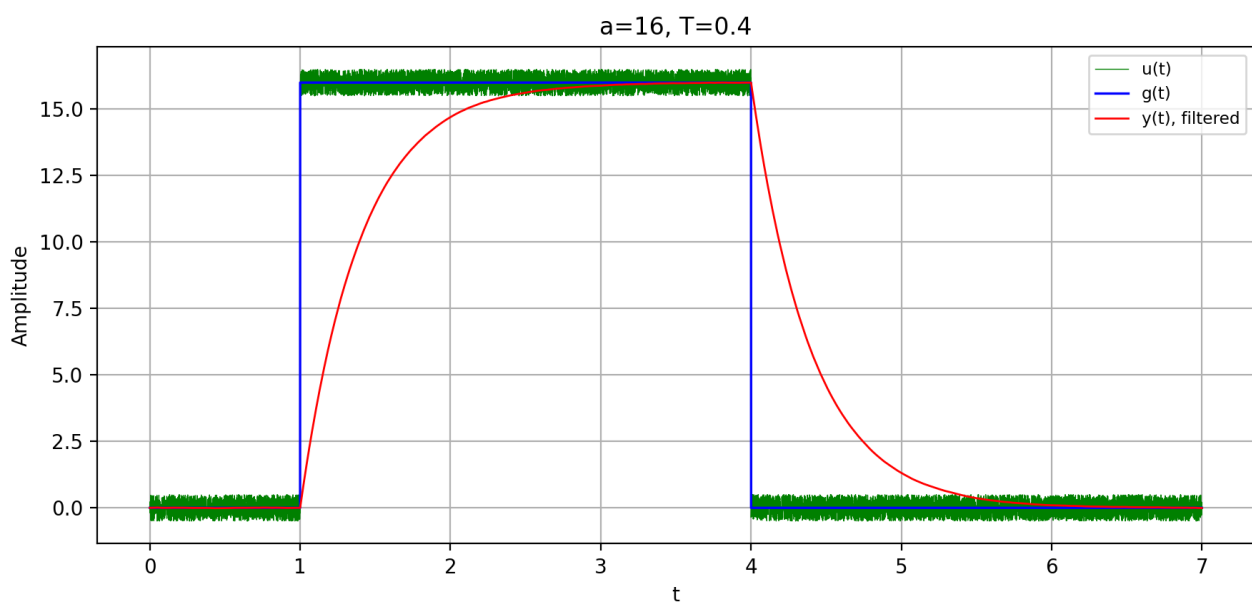
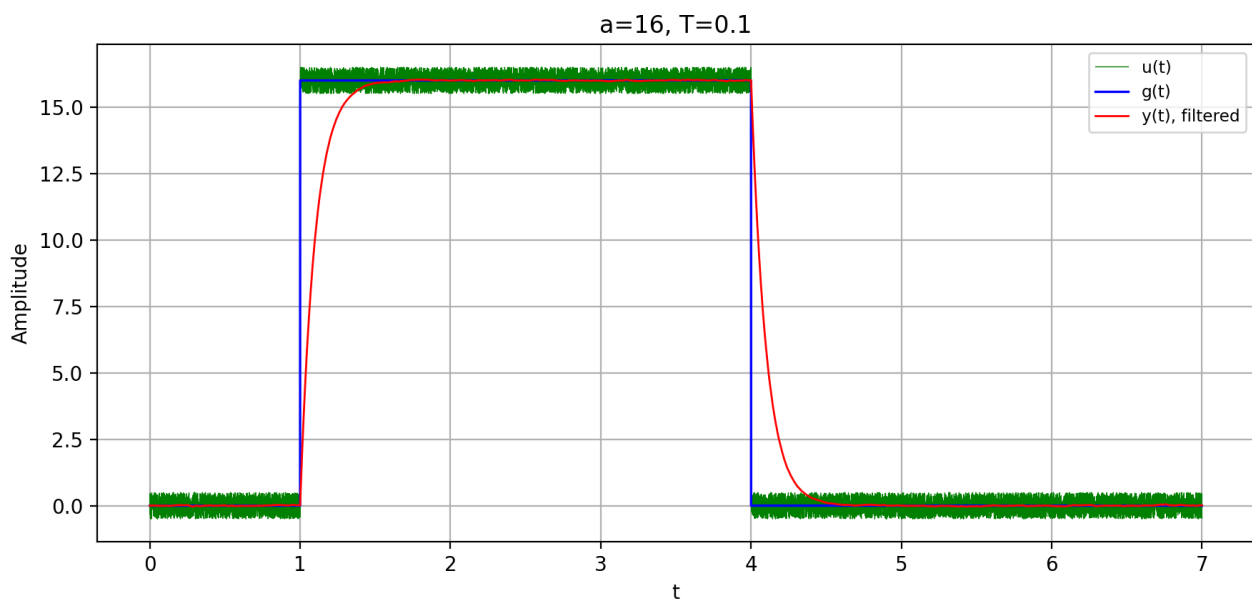
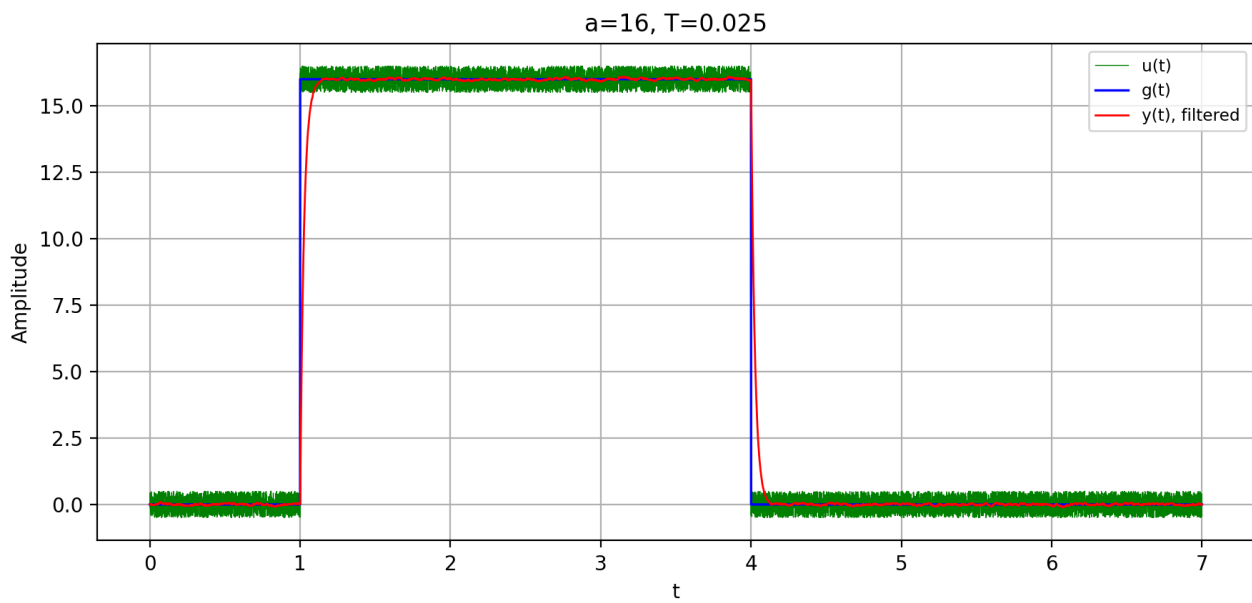


Рисунок 1.3. Сигнал

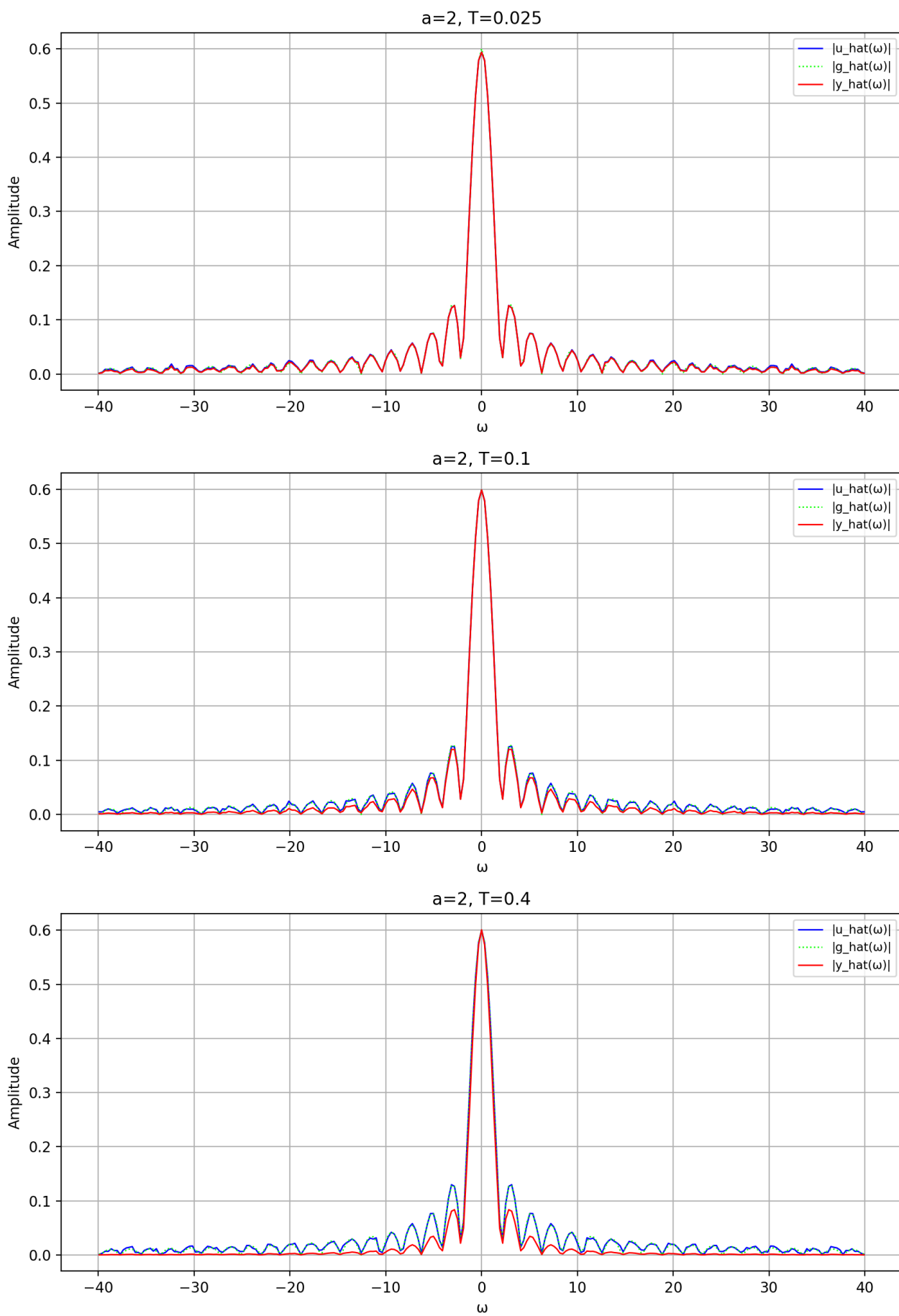


Рисунок 1.4. Фурье-образ

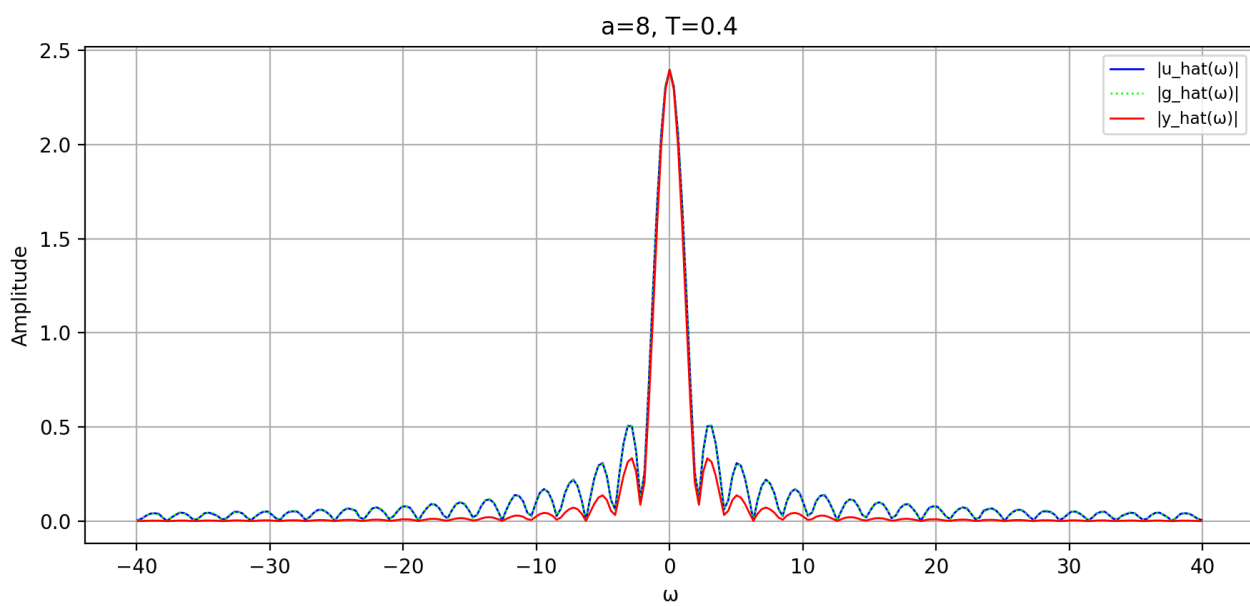
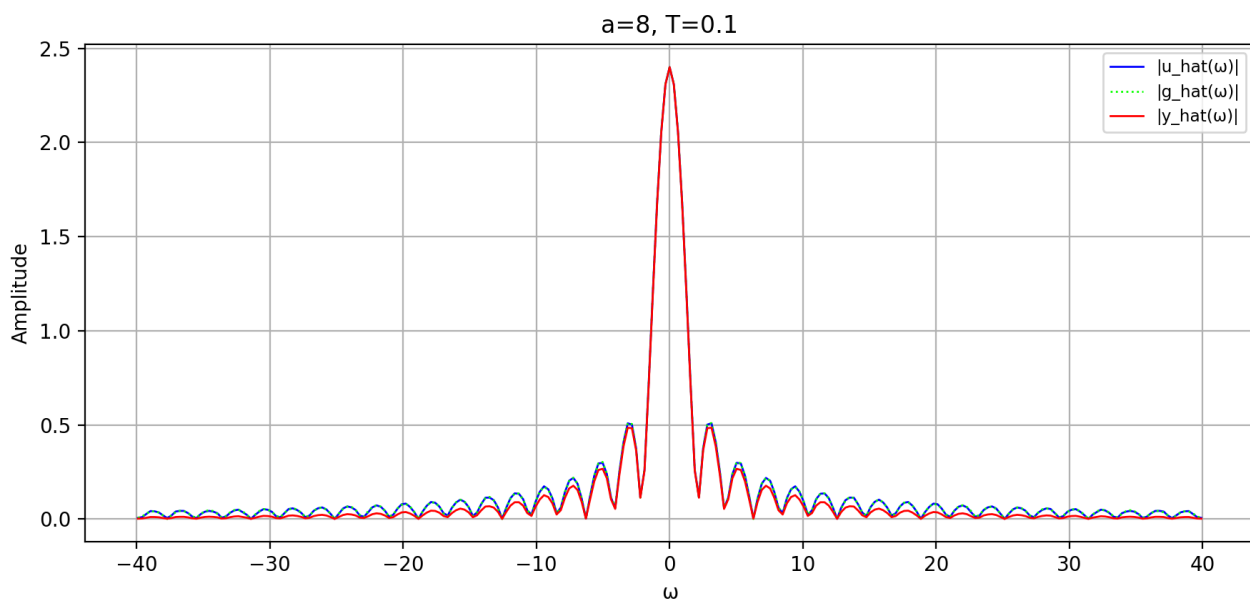
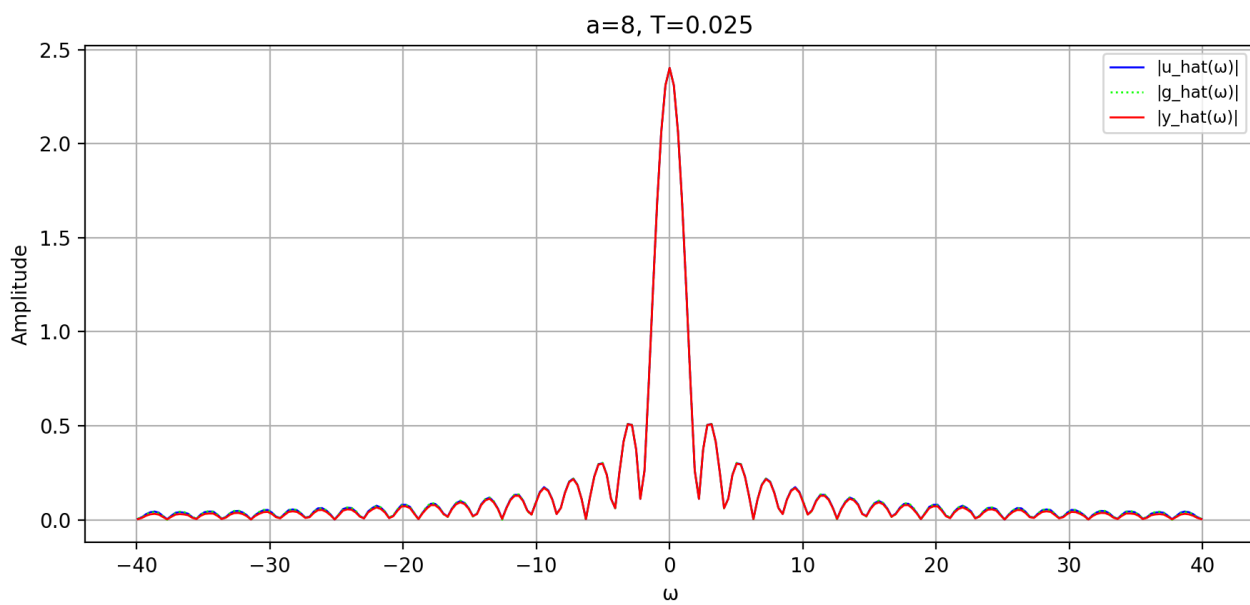


Рисунок 1.5. Фурье-образ

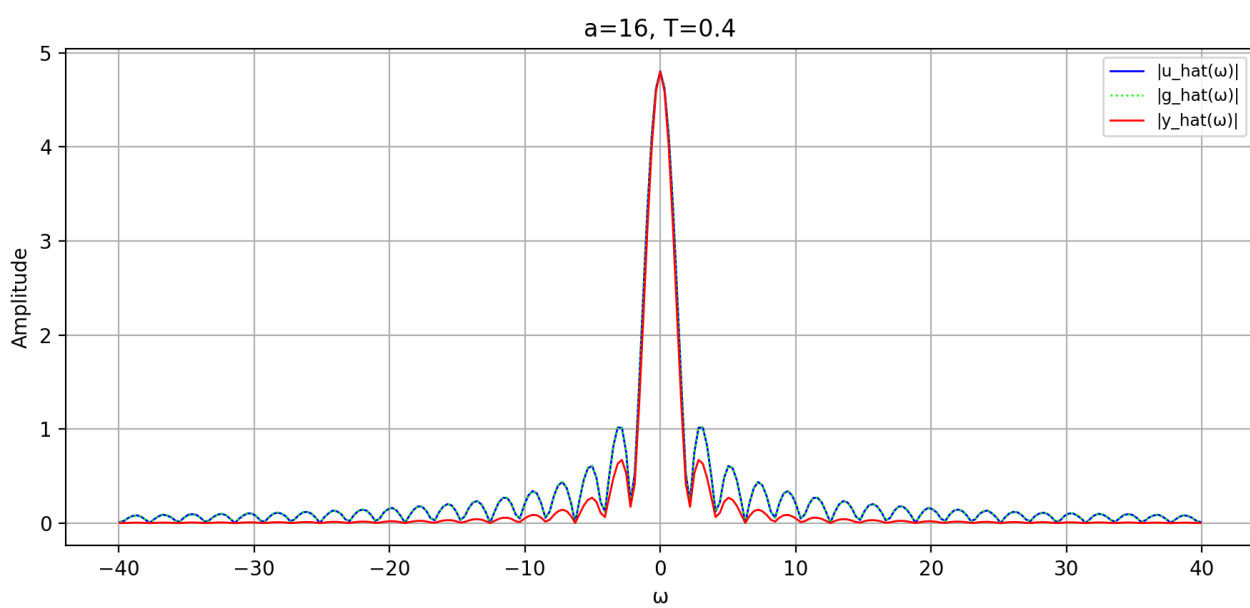
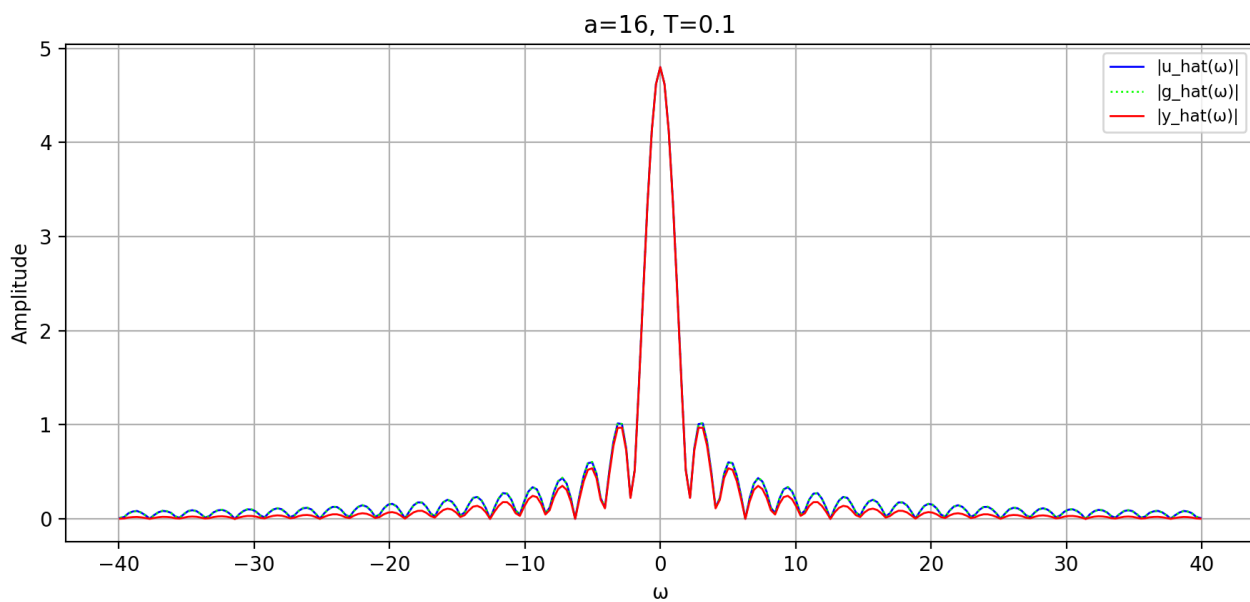
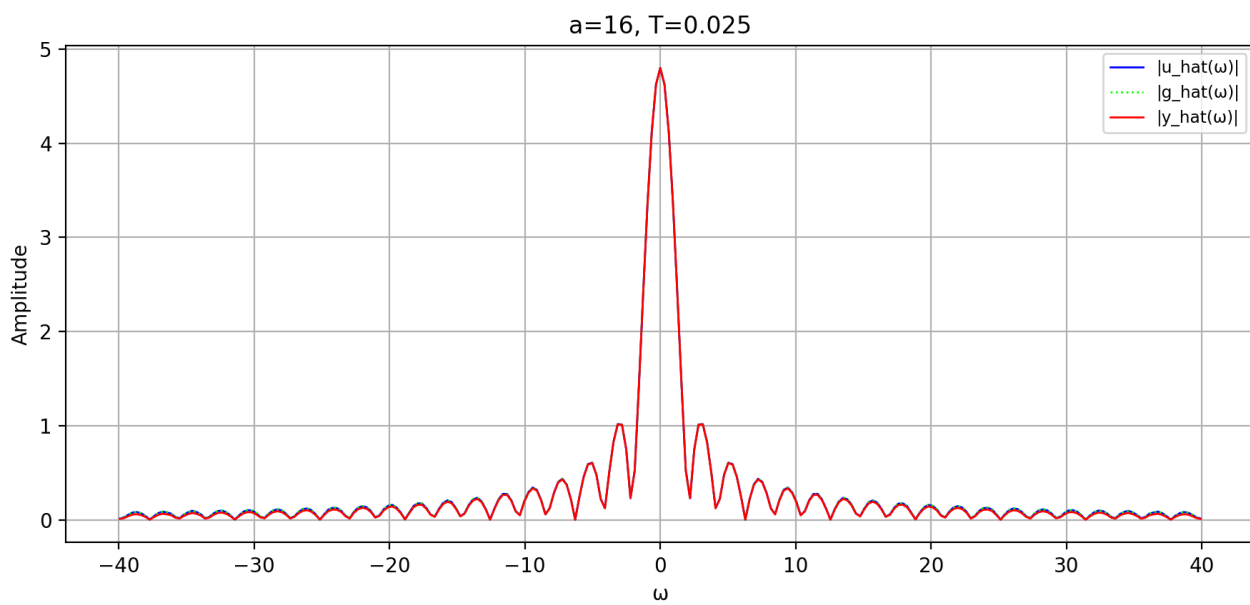


Рисунок 1.6. Фурье-образ

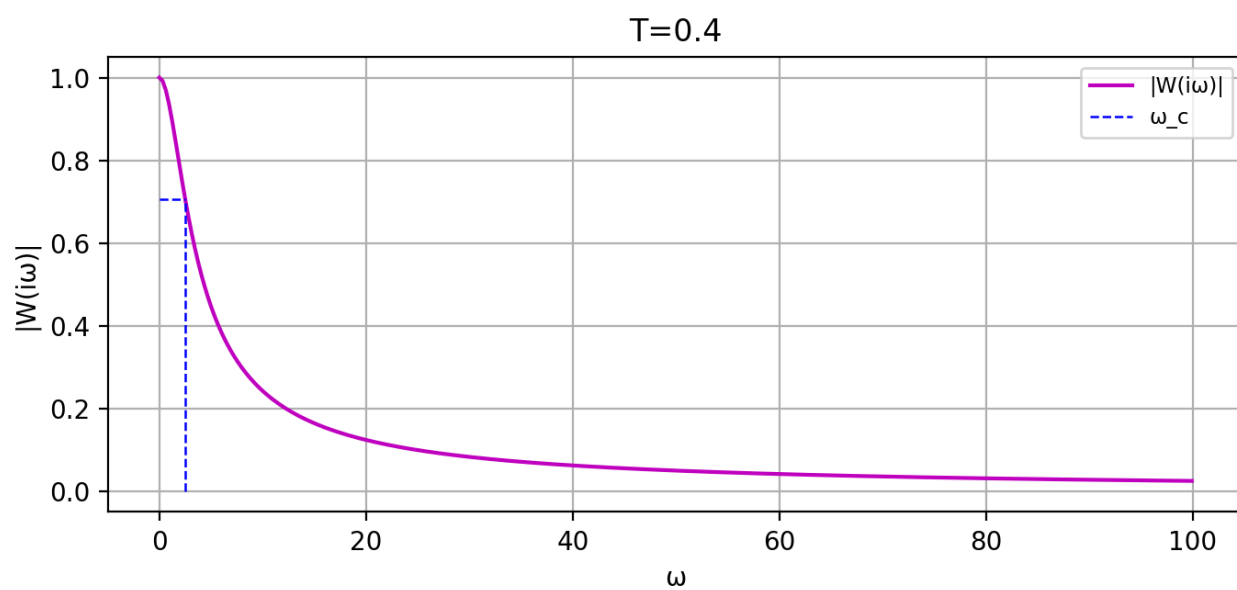
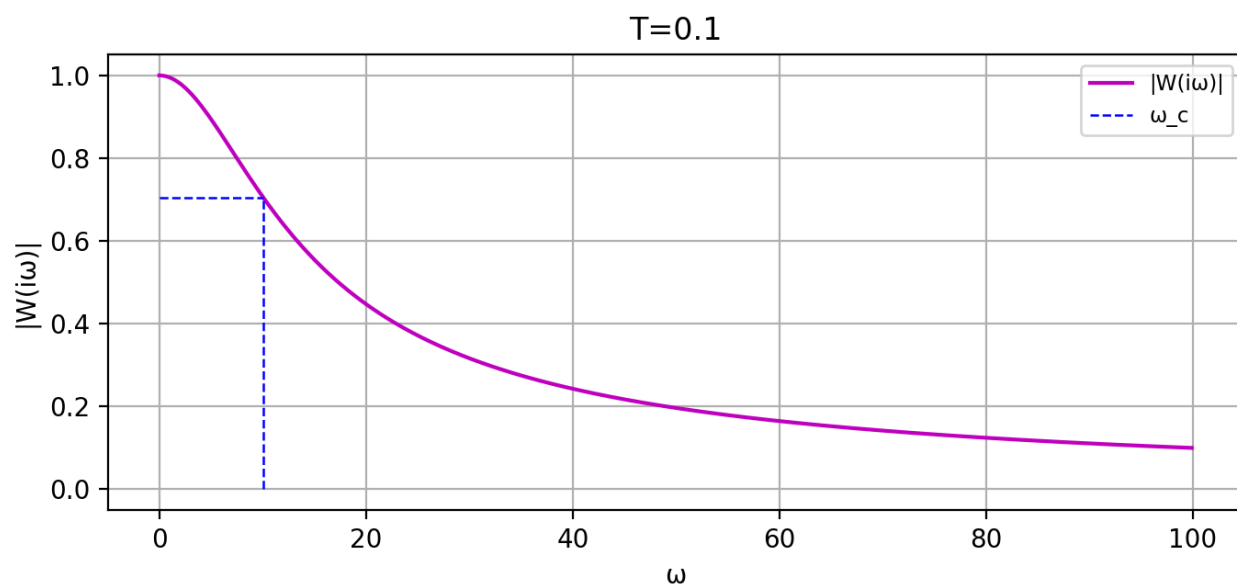
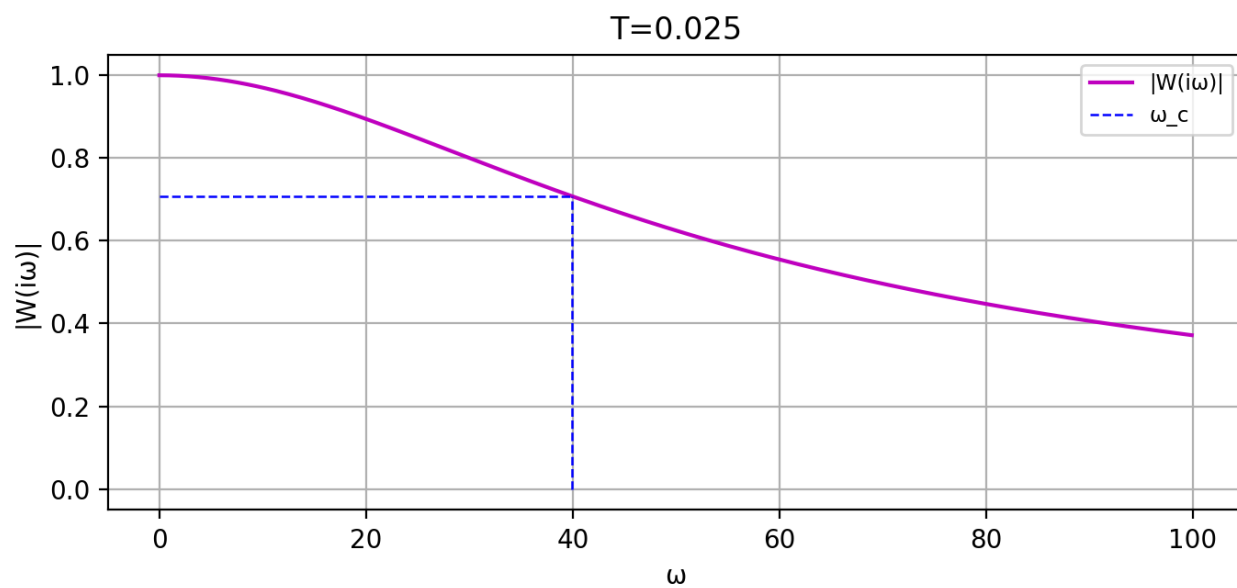


Рисунок 1.7. АЧХ

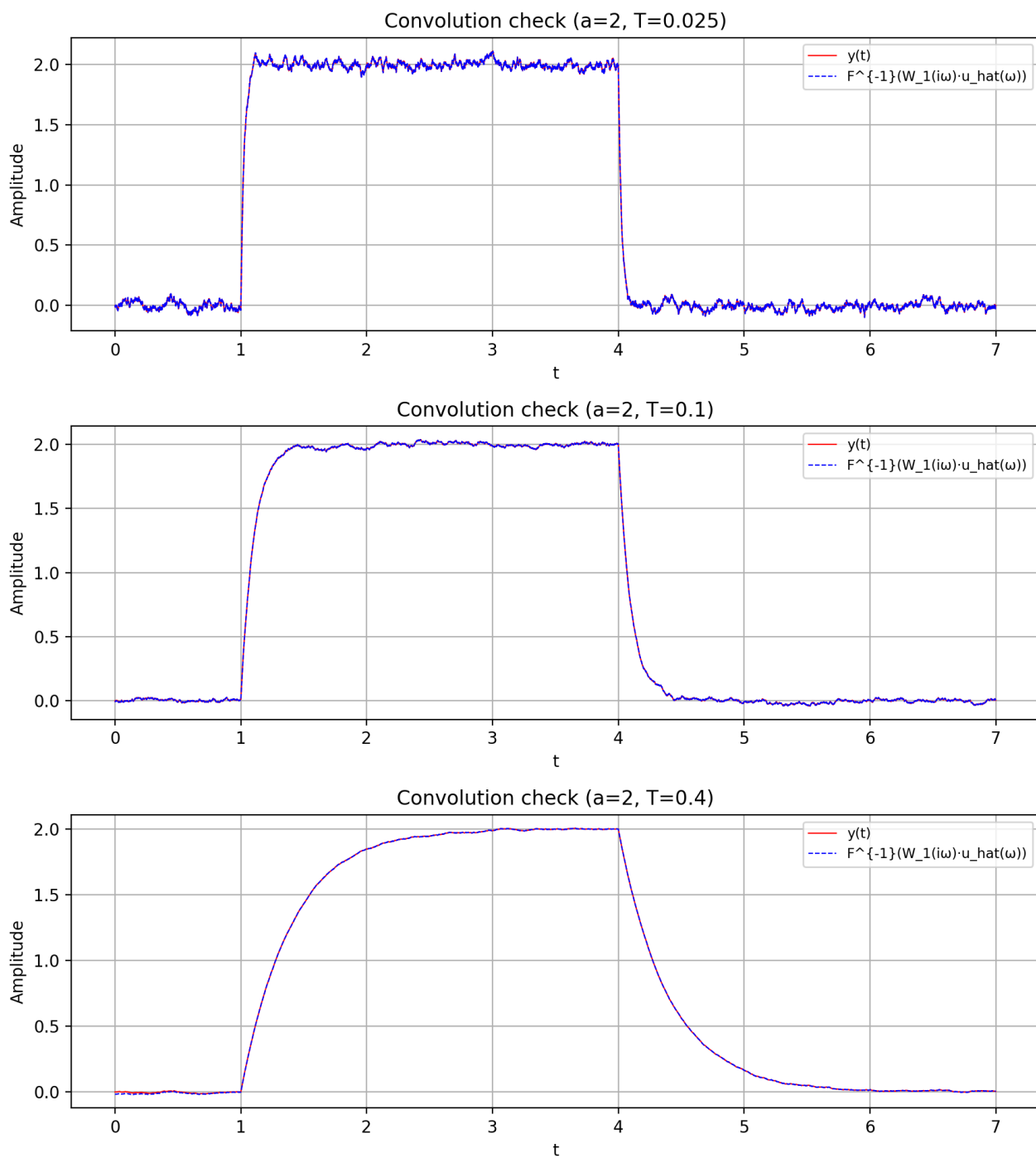


Рисунок 1.8. Временная область

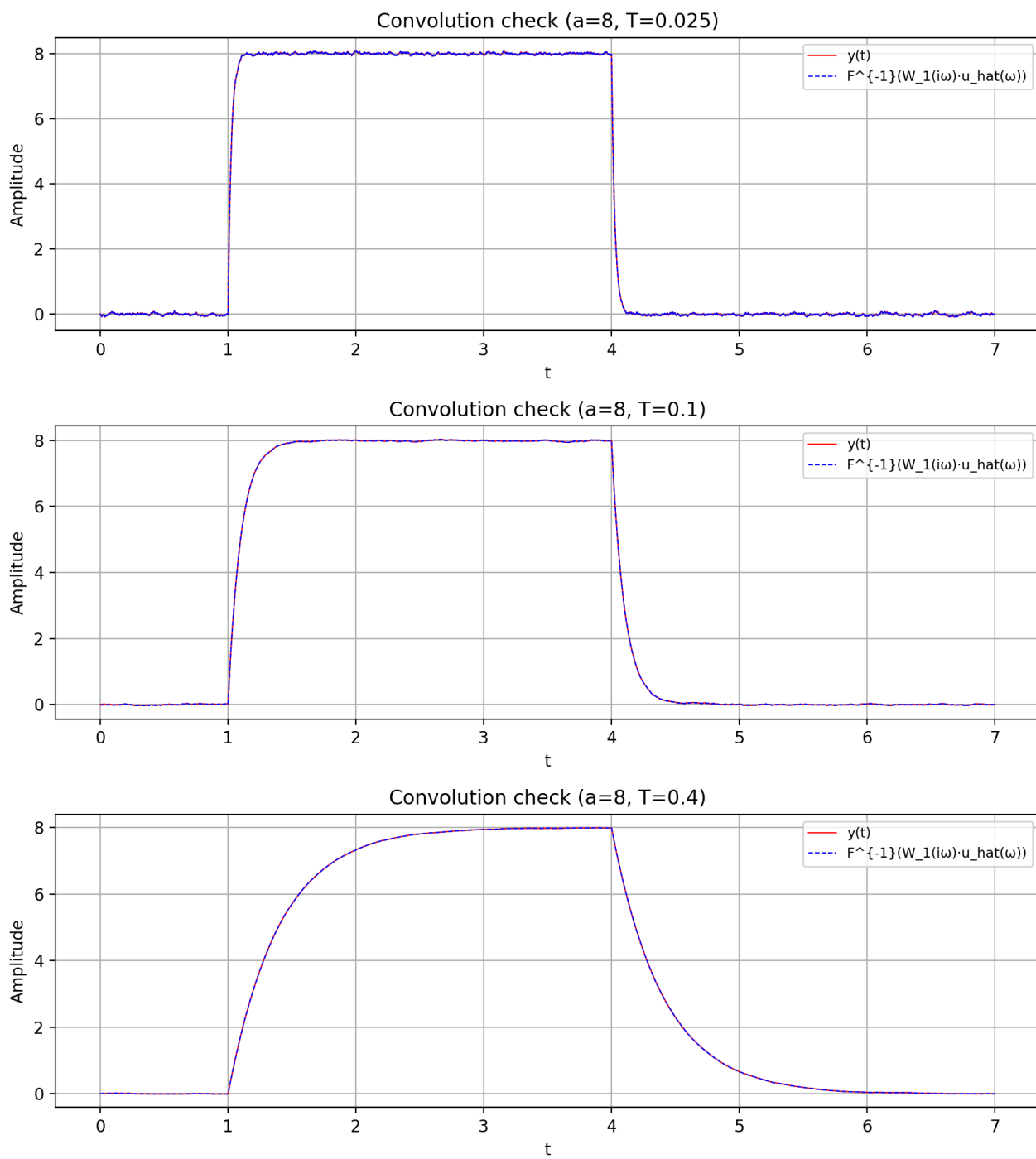


Рисунок 1.9. Временная область

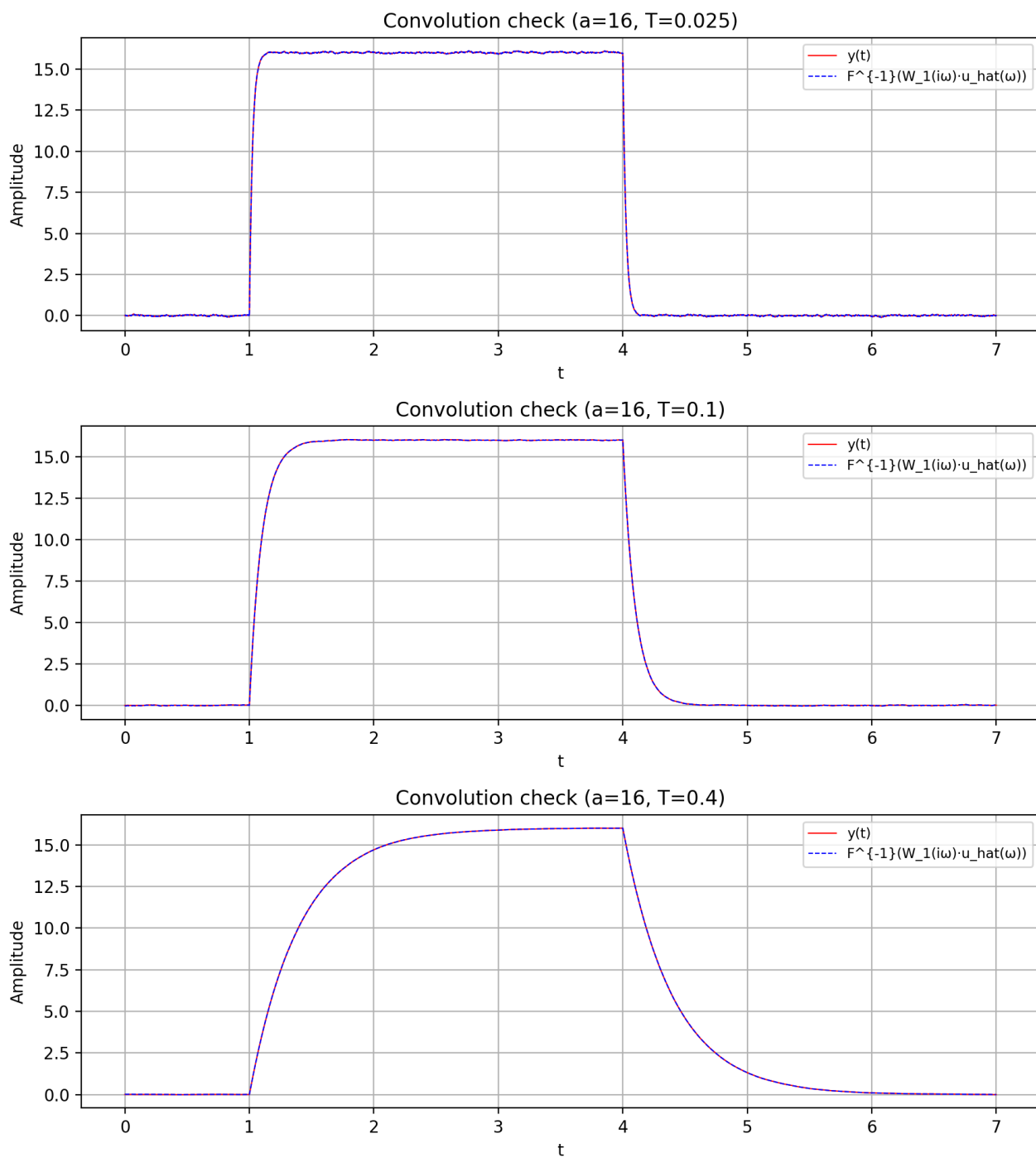


Рисунок 1.10. Временная область

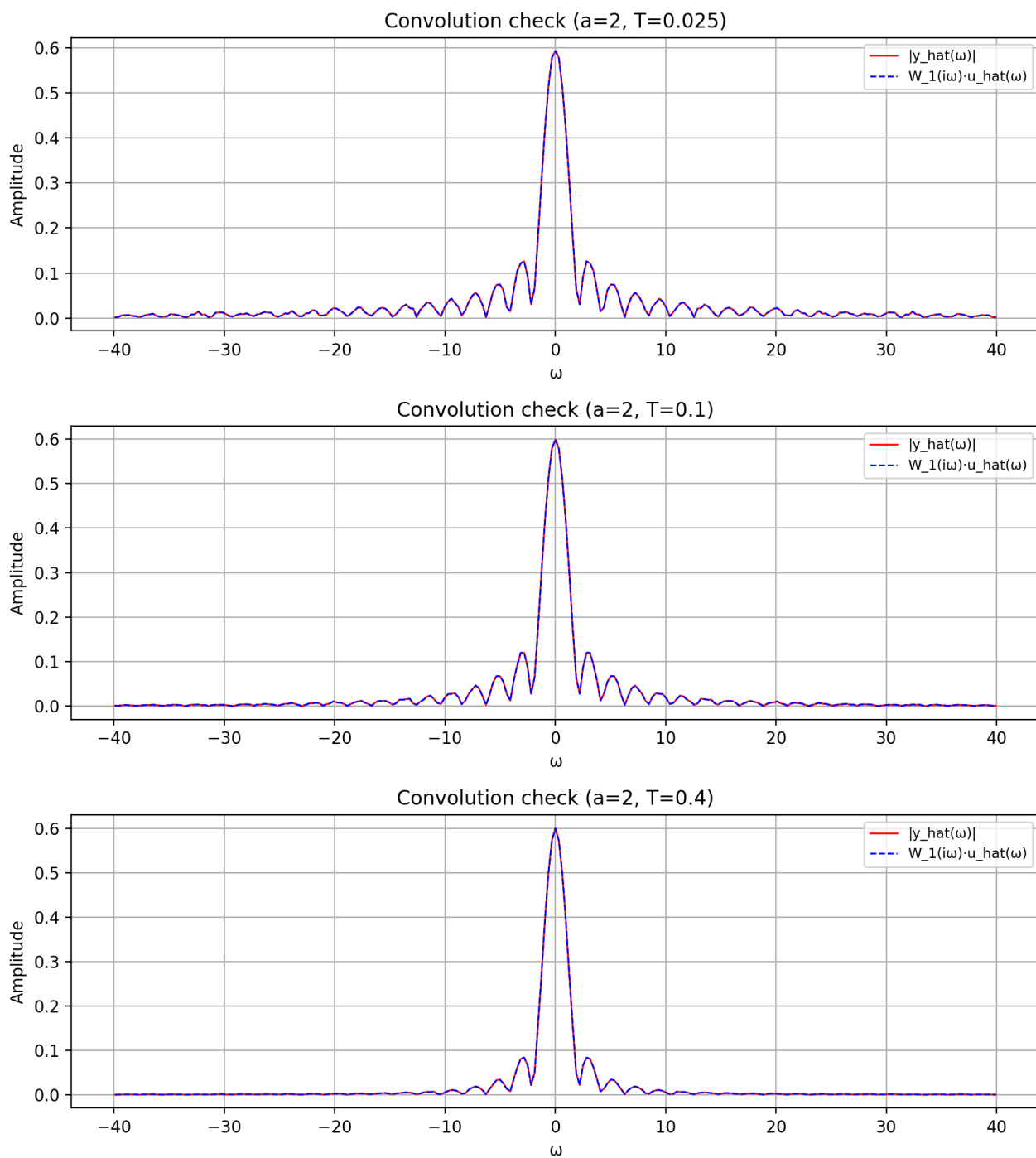


Рисунок 1.11. Частотная область

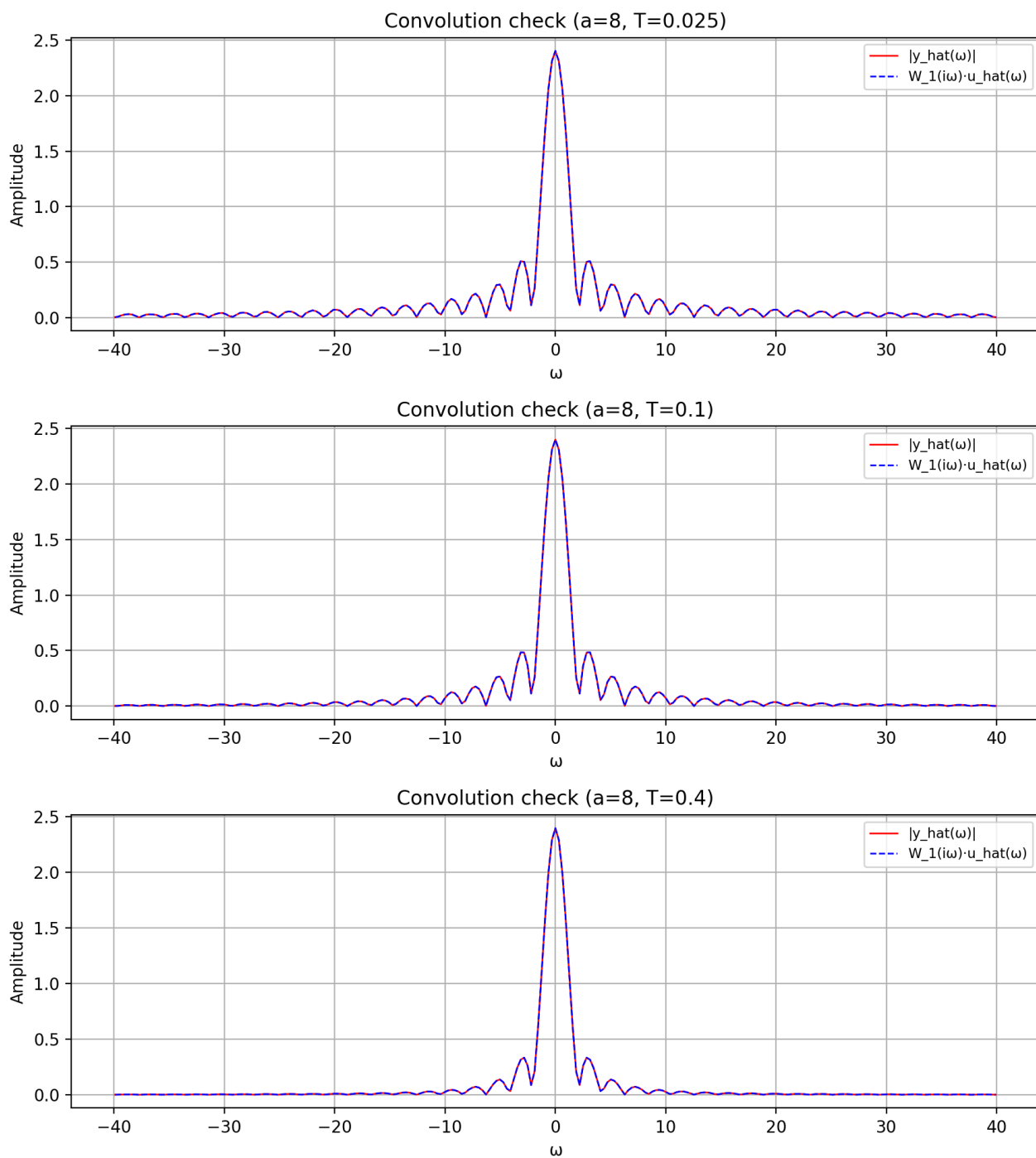


Рисунок 1.12. Частотная область

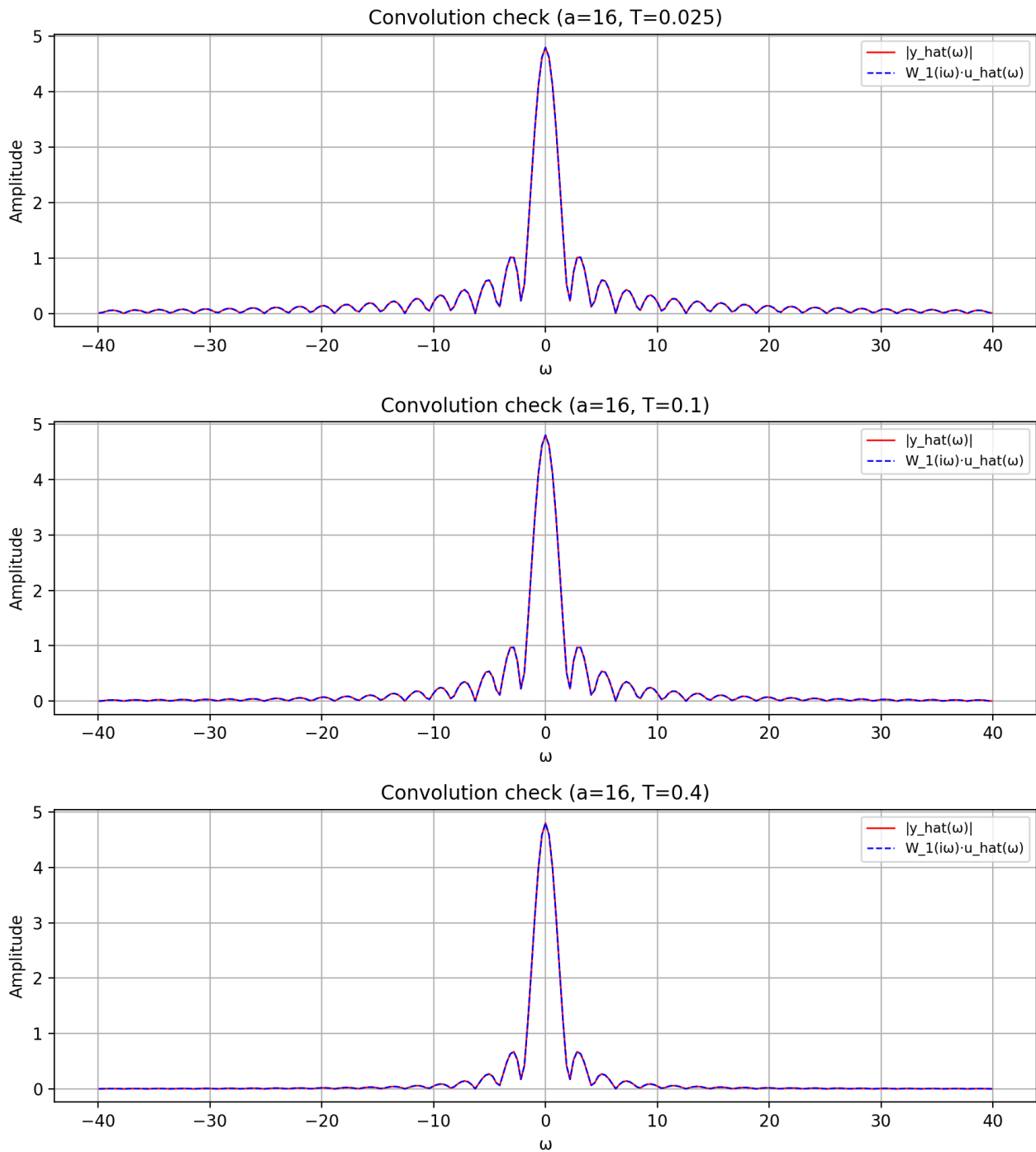


Рисунок 1.13. Частотная область

Выводы

Полученные в результате исследования данные согласуются с теоретическим представлением о динамическом фильтре низких частот.

Параметр T тем меньше можно выбирать, чем больше a , а при более близком a к амплитуде шума b (в нашем случае, $b = 0.5$), параметр T приходится увеличивать, иначе в отфильтрованном сигнале сохраняется много помех. Теорема о свёртке выполняется.

1.2. Режекторный полосовой фильтр

Примем $b = 0$. Рассмотрим линейный фильтр вида

$$W_2(p) = \frac{p^2 + a_1p + a_2}{p^2 + b_1p + b_2}.$$

Выберем значения параметров $a_1, a_2, b_1, b_2 \in \mathbb{R}$ исходя из условий:

- Фильтр должен быть устойчивым (корни полинома знаменателя p_1, p_2 должны иметь строго отрицательную вещественную часть).
- Для нижних частот ($\omega \rightarrow 0$) и верхних частот ($\omega \rightarrow \infty$) АЧХ фильтра должна быть равна 1.
- Для некоторой частоты ω_0 АЧХ фильтра должна быть равна 0.

Построим сравнительные графики исходного и фильтрованного сигналов, графики модулей их Фурье-образов, а также АЧХ фильтра. Исследуем влияние параметра c и d на эффективность фильтрации.

Обеспечим выполнимость условий:

- Корни полинома $p^2 + b_1p + b_2$ гарантированно будут иметь отрицательную вещественную часть при $\text{Re}\left(\frac{-b_1 \pm \sqrt{b_1^2 - 4b_2}}{2}\right) < 0$
- При $\omega \rightarrow 0$: $W(i\omega) = \frac{a_2}{b_2} = 1 \Rightarrow a_2 = b_2$
- При $\omega \rightarrow \infty$: $W(i\omega) = \lim_{\omega \rightarrow \infty} \frac{(i\omega)^2 + a_1(i\omega) + a_2}{(i\omega)^2 + b_1(i\omega) + b_2} = \frac{-1 + \frac{a_1}{i\omega} + \frac{a_2}{\omega^2}}{-1 + \frac{b_1}{i\omega} + \frac{b_2}{\omega^2}} \rightarrow 1$, что выполняется автоматически при $a_2 = b_2$
- Для частоты ω_0 АЧХ равна нулю: $|W(i\omega_0)| = 0 \Rightarrow (i\omega_0)^2 + a_1(i\omega_0) + a_2 = 0$. Отсюда получаем систему:

$$\begin{cases} -\omega_0^2 + a_2 = 0 \\ a_1\omega_0 = 0 \end{cases}$$

Так как $\omega_0 \neq 0$, то $a_1 = 0$ и $a_2 = \omega_0^2$.

Рассмотрим следующие параметры:

$$a = 1, \quad c = \{0.1, 0.5\}, \quad d = \{20, 25, 30\}, \quad a_1 = 0, \quad a_2 = b_2 = 625, \quad b_1 = \{5, 20, 70, 200\}$$

Графики сигналов представлены на рис. 1.14 - 1.21, графики Фурье-образов на рис. 1.22 - 1.29, АЧХ фильтра на рис. 1.30.

Анализ

Параметр c влияет на амплитуду гармоники, d - на её частоту. b_1 - на ширину полосы $\Delta\omega$ при $|W(i\omega)| = \frac{1}{\sqrt{2}}$, покажем это:

Рассмотрим передаточную функцию режекторного фильтра второго порядка:

$$W_2(p) = \frac{p^2 + \omega_0^2}{p^2 + b_1p + \omega_0^2}$$

где ω_0 - центральная частота подавления.

Учитывая определение АЧХ фильтра:

$$|W(i\omega)| = \frac{|\omega_0^2 - \omega^2|}{\sqrt{(\omega_0^2 - \omega^2)^2 + (b_1\omega)^2}}$$

Получаем:

$$\frac{(\omega_0^2 - \omega^2)^2}{(\omega_0^2 - \omega^2)^2 + (b_1\omega)^2} = \frac{1}{2}; \quad (\omega_0^2 - \omega^2)^2 = (b_1\omega)^2; \quad \omega_0^2 - \omega^2 = \pm b_1\omega$$

$$\omega_{\pm} = \frac{\pm b_1 + \sqrt{b_1^2 + 4\omega_0^2}}{2} \quad (\text{для } \omega > 0)$$

Разность граничных частот дает искомую ширину полосы:

$$\Delta\omega = \omega_+ - \omega_- = b_1$$

Данный фильтр лучше всего подавляет гармонику в том случае, когда её частота совпадает с ω_0 , что согласуется с теоретическими ожиданиями: на этой частоте АЧХ фильтра зануляется. Чем больше разница $|d - \omega_0|$, тем меньше гармоника подавляется. Причем из графика АЧХ видно, что при удалении d от ω_0 подавление происходит сильнее, если d находится в области более высоких частот, чем ω_0 . В то же время, при увеличении b_1 область, в которой гармоника подавляется эффективно, увеличивается, и поэтому возрастает разница $|d - \omega_0|$, при которой всё ещё получится хорошо приблизиться к исходному сигналу после фильтрации. Однако, при увеличении b_1 растёт и запаздывание сигнала (аналогичное явление мы наблюдали в предыдущем пункте), поэтому увеличивать b_1 очень сильно тоже не получится. На графике АЧХ это довольно наглядно, чем шире окно $\Delta\omega = b_1$, тем у большего количества частот "режется" амплитуда.

При увеличении амплитуды гармоники сглаживание происходит немного хуже, но при совпадении $d = \omega_0$ данный параметр почти не влияет.

Так же замечен эффект Гибса, который усиливается при увеличении b_1 .

На графиках 1.31 - 1.46 представлена проверка теоремы о свёртке для временной и частотной области. Эксперимент подтвердил теоретические ожидания. Так же мы можем заметить, что для данного сигнала графики в некоторой степени расходятся в области около нуля (если точнее, в начальный момент времени и некоторое малое время спустя него), что вызвано особенностью того, как считается функция `lsim` (начальными условиями для неё). В следующем задании мы явно зададим начальные условия для того, чтобы скорректировать данную погрешность и увидеть, как с ней можно бороться.

Выводы

Полученные в результате исследования данные согласуются с теоретическим представлением о динамическом режекторном полосовом фильтре.

Лучшая фильтрация происходит при $d = \omega_0$ для любого b_1 - по сути, фильтровать именно эту частоту и есть задача данного фильтра. При увеличении b_1 до некоторых значений мы можем эффективно подавлять большую полосу частот. При слишком сильном увеличении данного параметра фильтрованный сигнал начинает сильно запаздывать. Параметр c также оказывает влияние на качество фильтрации.

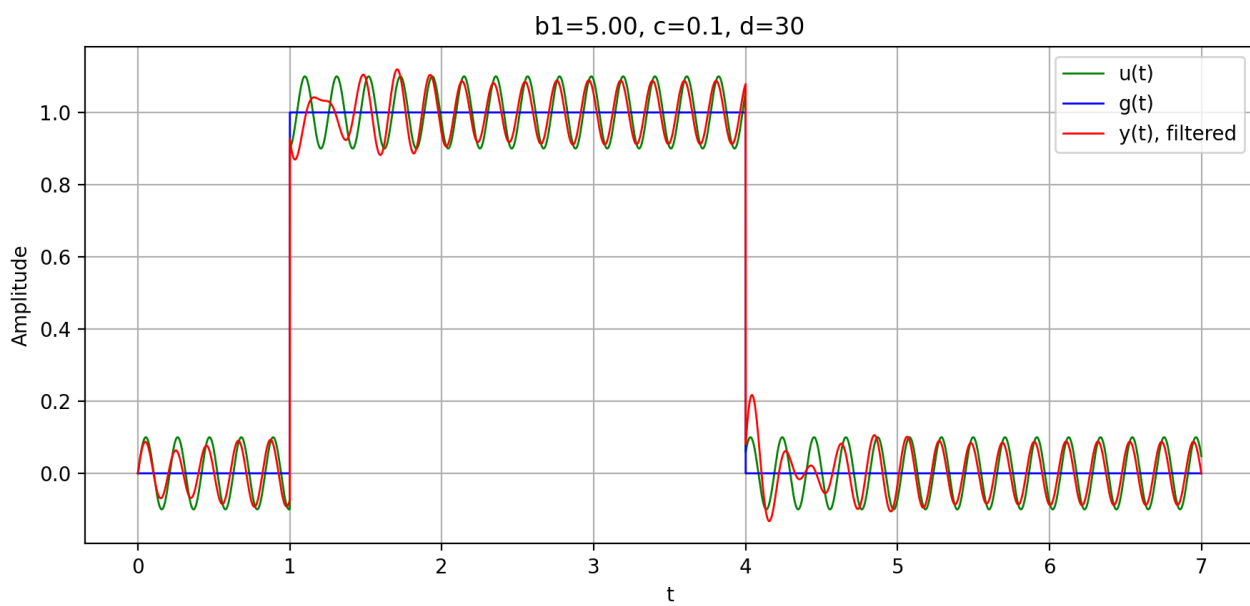
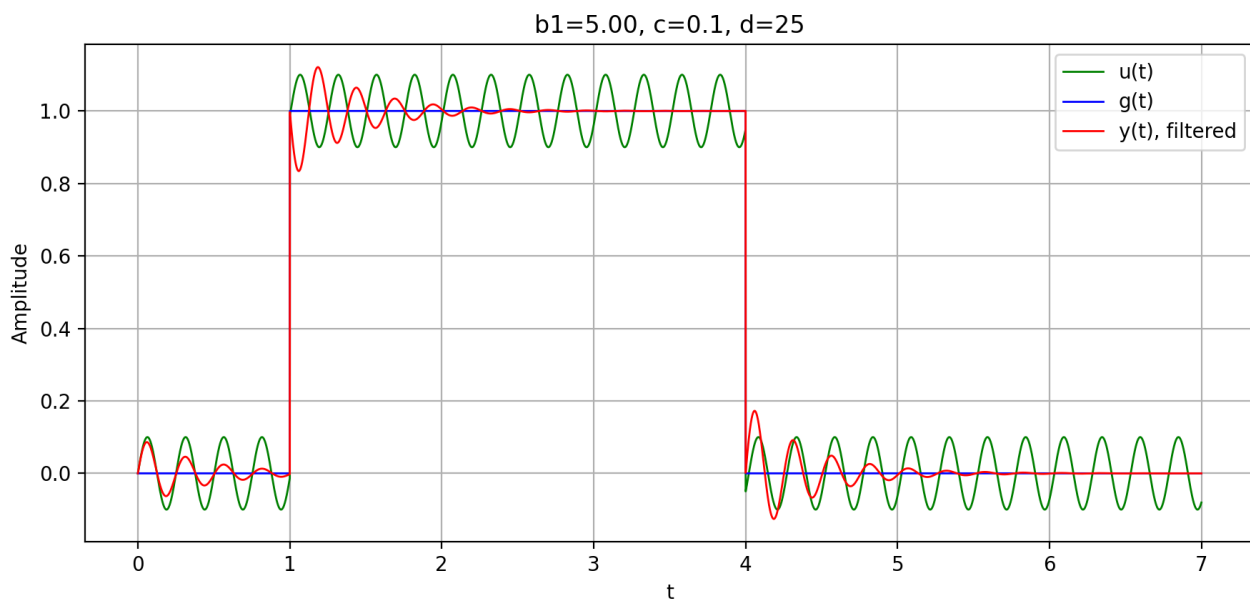
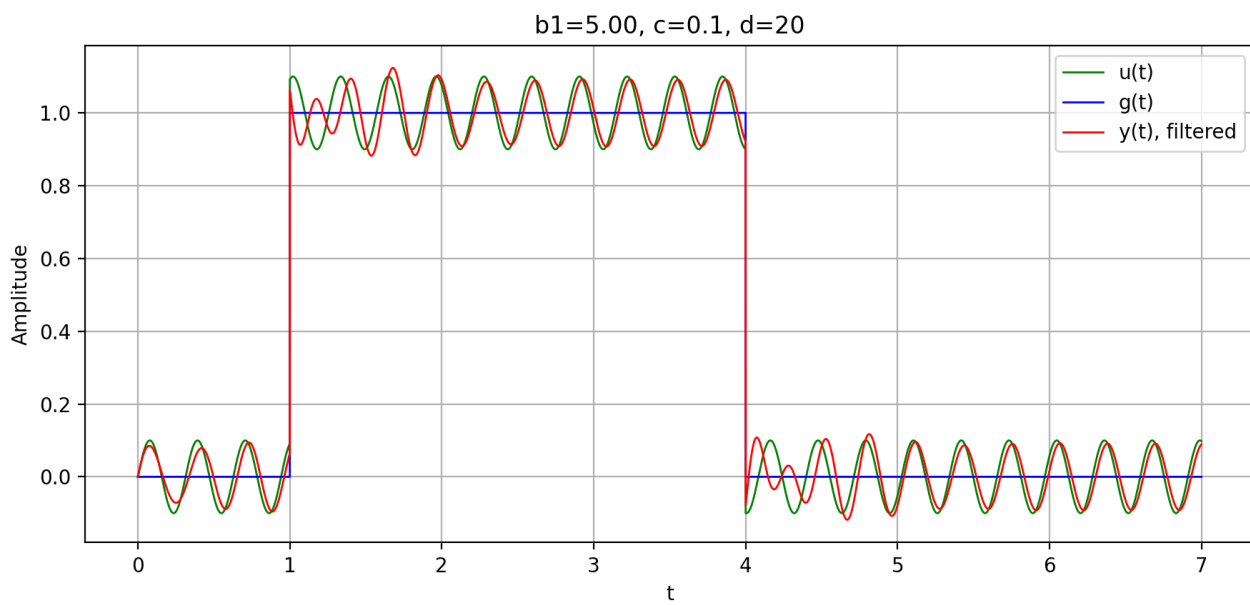


Рисунок 1.14. Сигнал

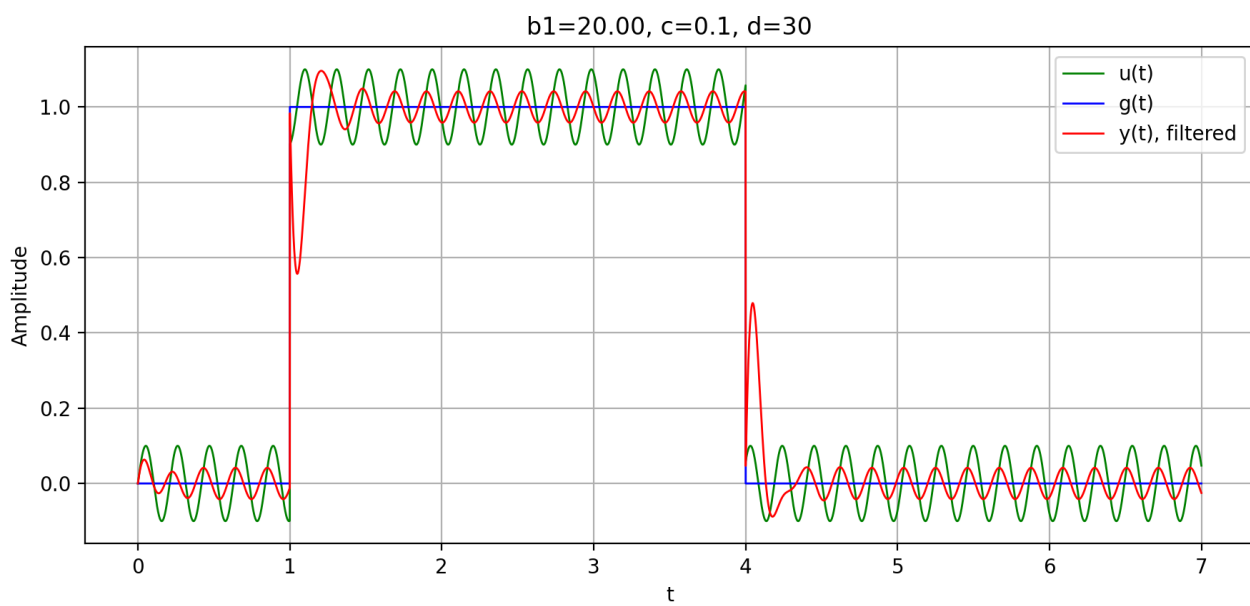
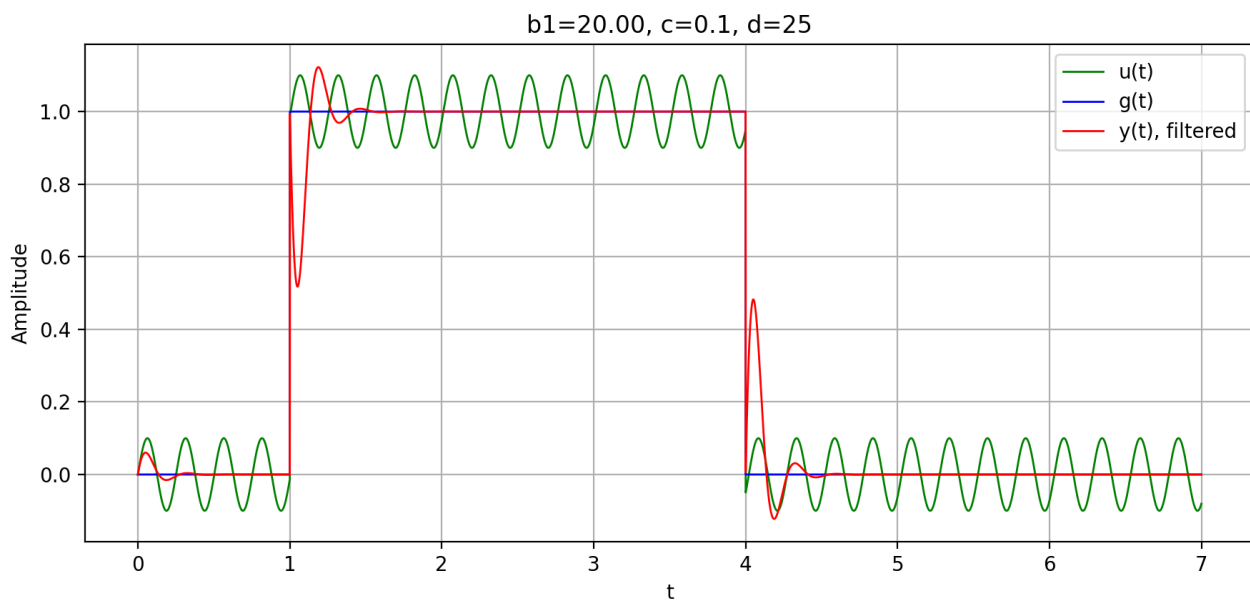
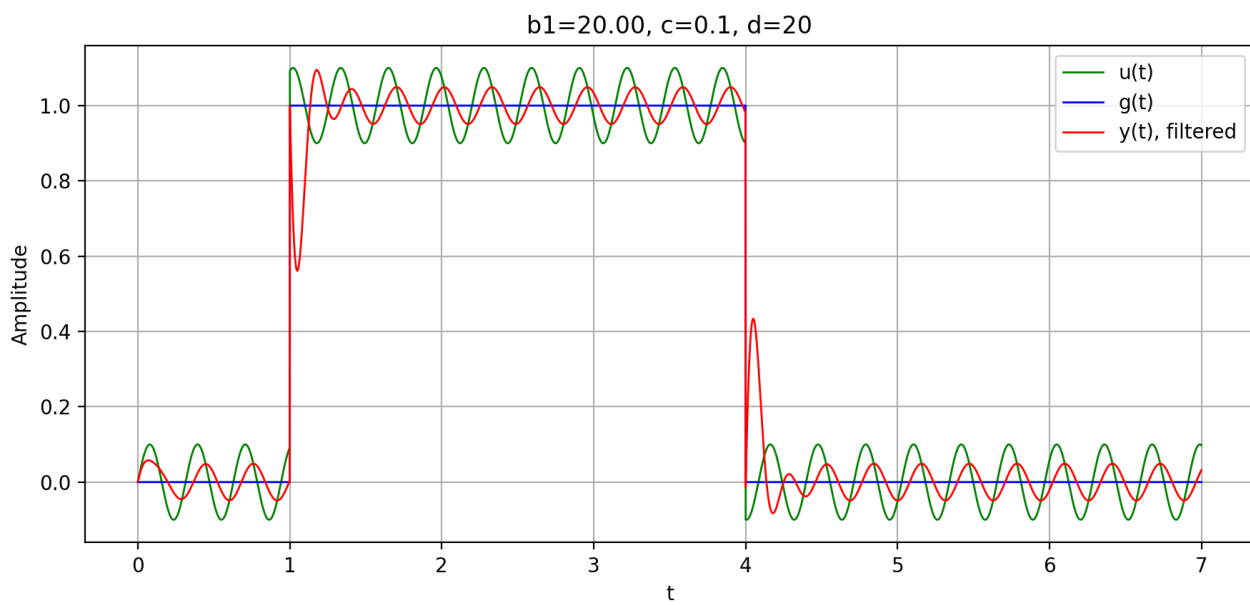


Рисунок 1.15. Сигнал

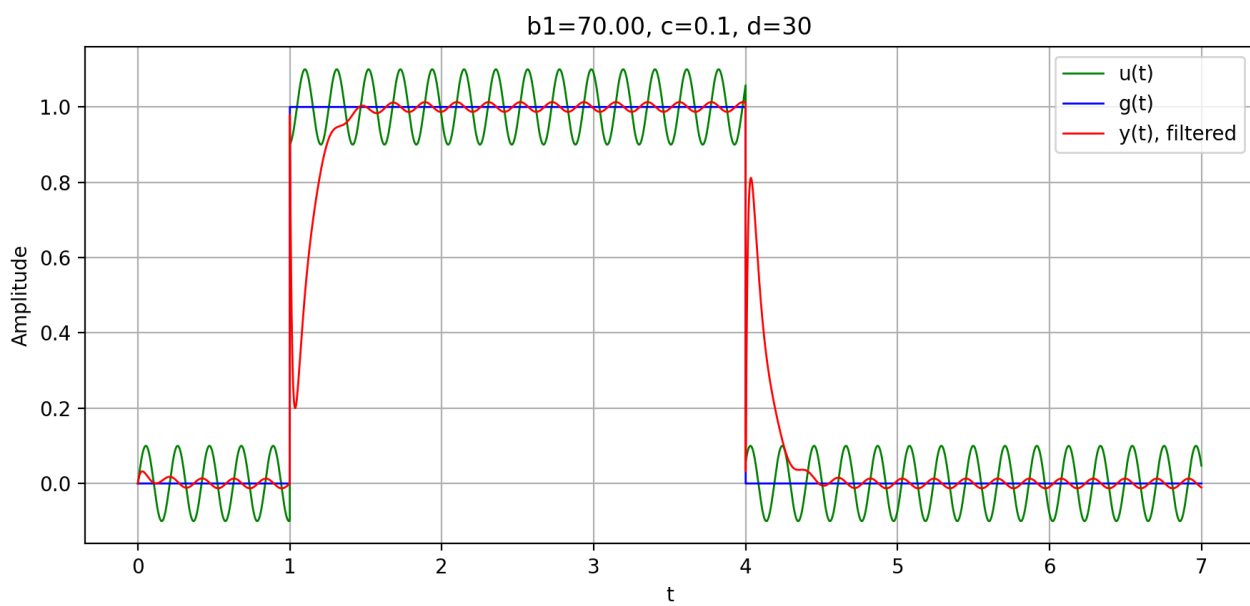
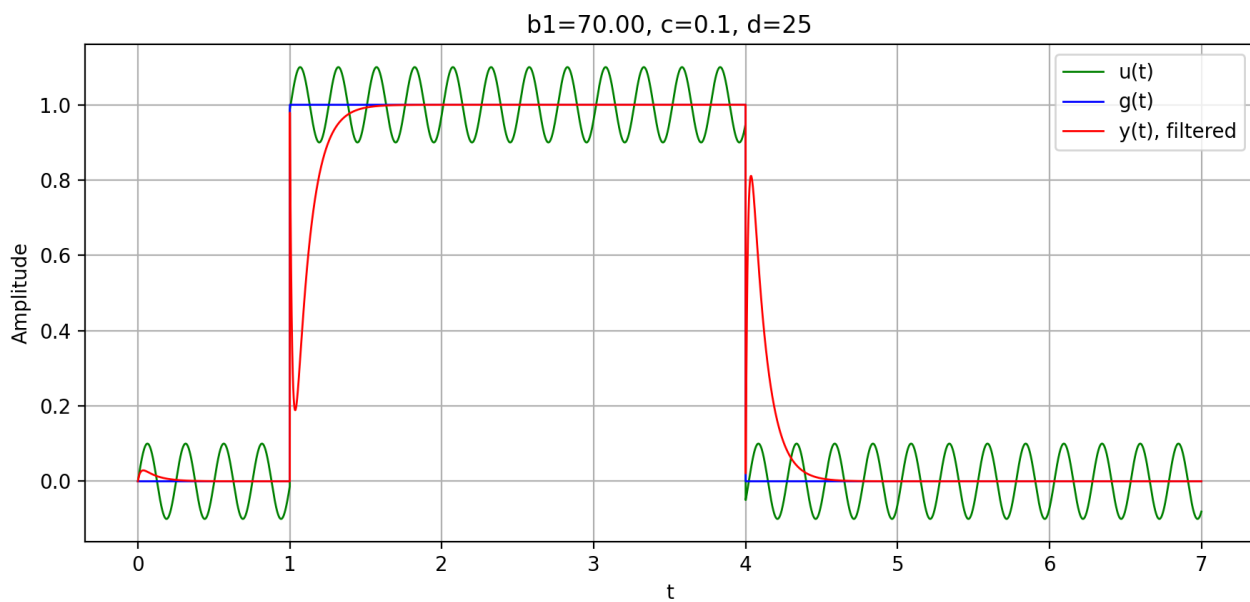
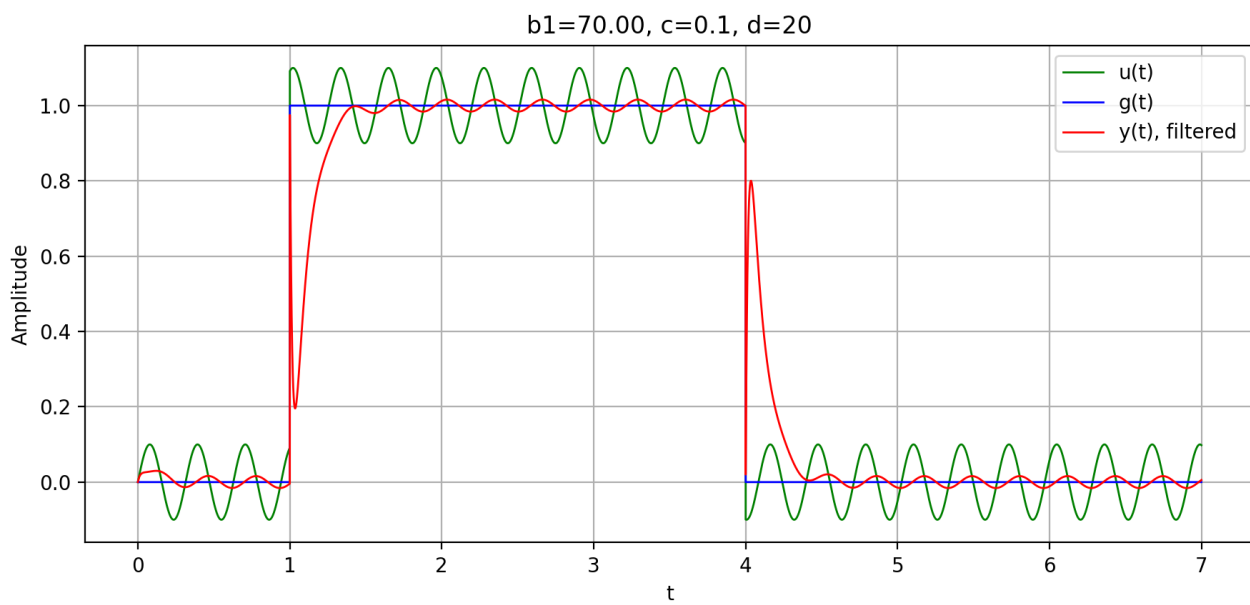


Рисунок 1.16. Сигнал

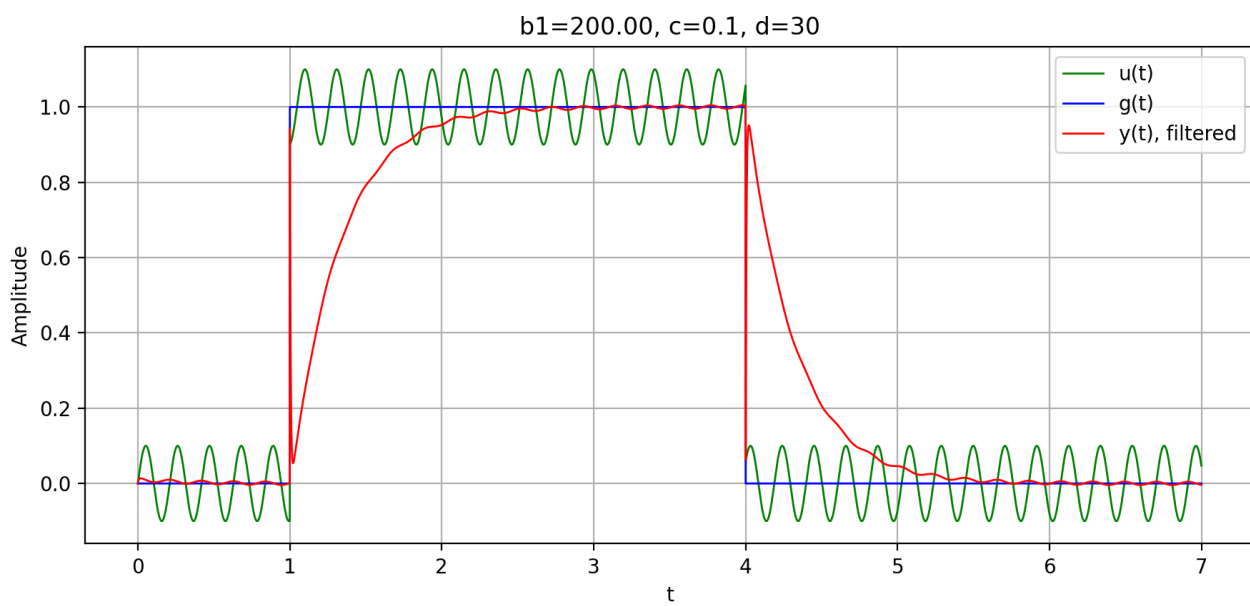
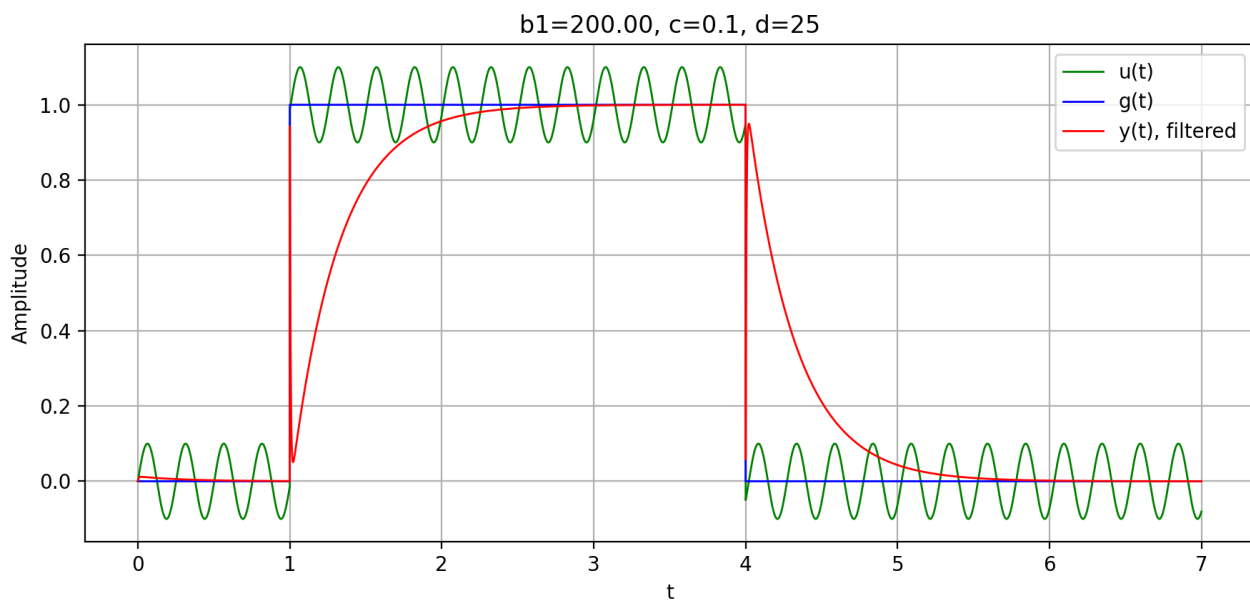
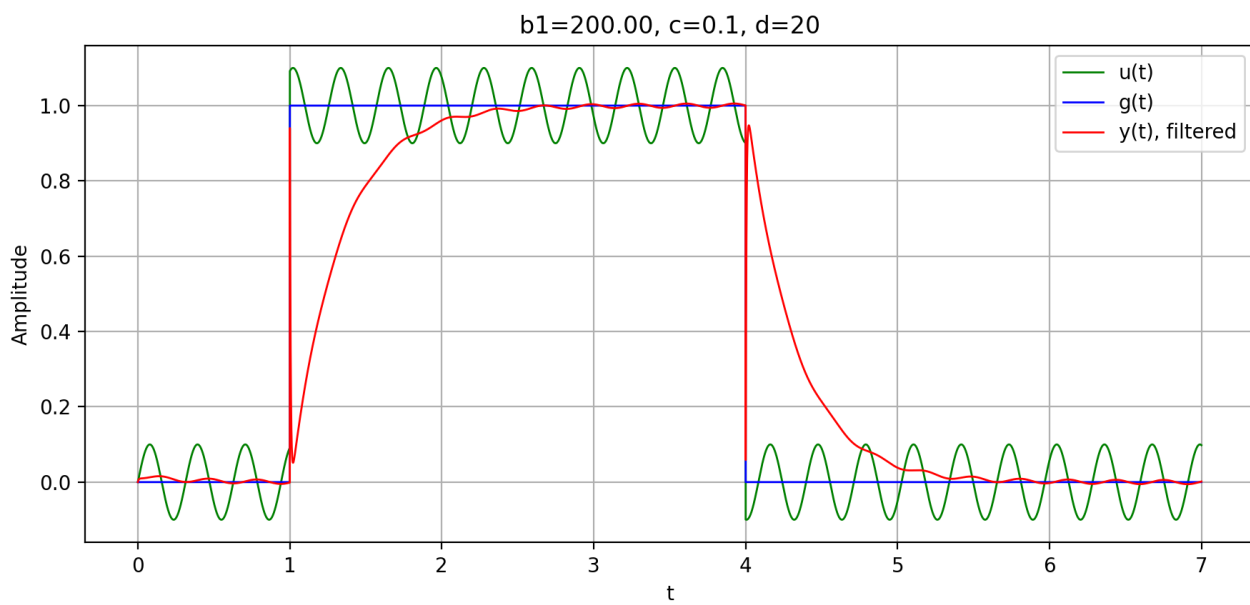


Рисунок 1.17. Сигнал

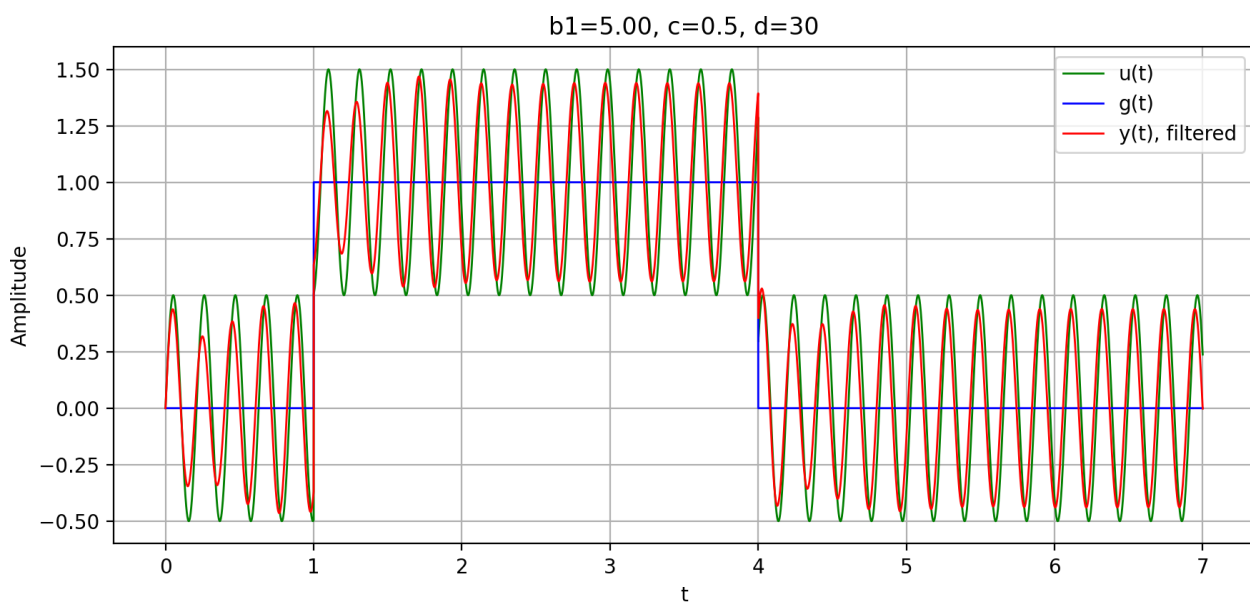
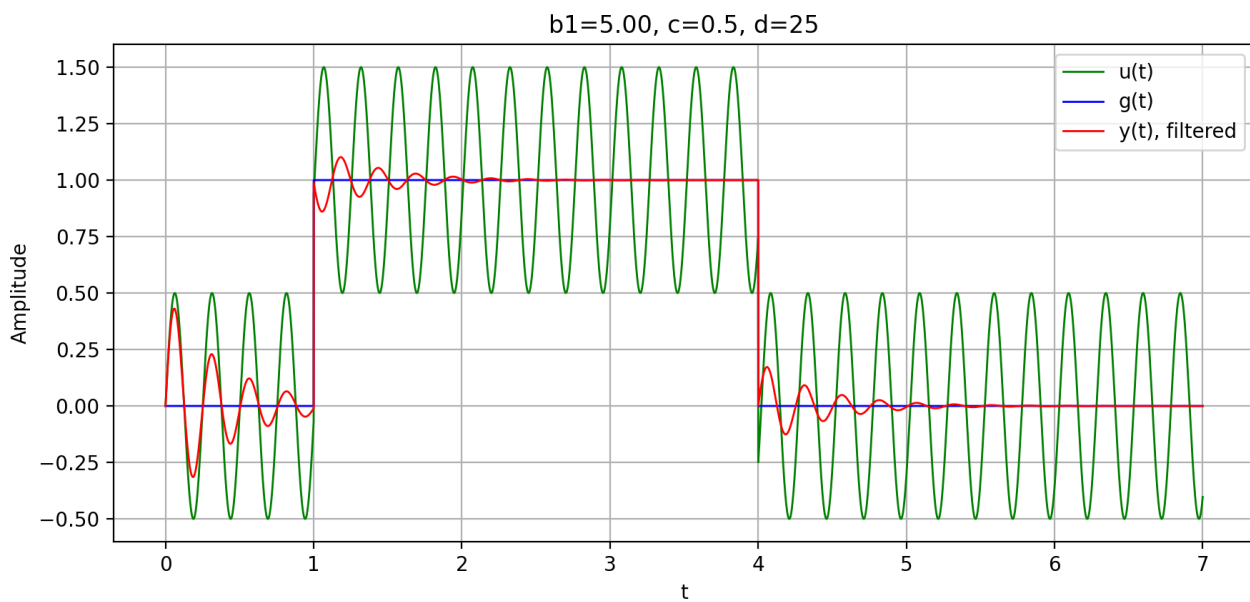
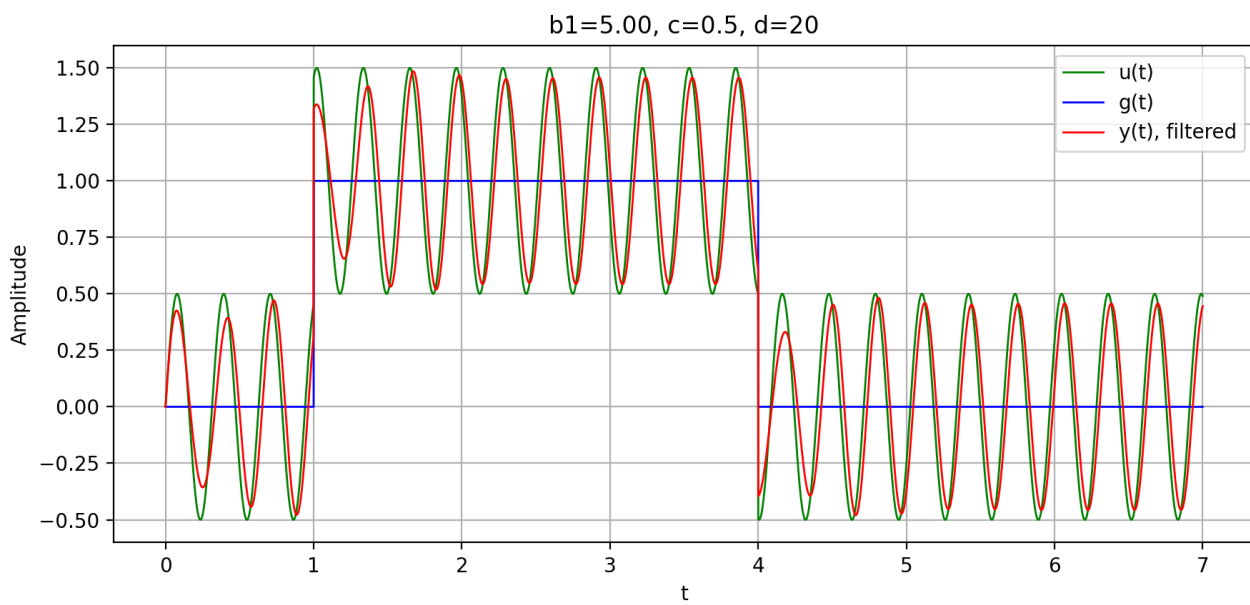


Рисунок 1.18. Сигнал

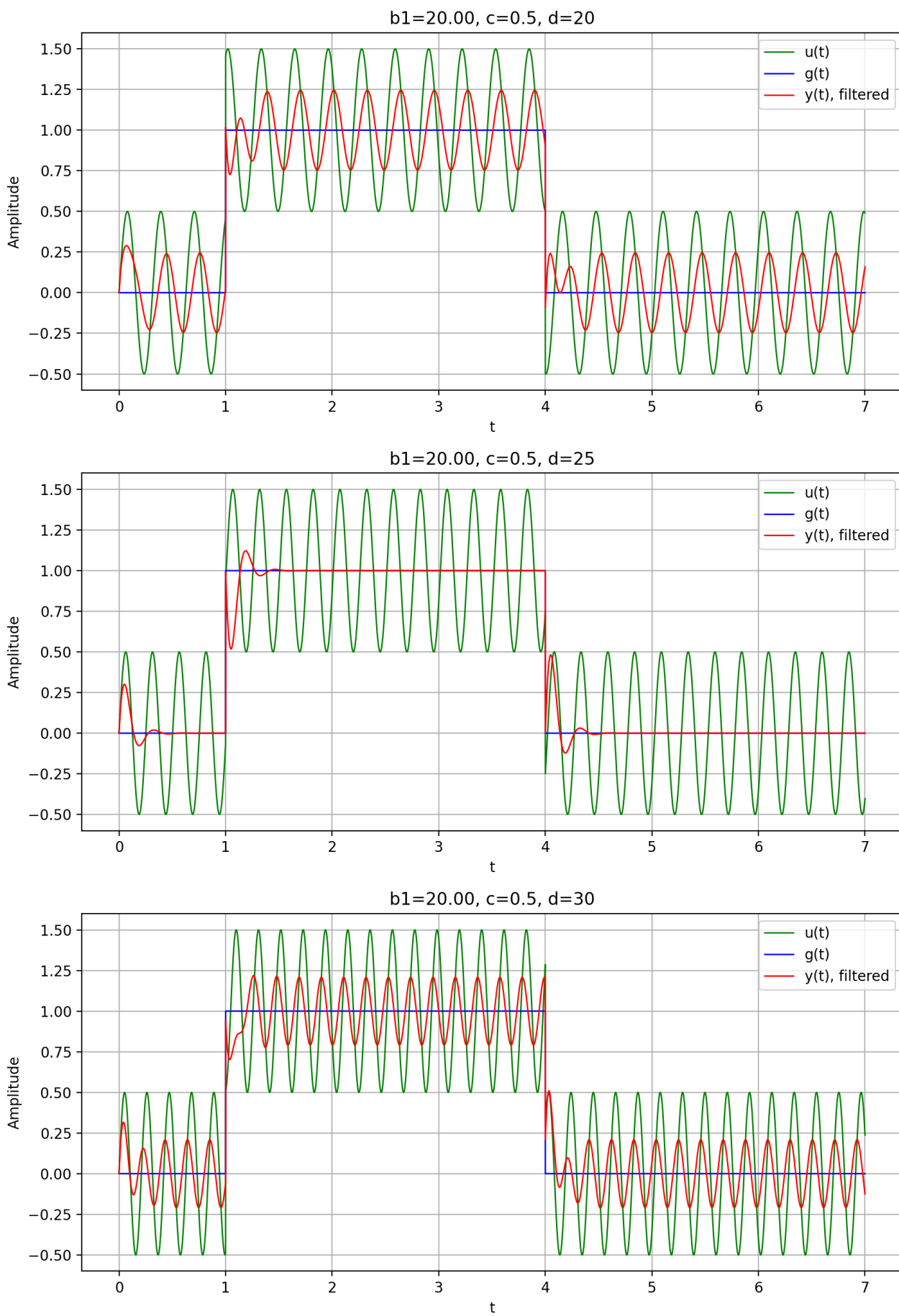


Рисунок 1.19. Сигнал

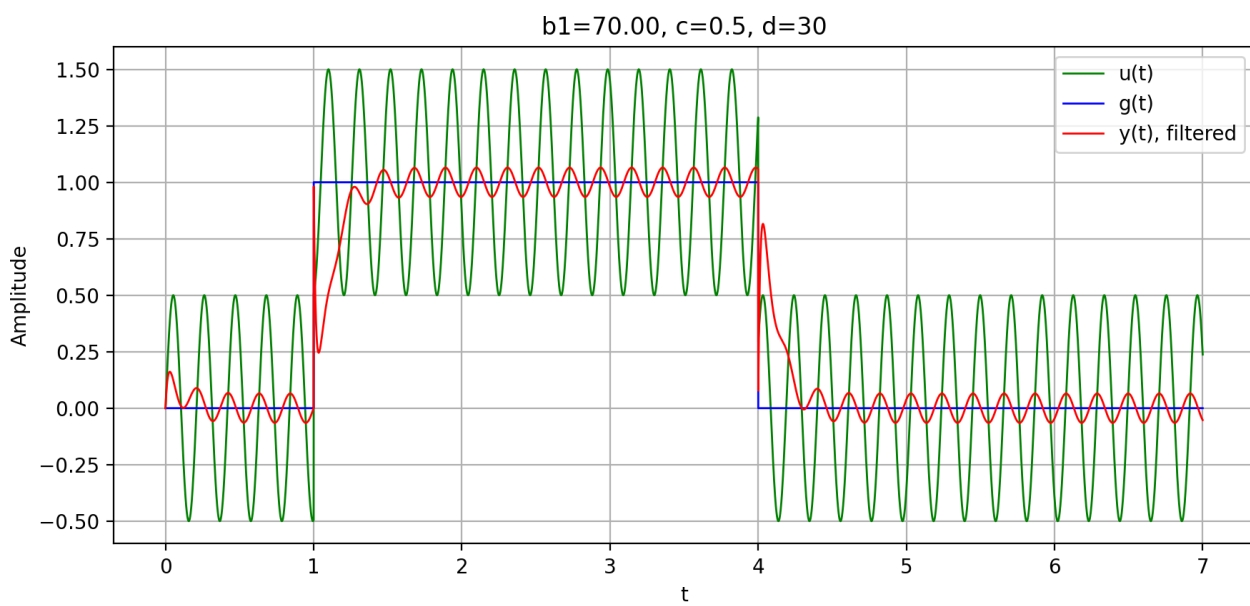
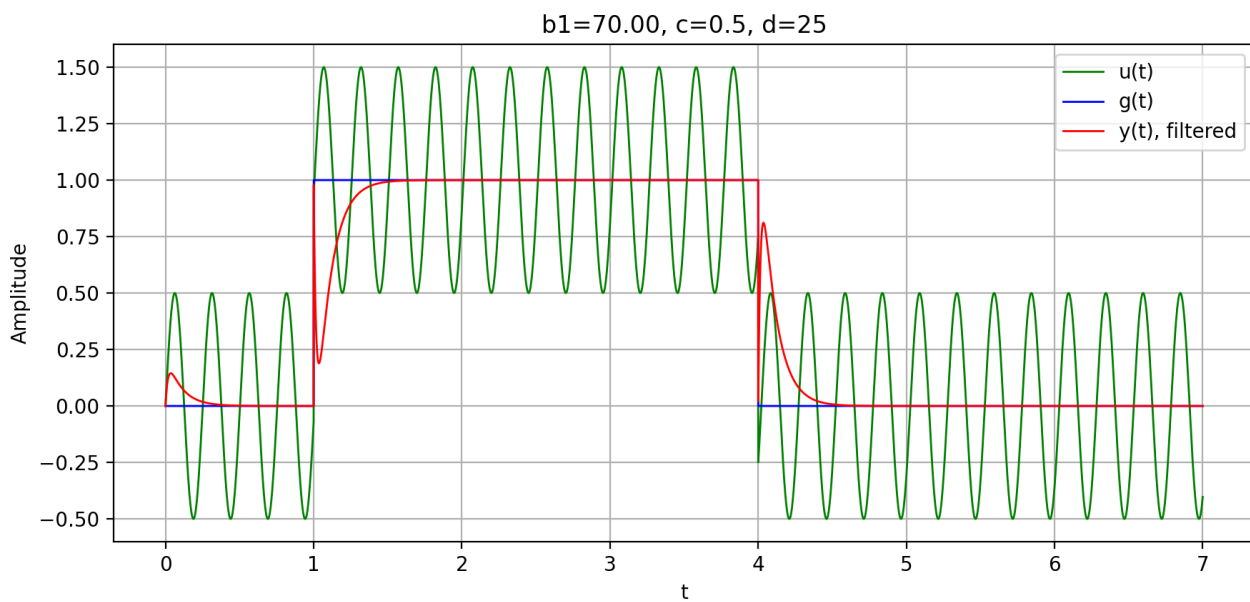
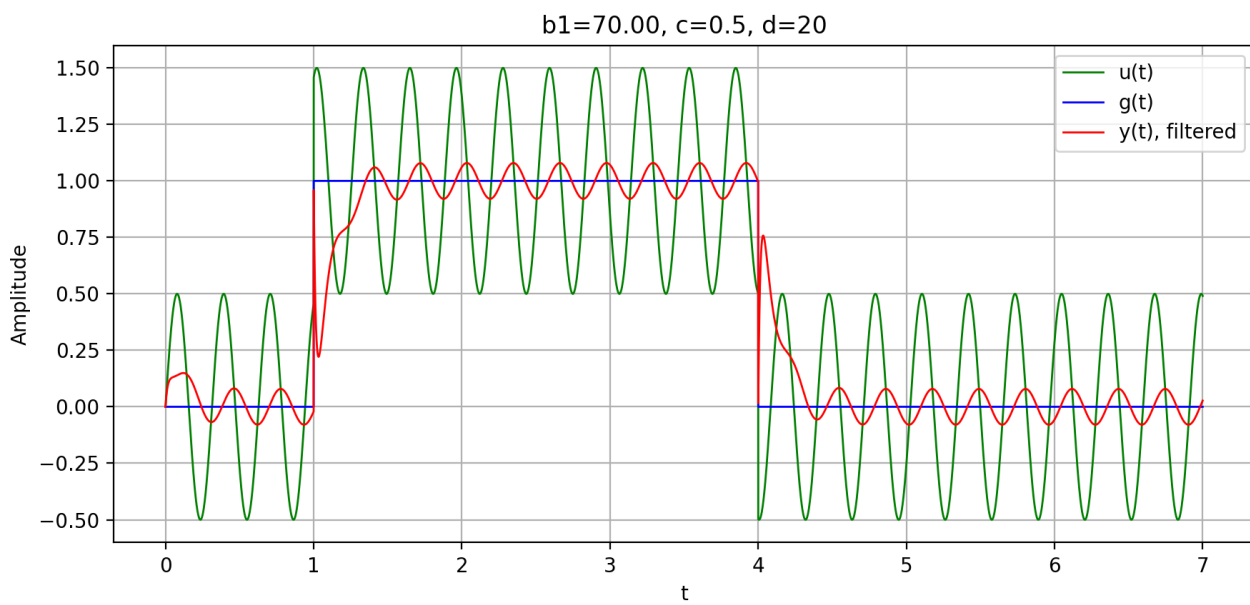


Рисунок 1.20. Сигнал

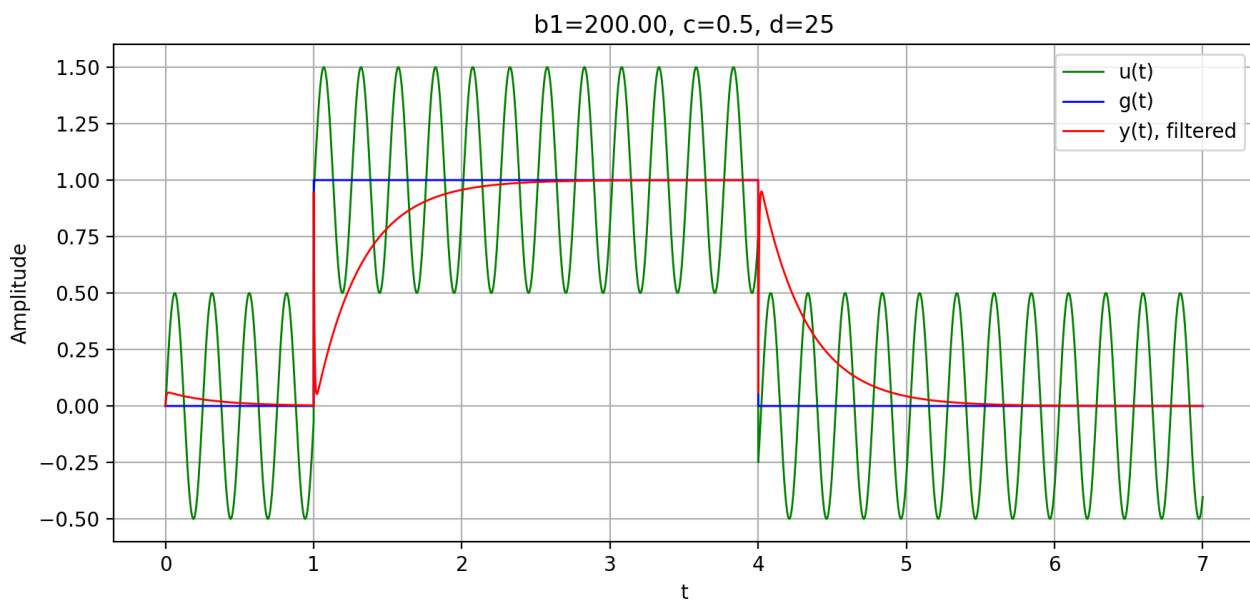
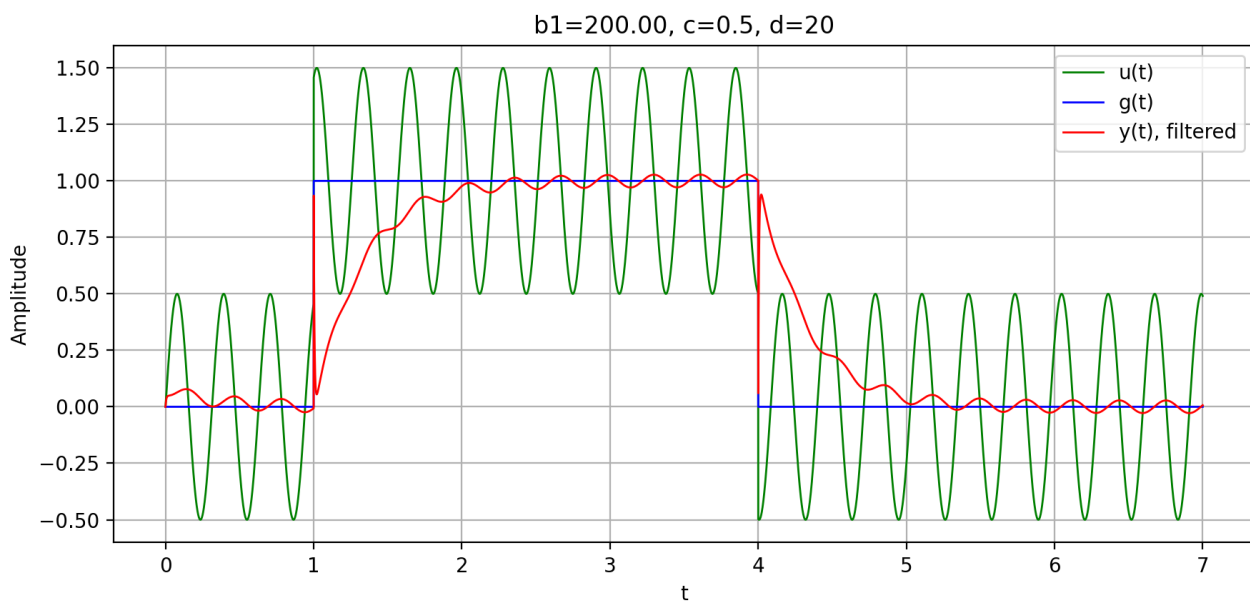


Рисунок 1.21. Сигнал

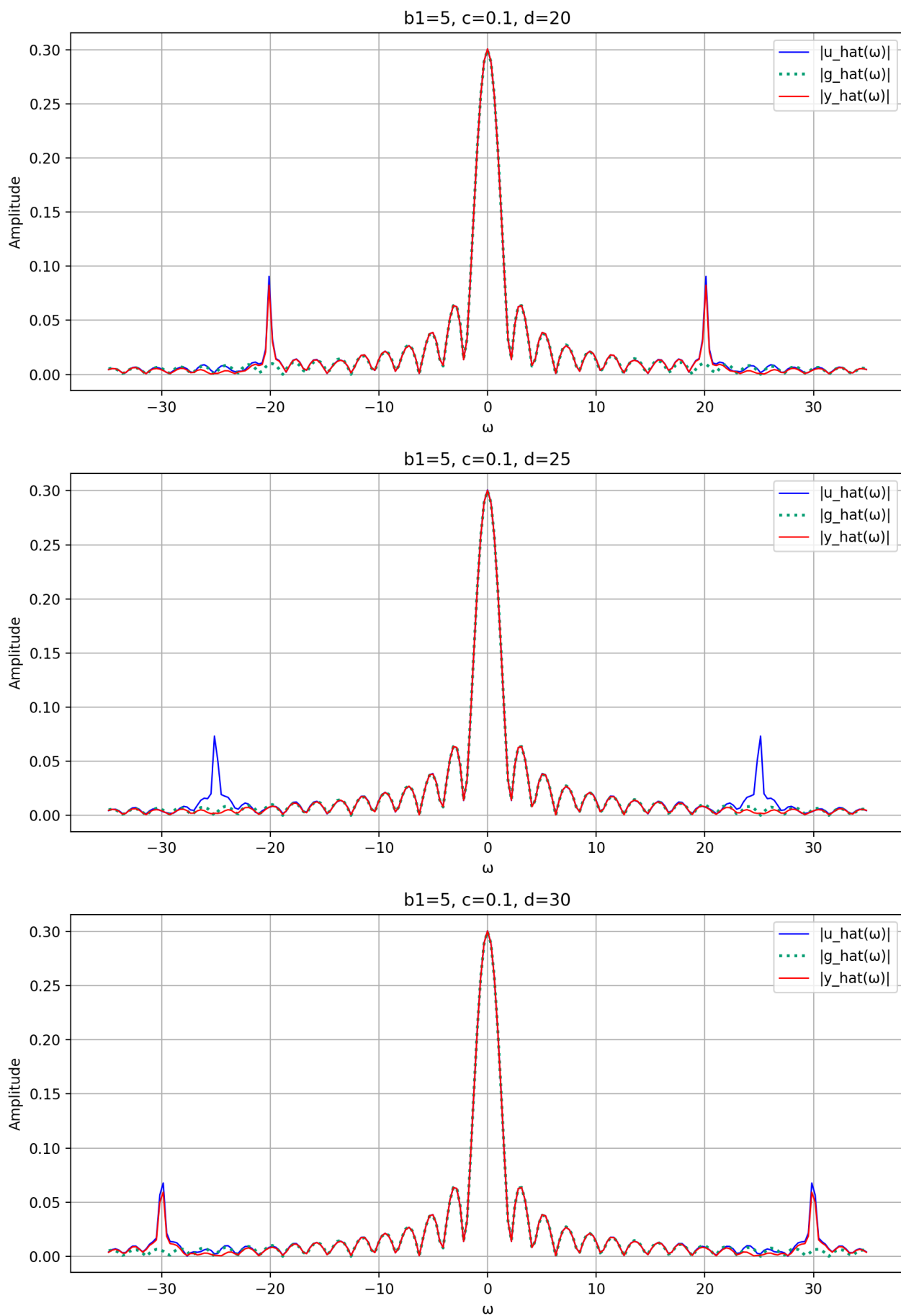


Рисунок 1.22. Фурье-образ

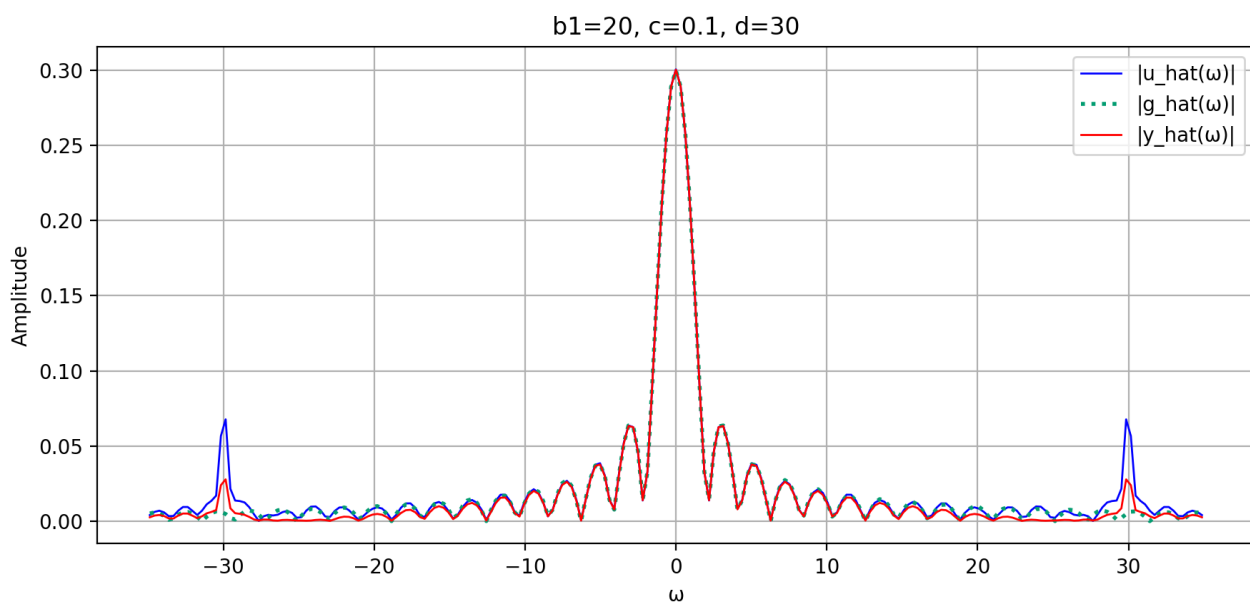
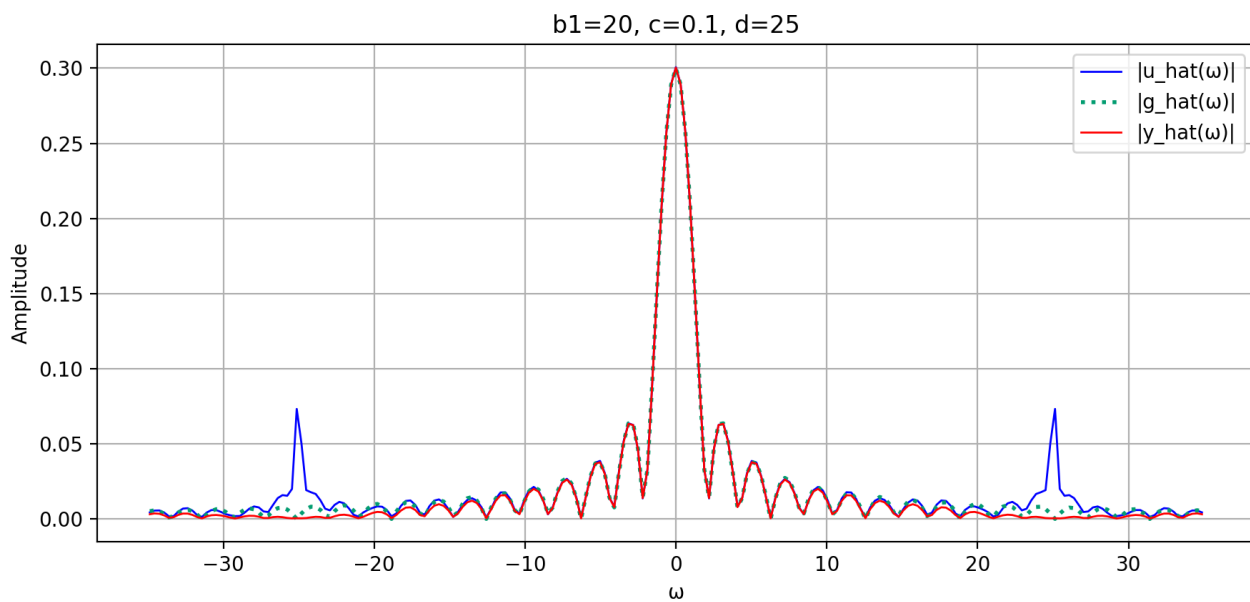
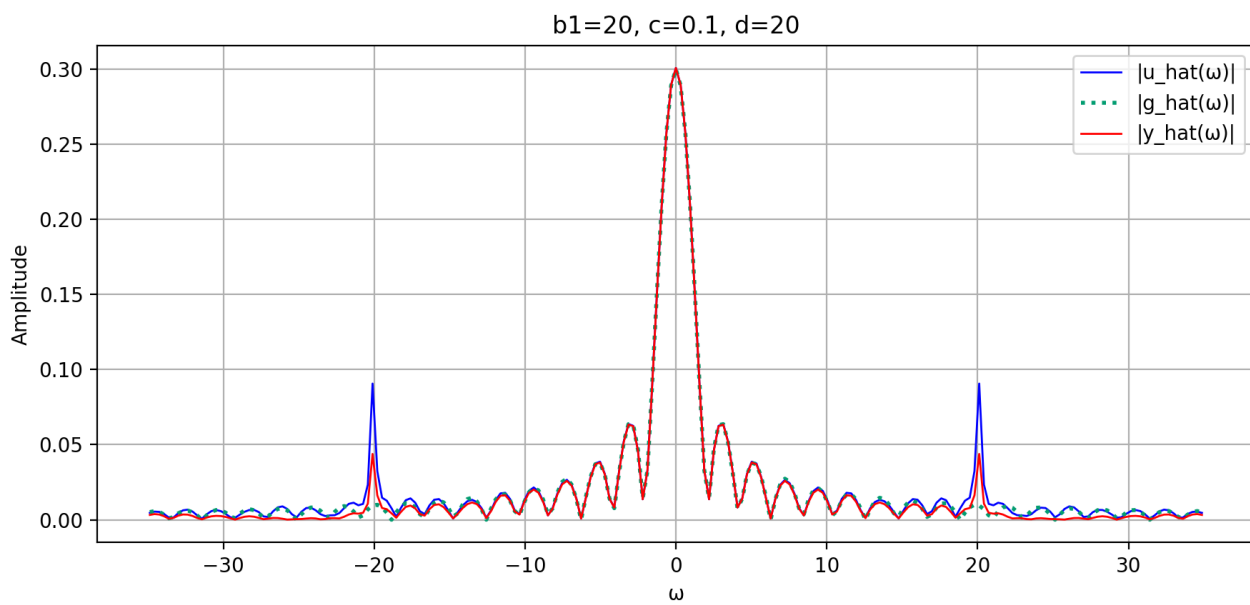


Рисунок 1.23. Фурье-образ

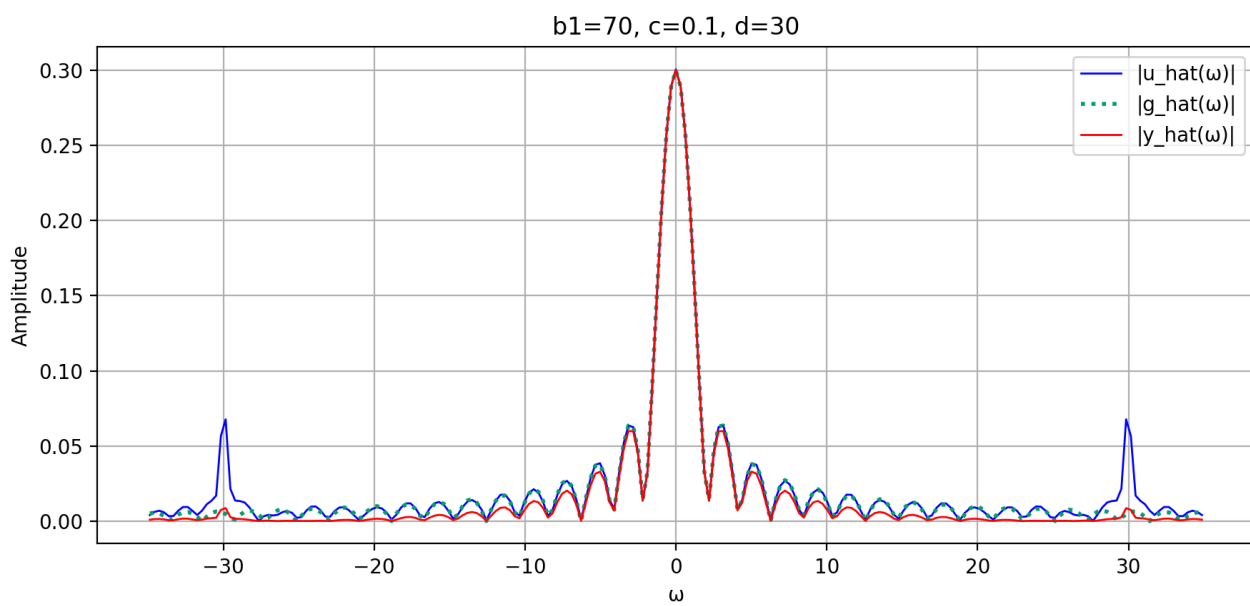
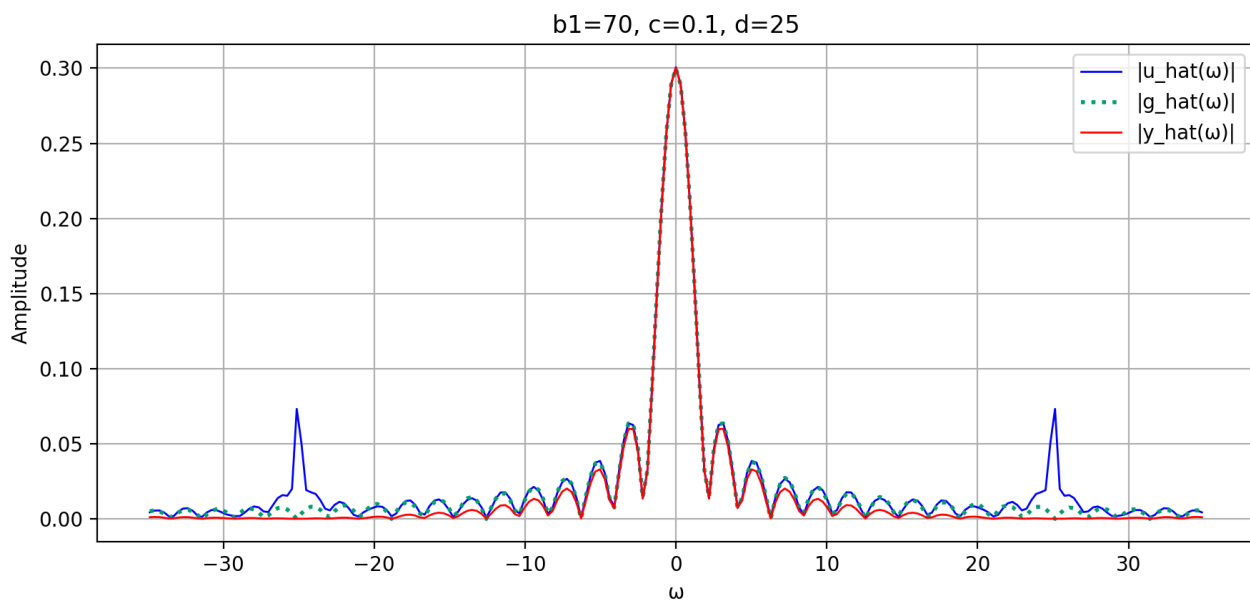
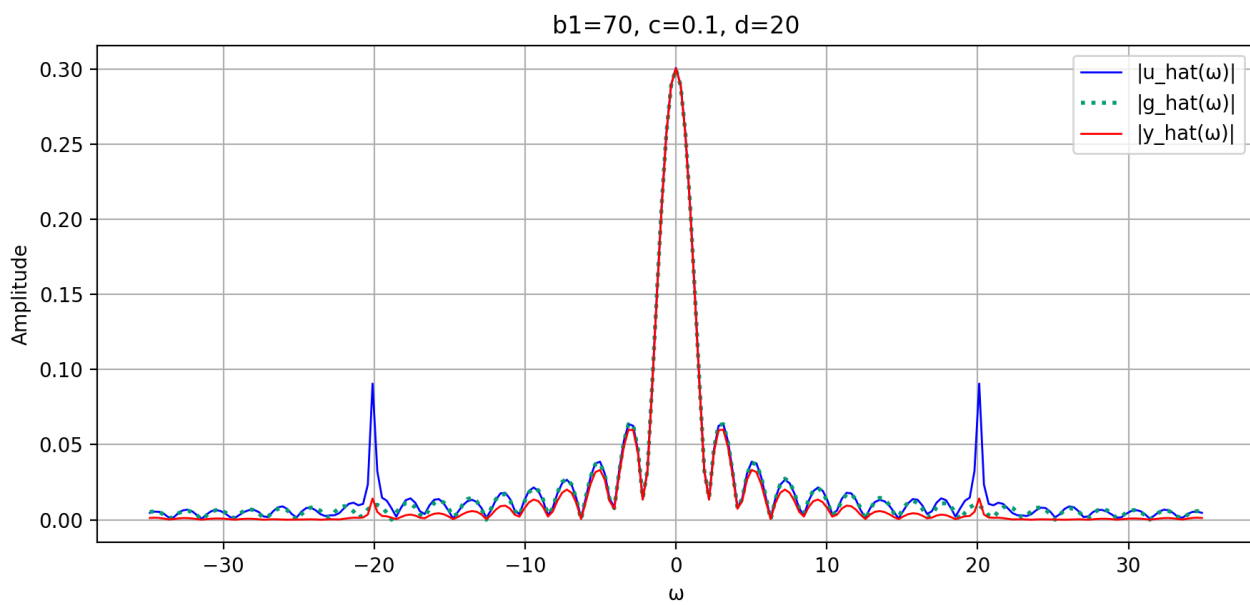


Рисунок 1.24. Фурье-образ

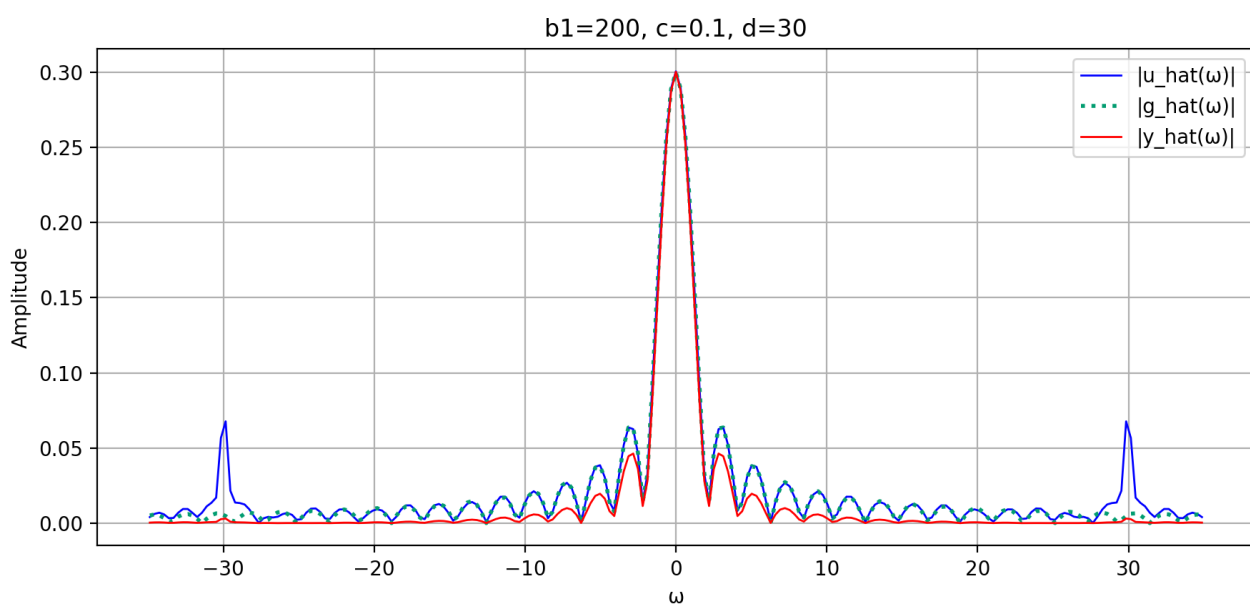
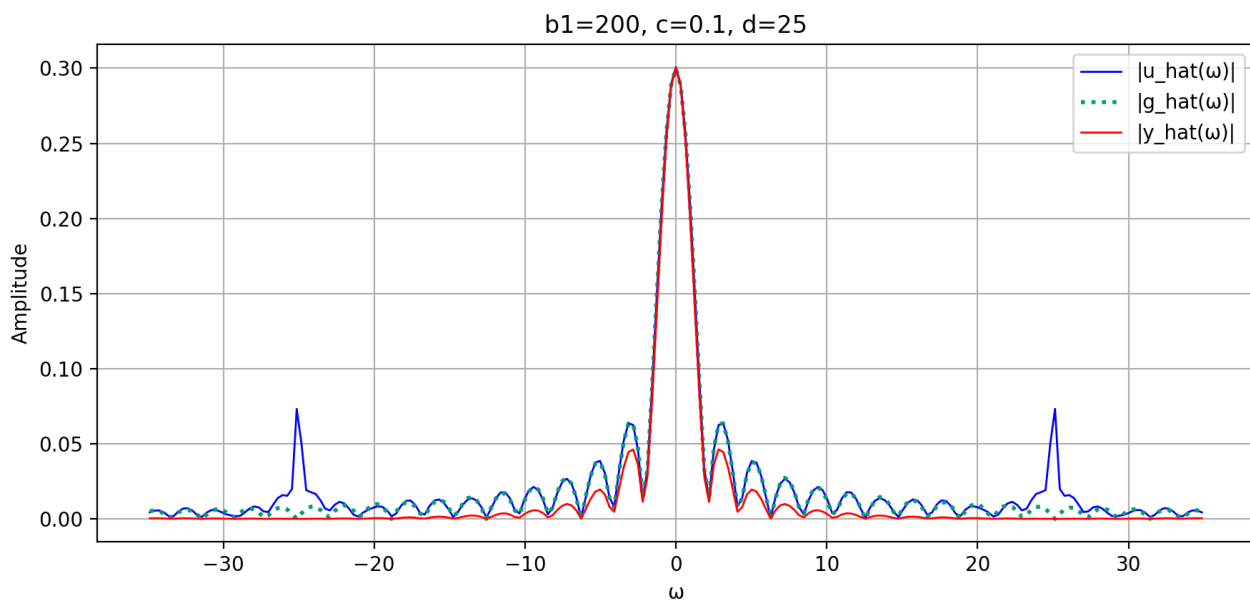
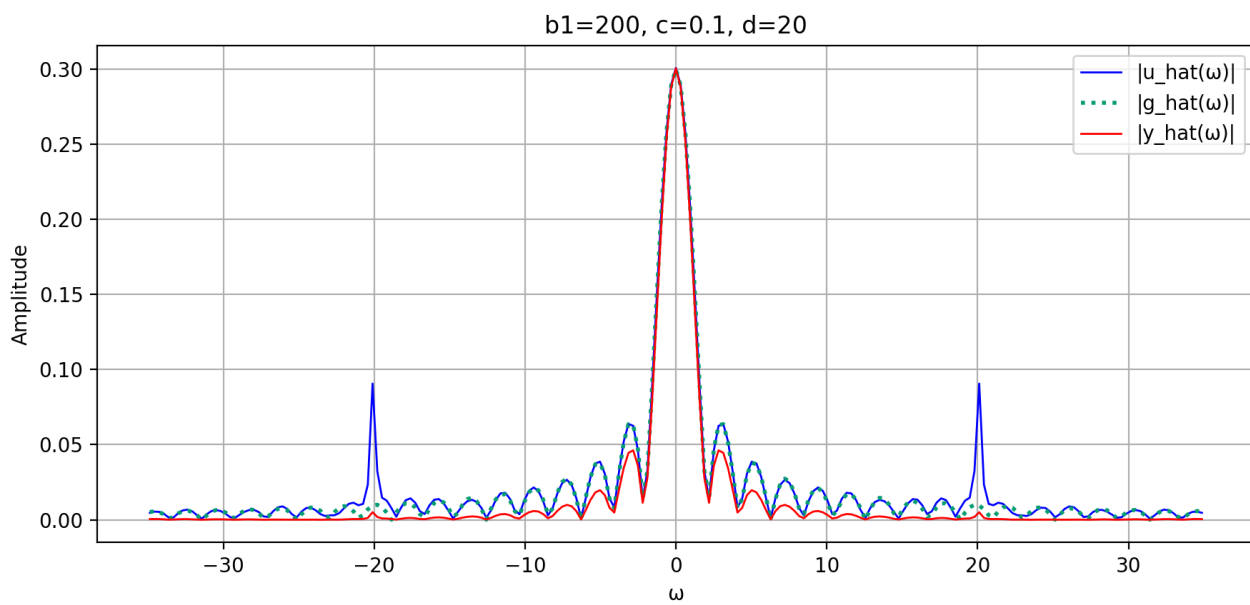


Рисунок 1.25. Фурье-образ

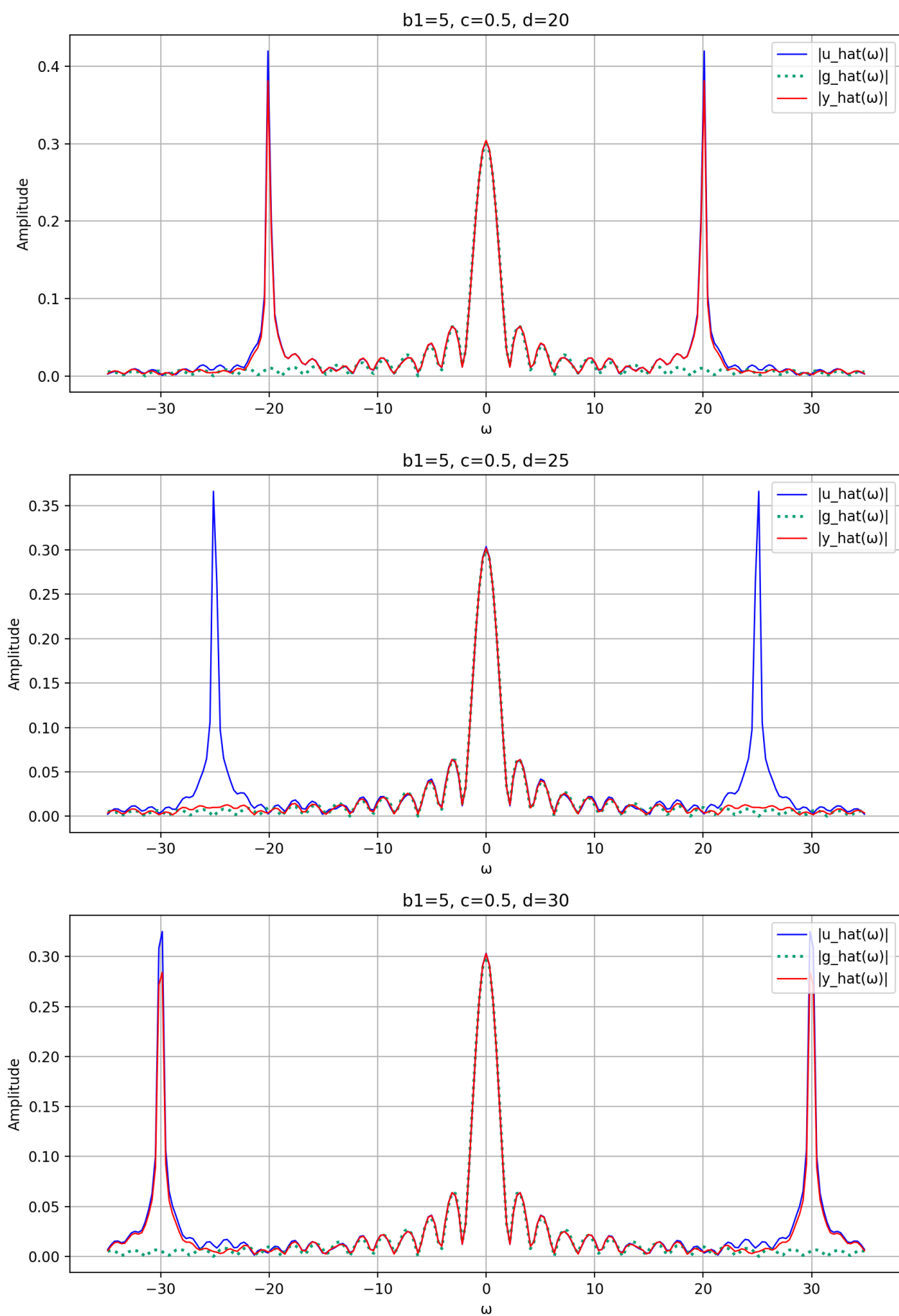


Рисунок 1.26. Фурье-образ

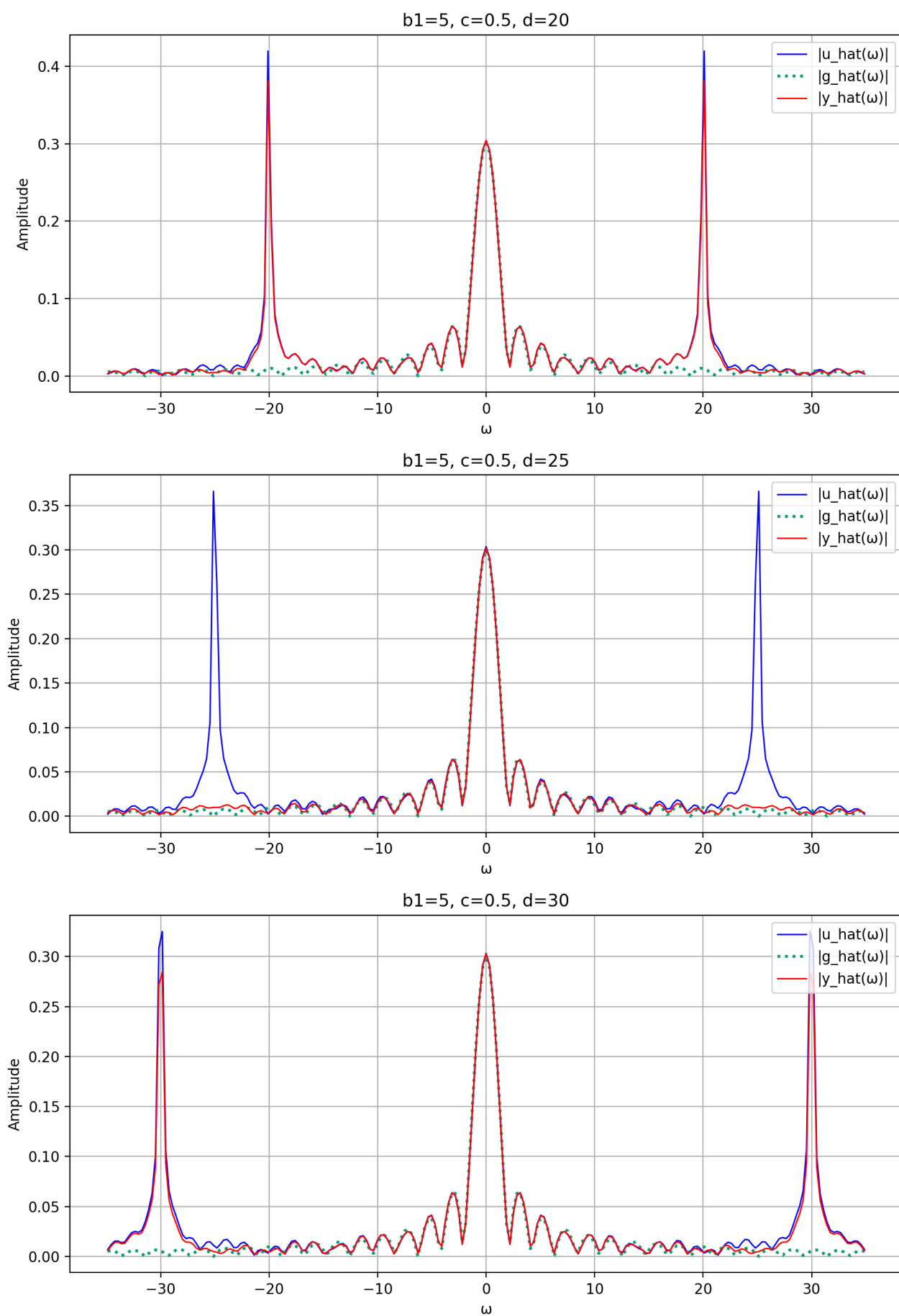


Рисунок 1.27. Фурье-образ

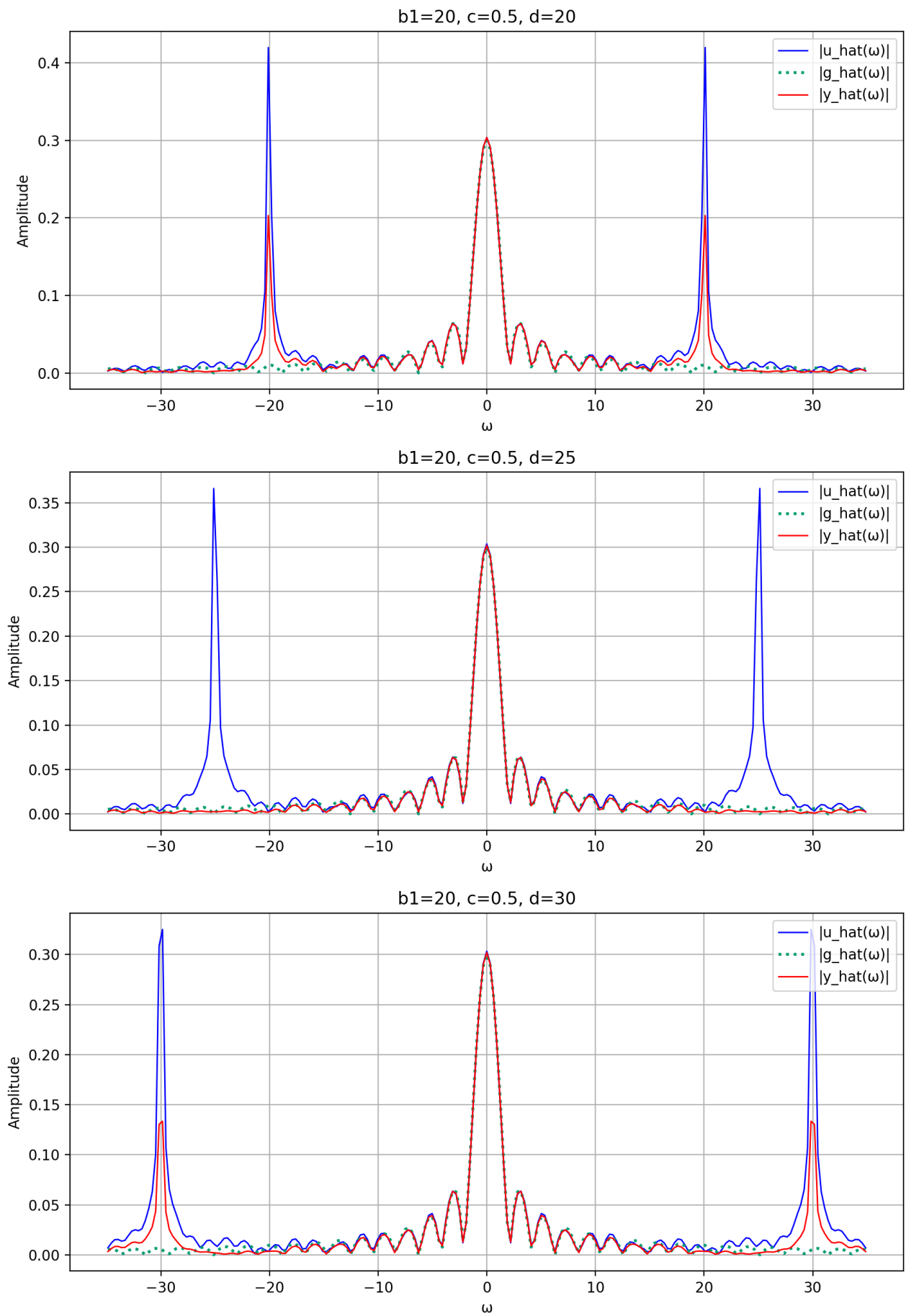


Рисунок 1.28. Фурье-образ

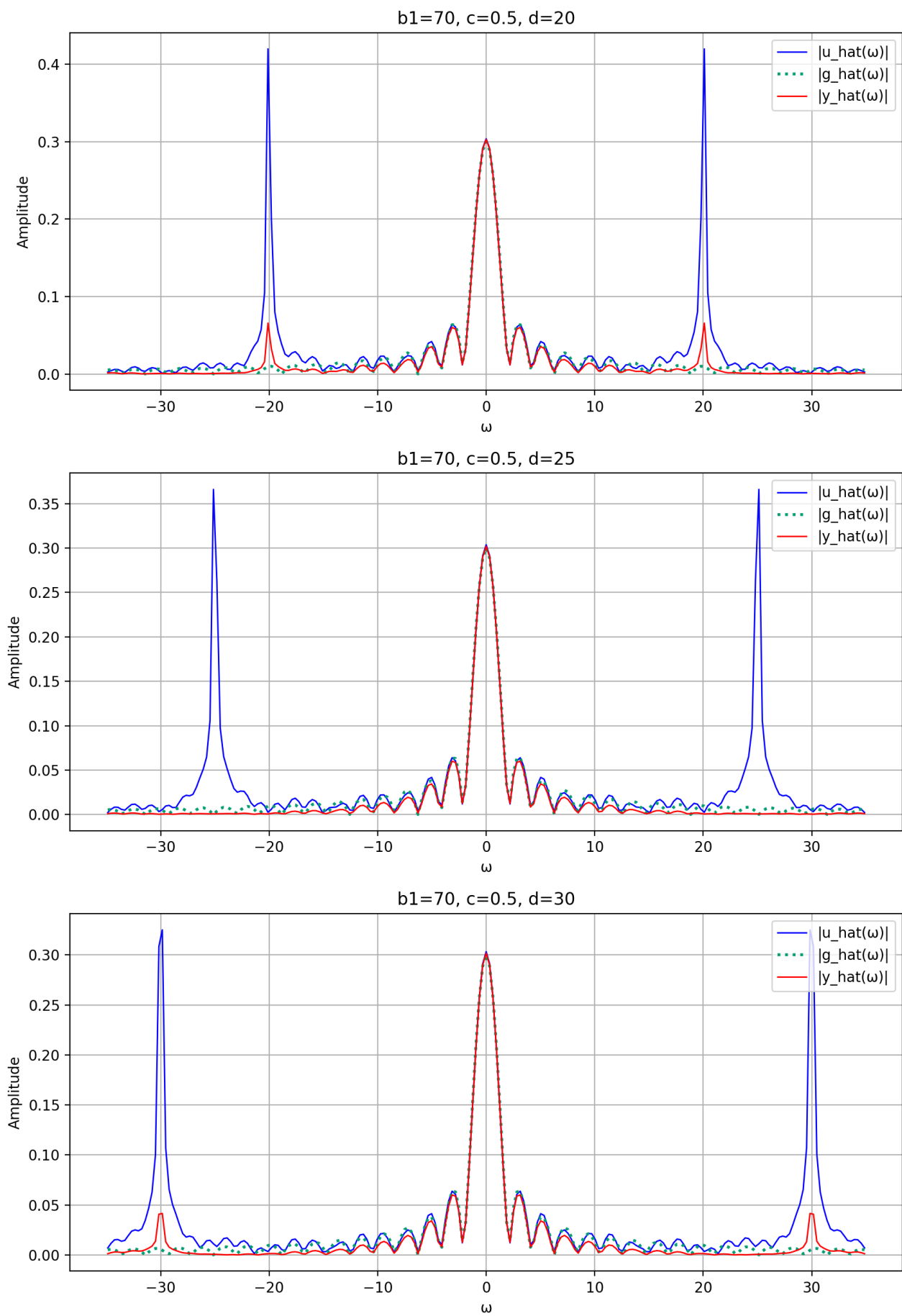


Рисунок 1.29. Фурье-образ

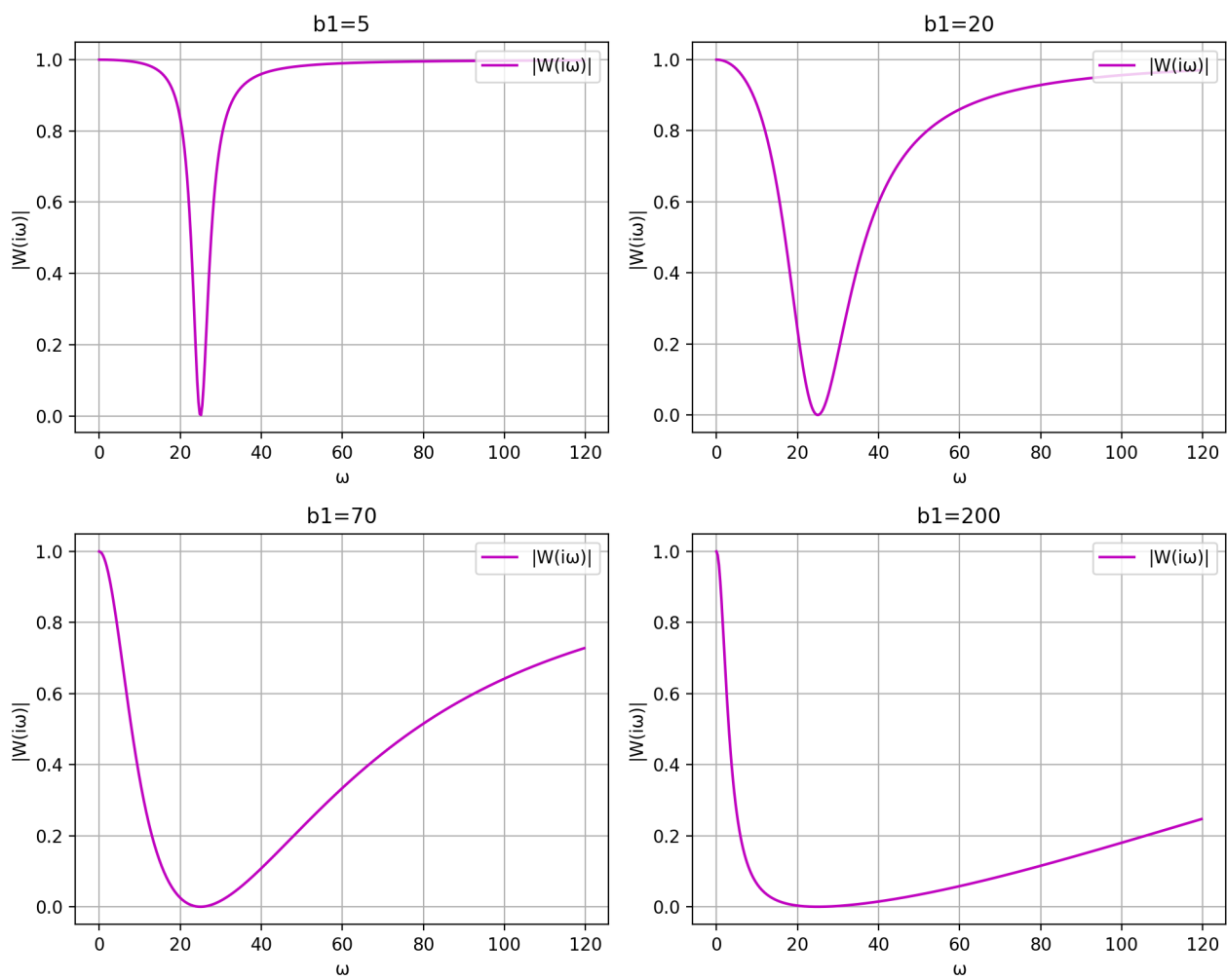


Рисунок 1.30. АЧХ

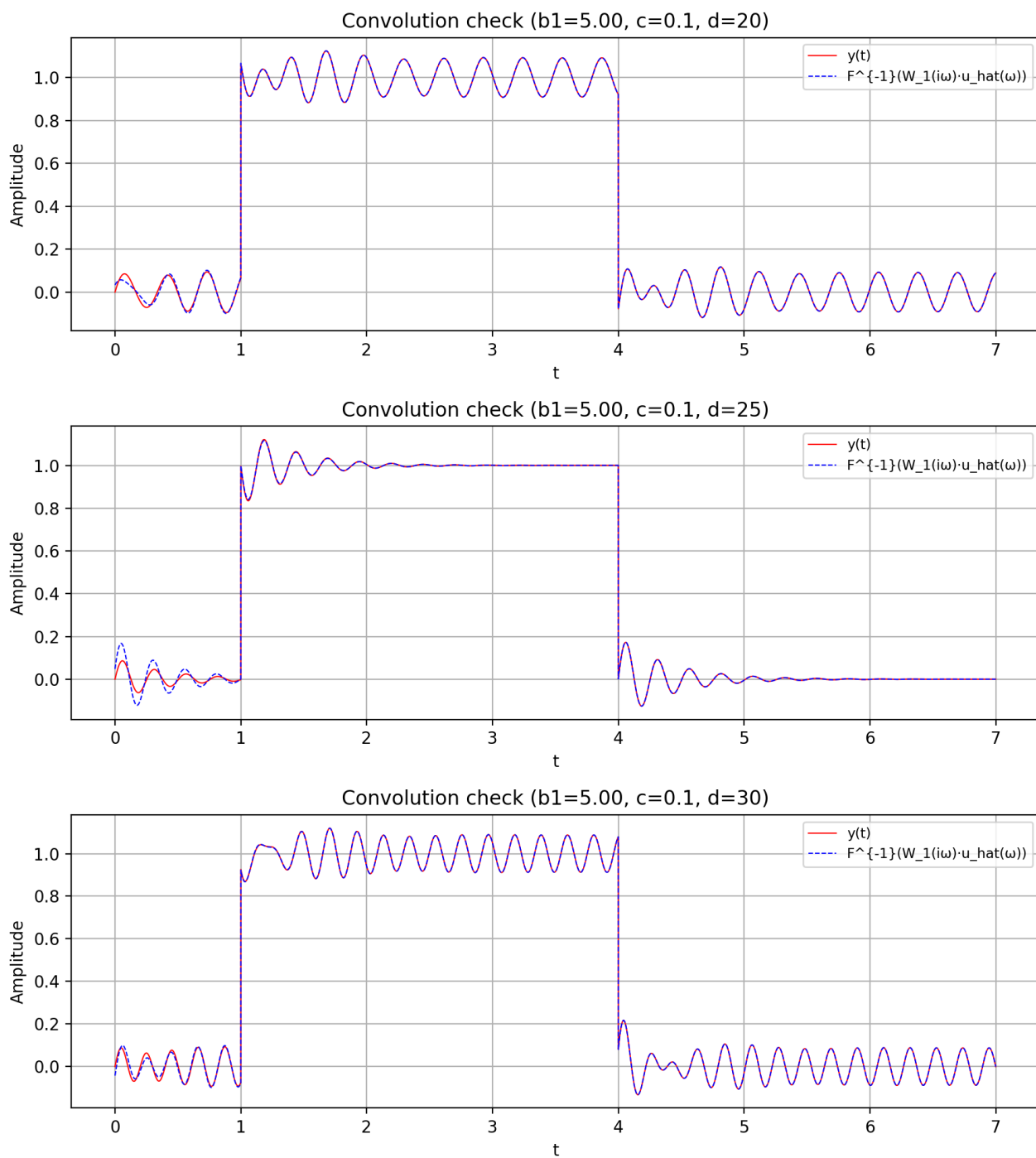


Рисунок 1.31. Временная область

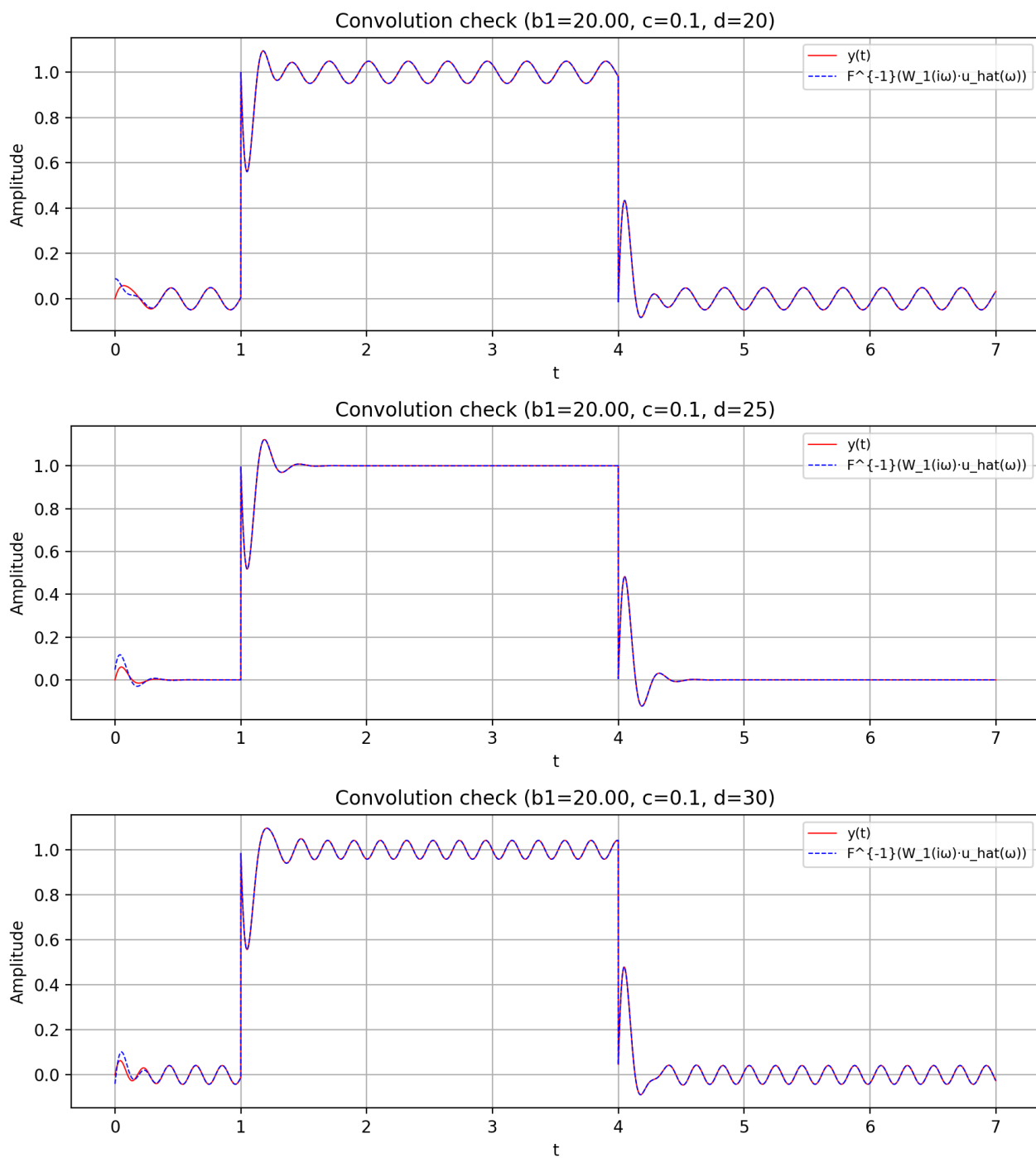


Рисунок 1.32. Временная область

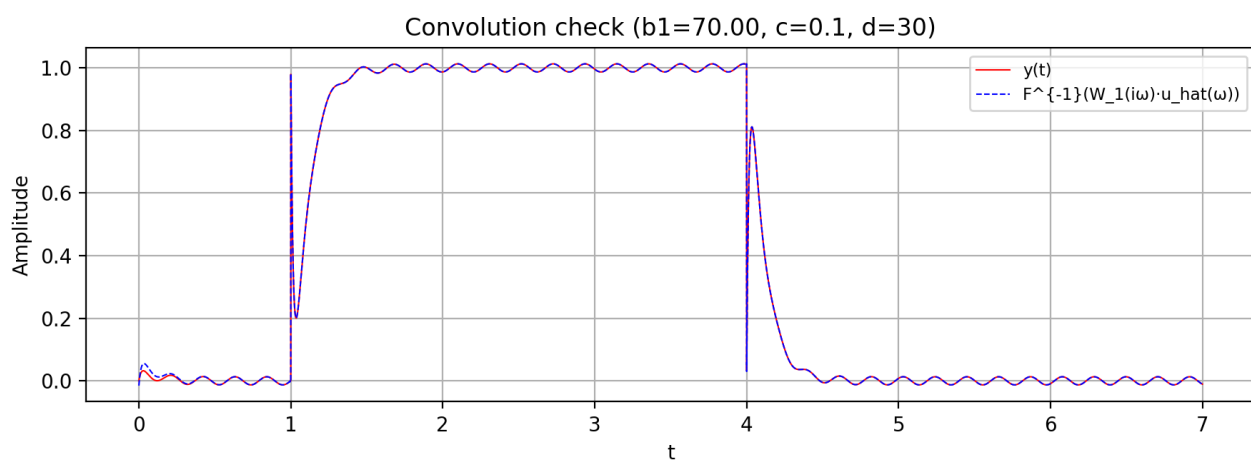
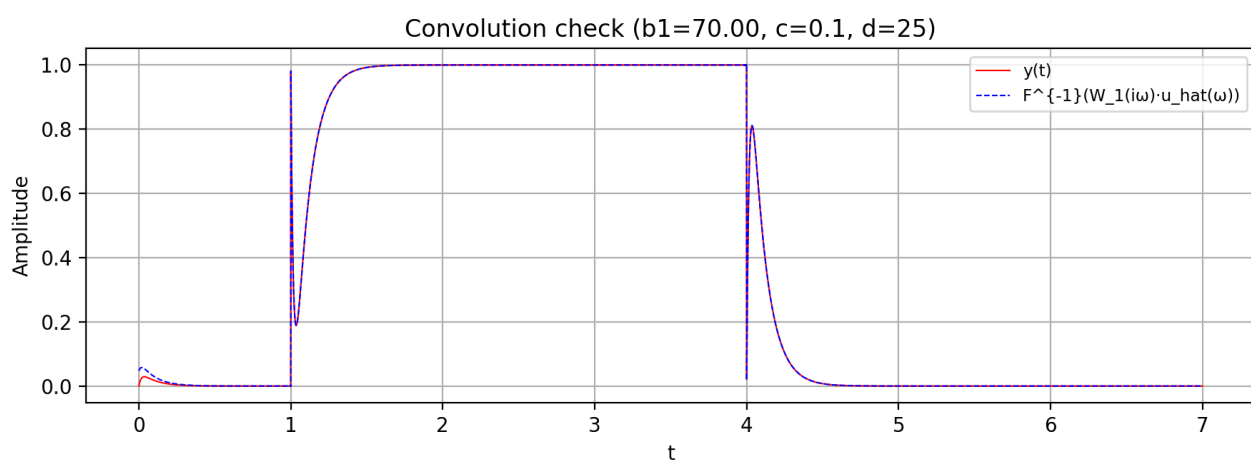
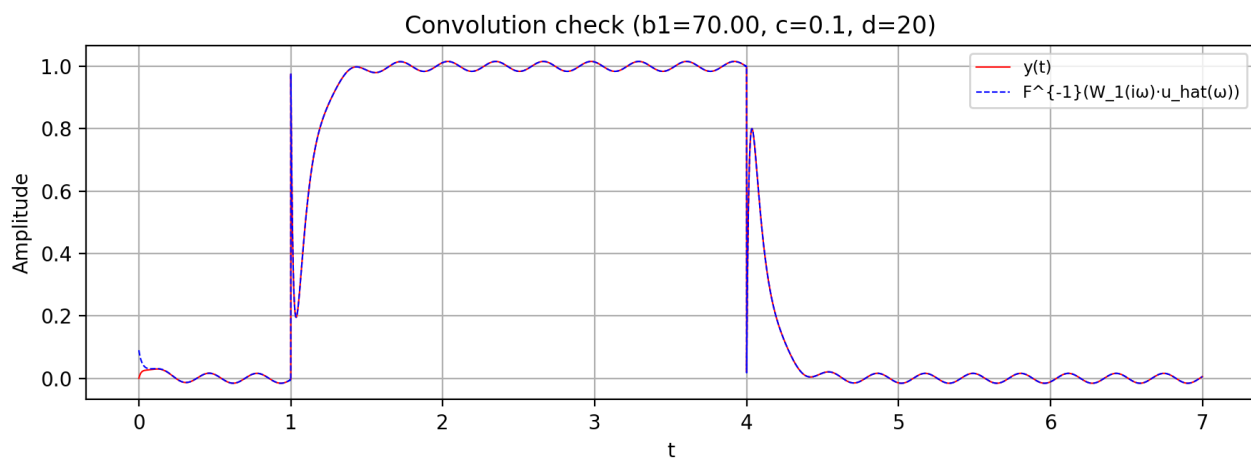


Рисунок 1.33. Временная область

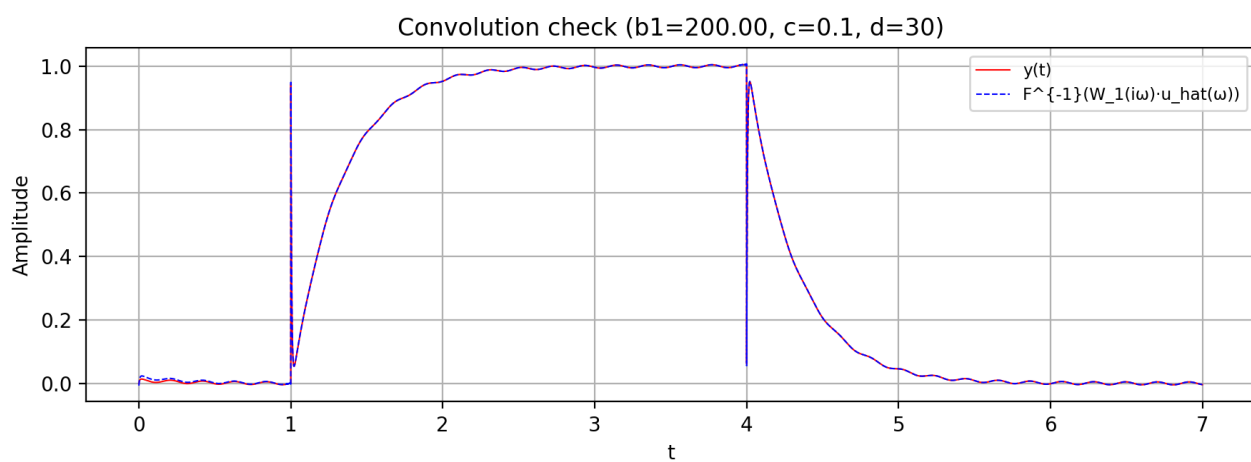
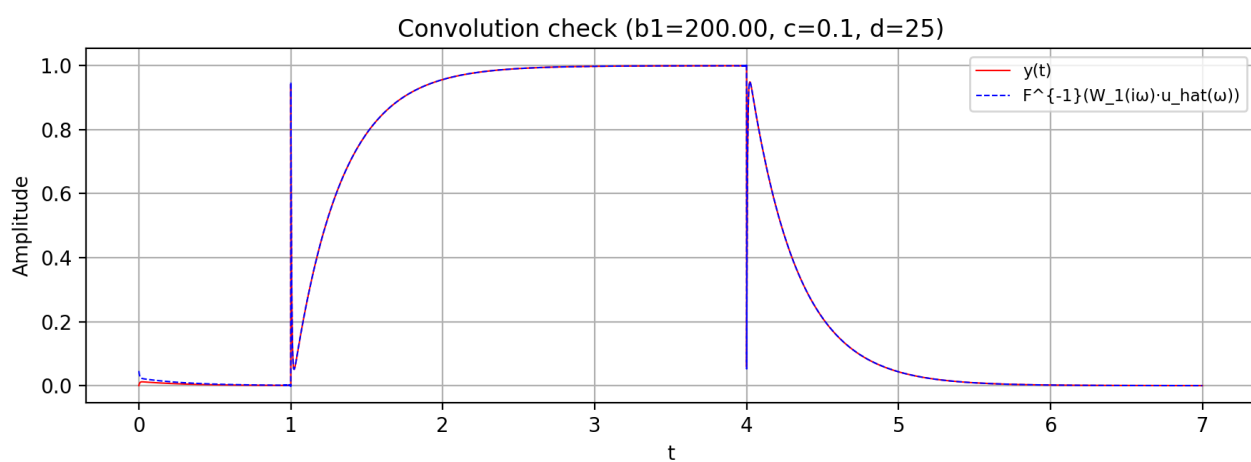
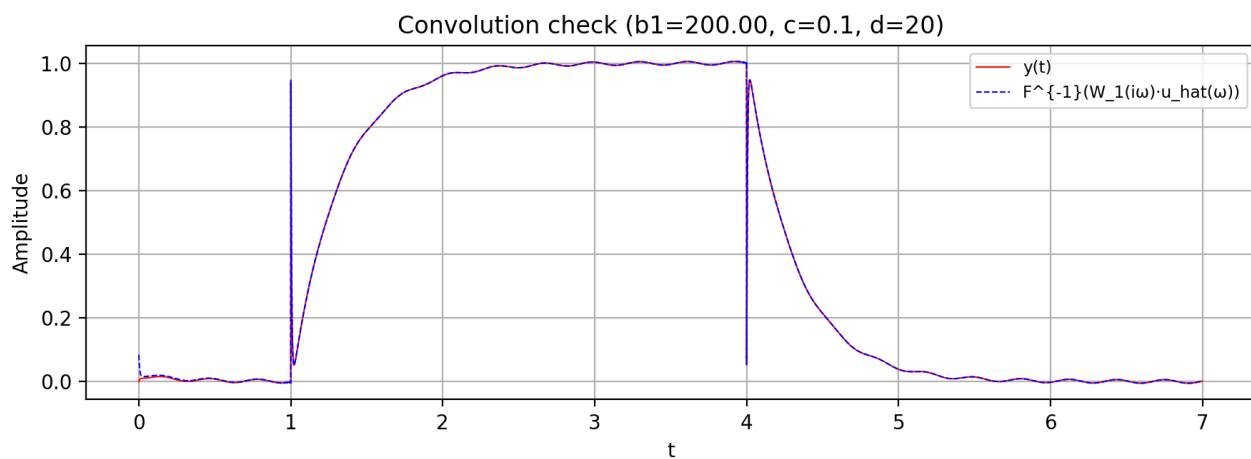


Рисунок 1.34. Временная область

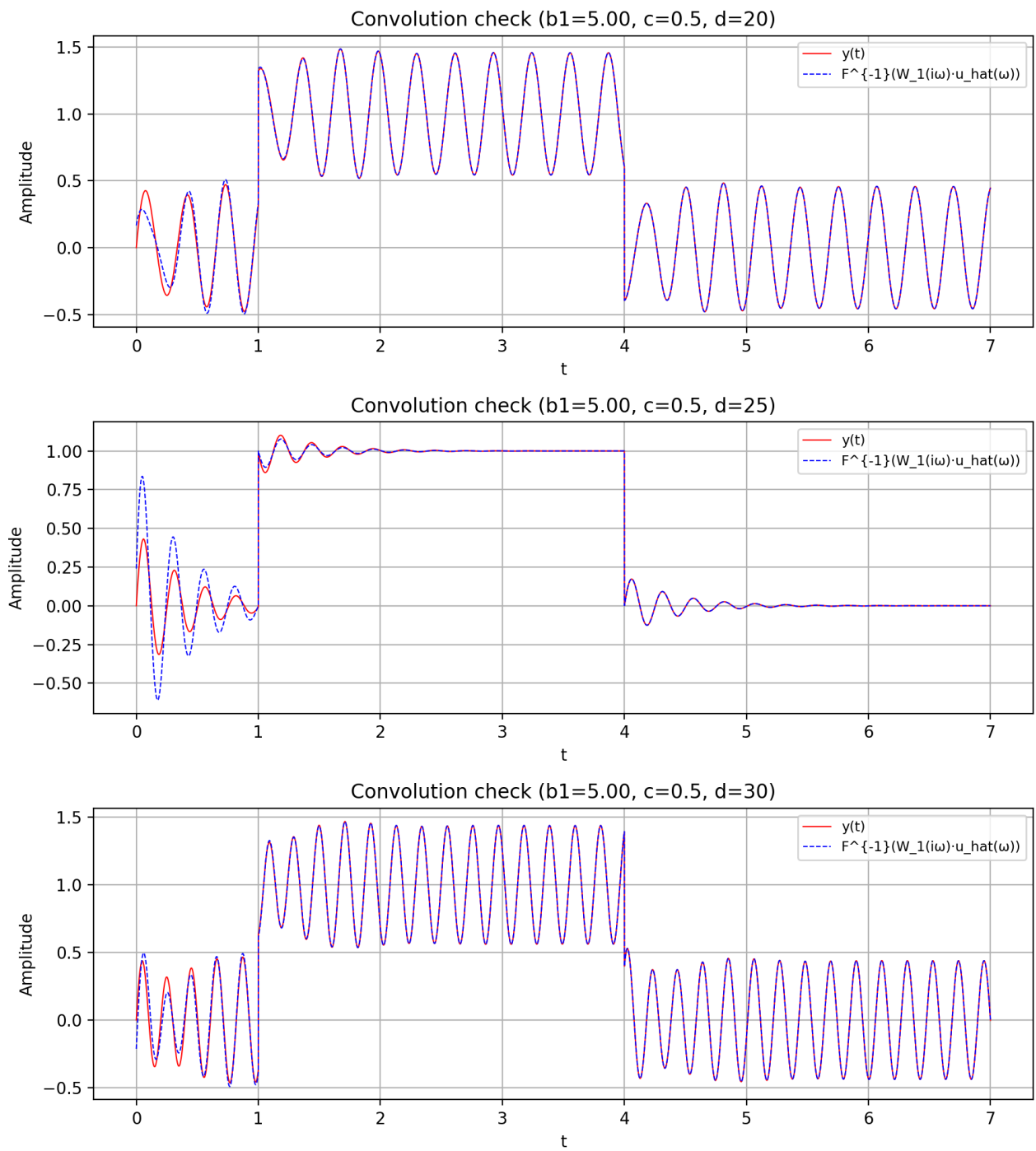


Рисунок 1.35. Временная область

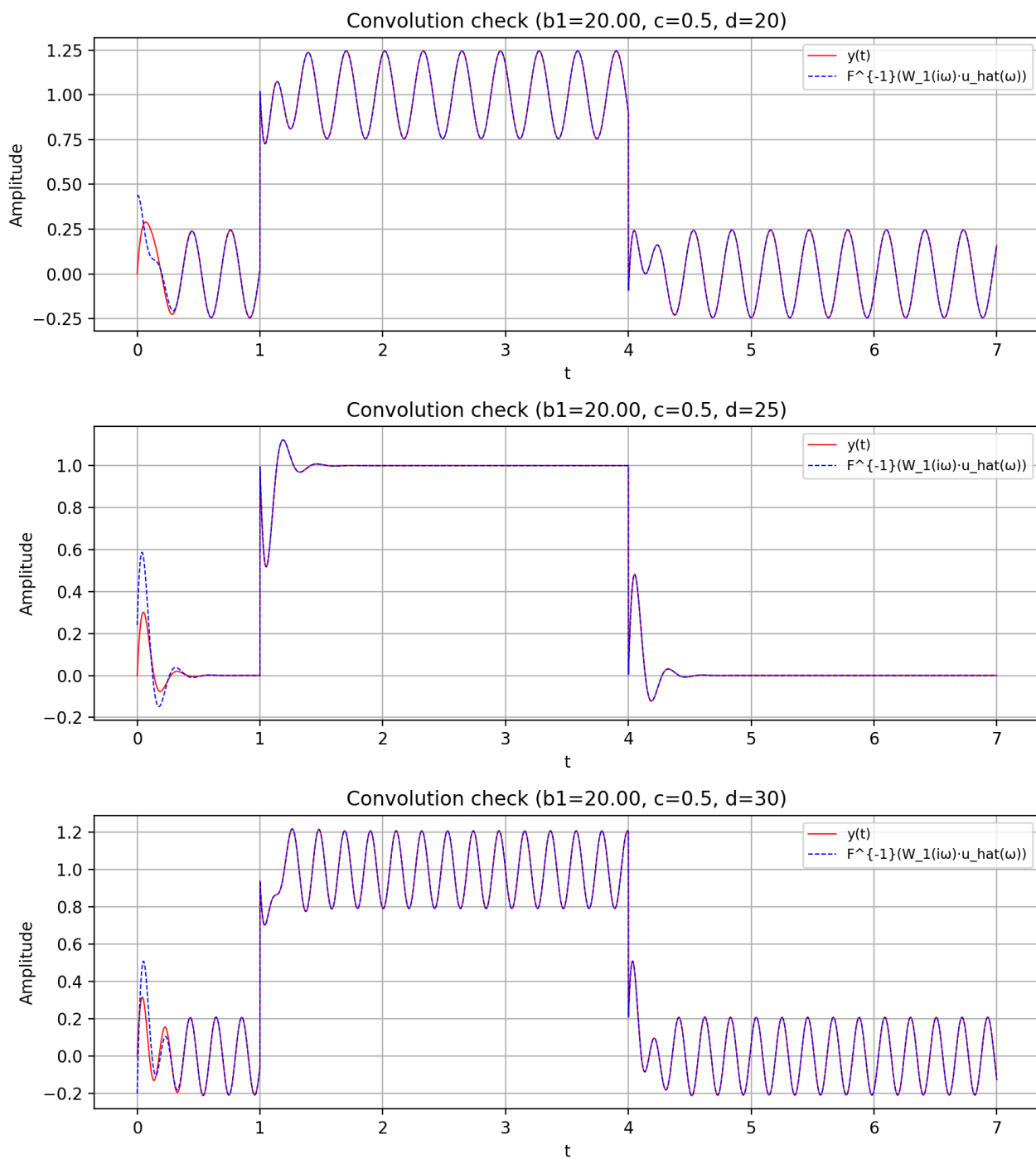


Рисунок 1.36. Временная область

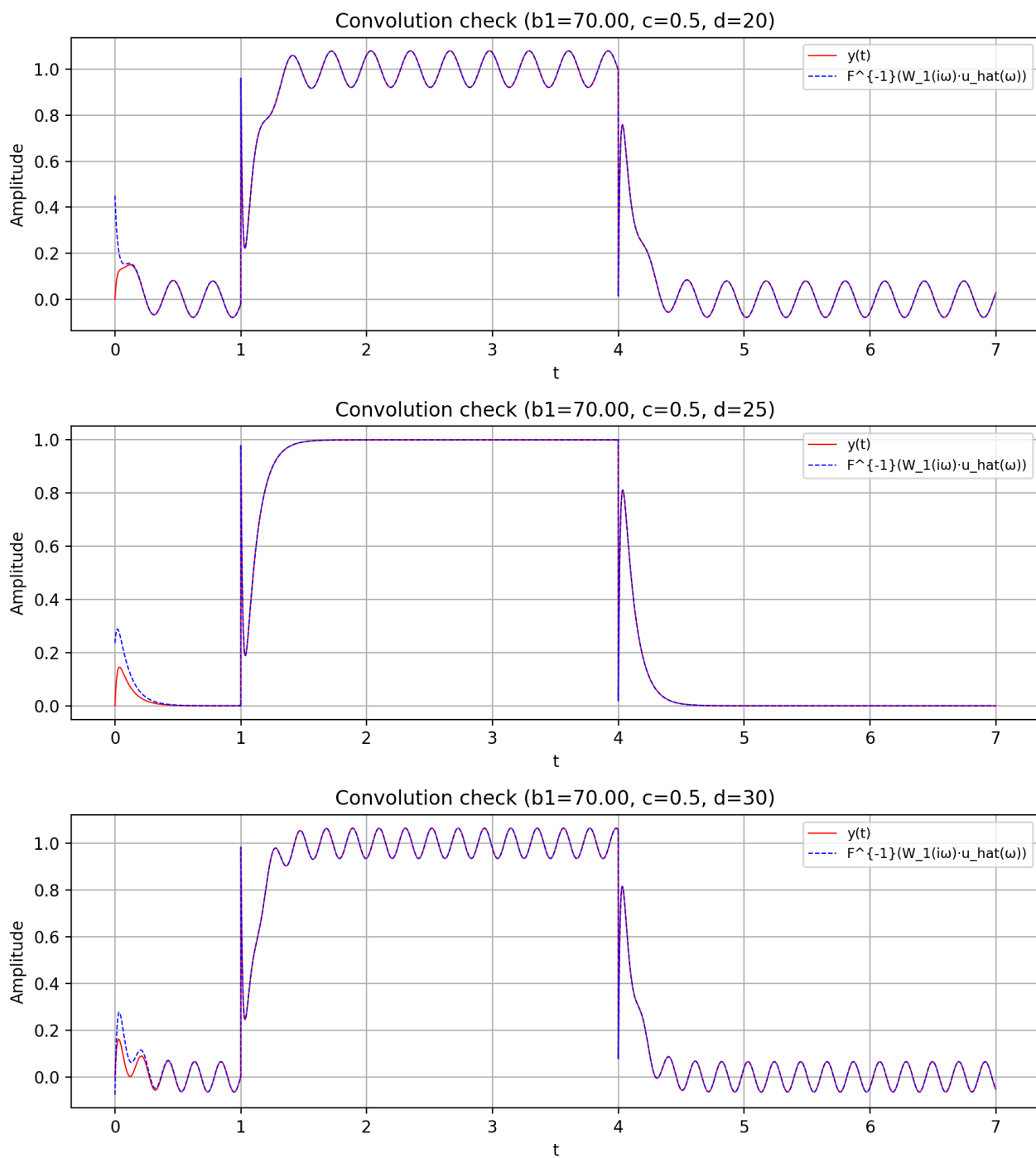


Рисунок 1.37. Временная область

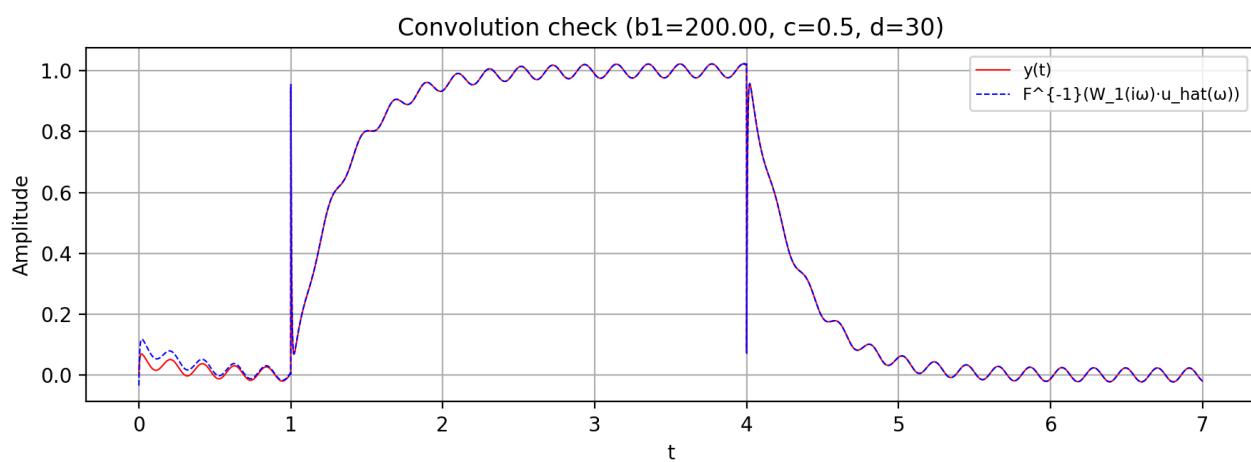
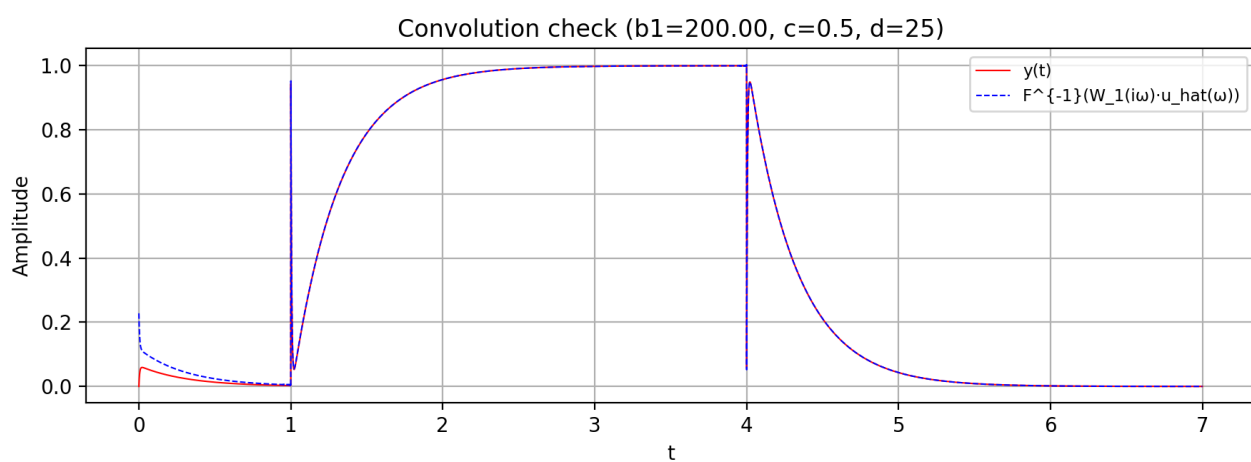
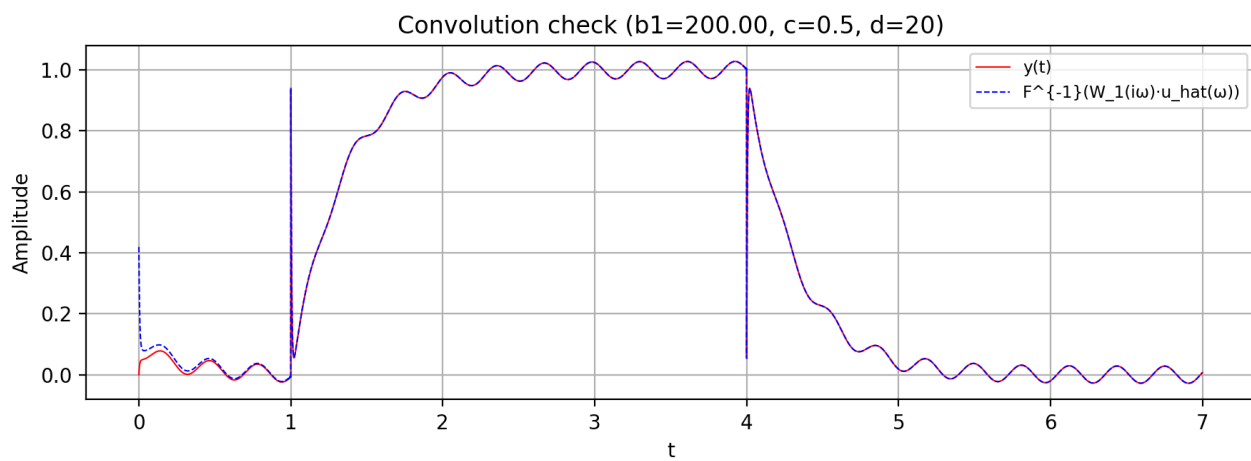


Рисунок 1.38. Временная область

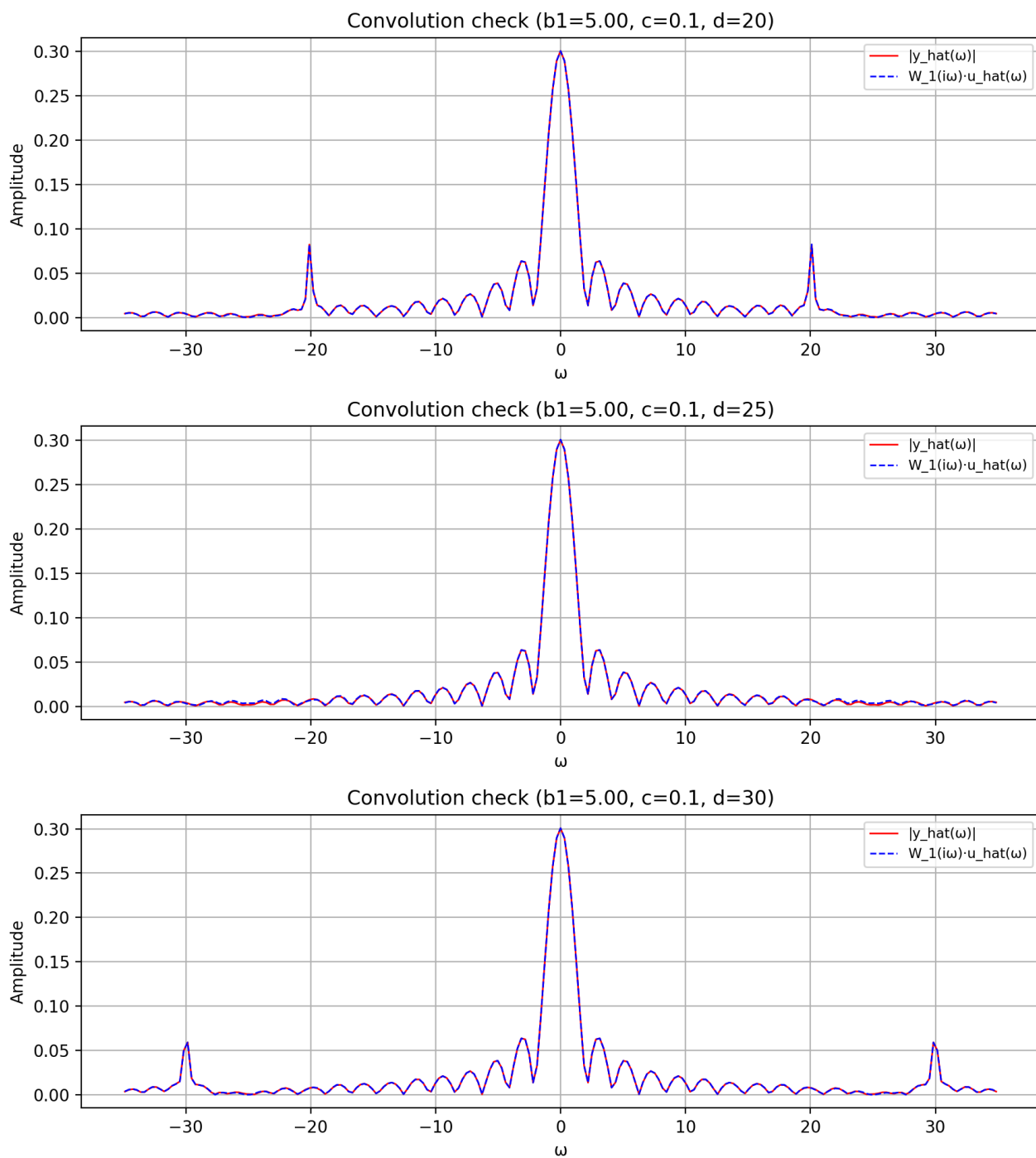


Рисунок 1.39. Частотная область

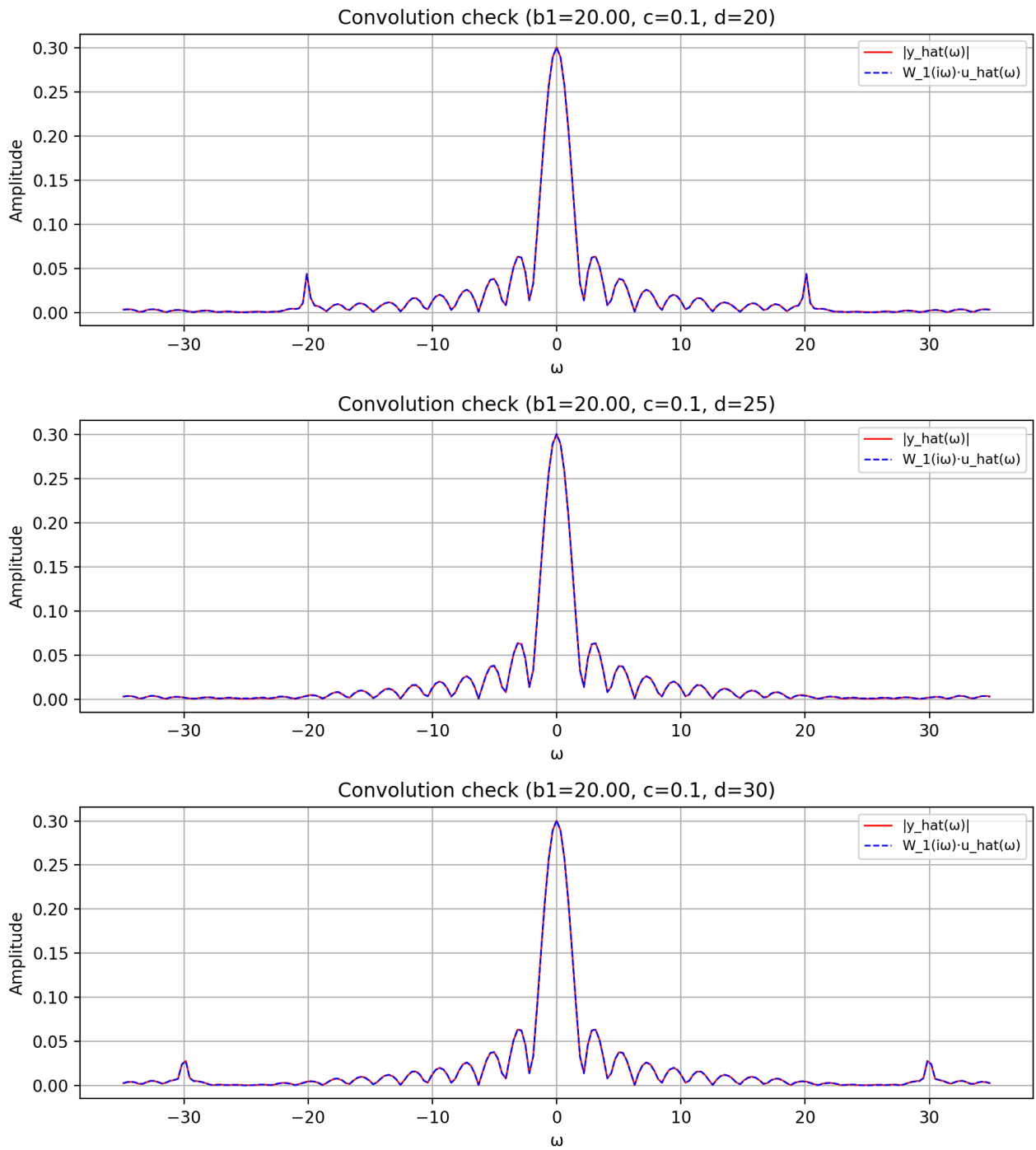


Рисунок 1.40. Частотная область

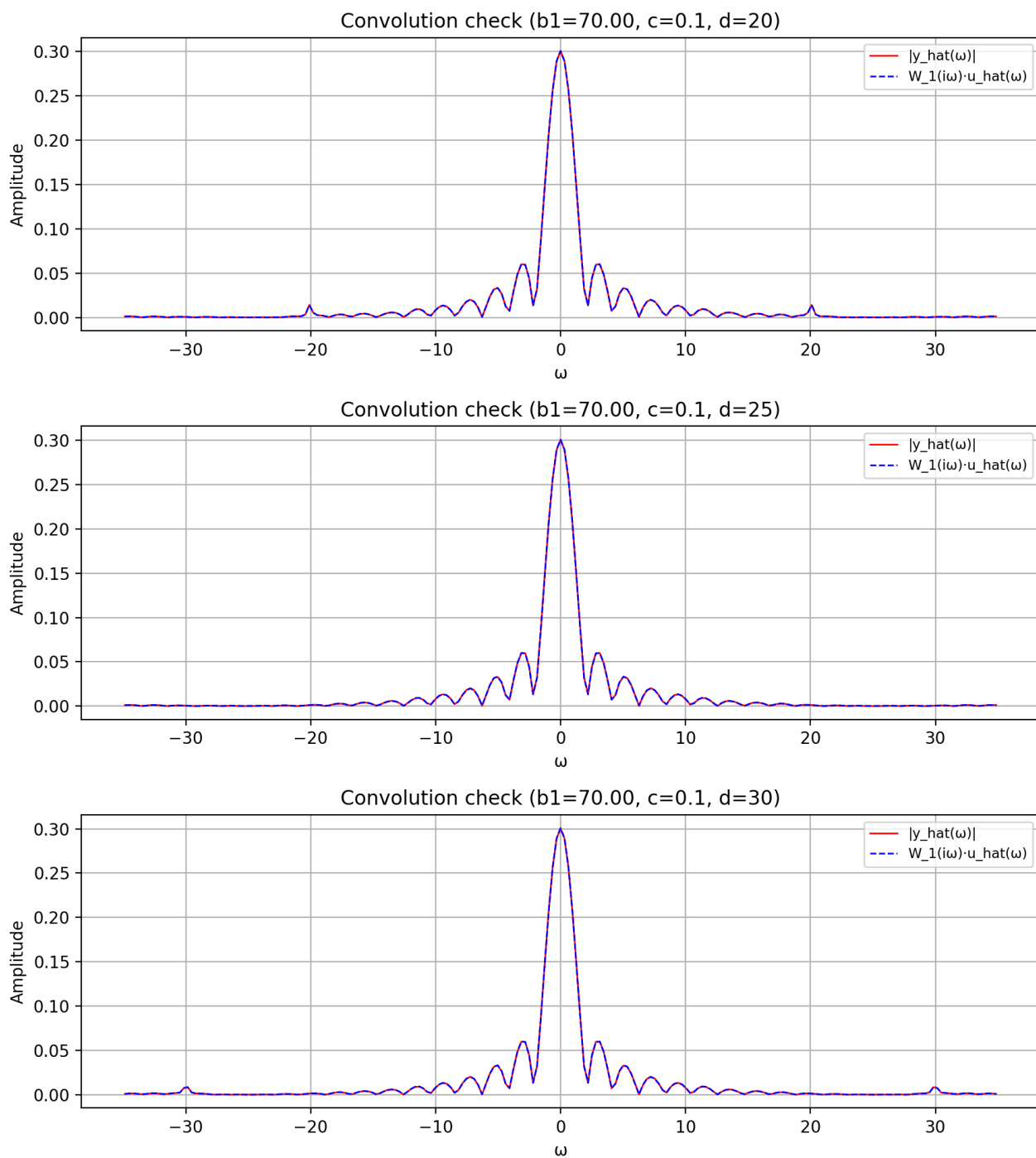


Рисунок 1.41. Частотная область

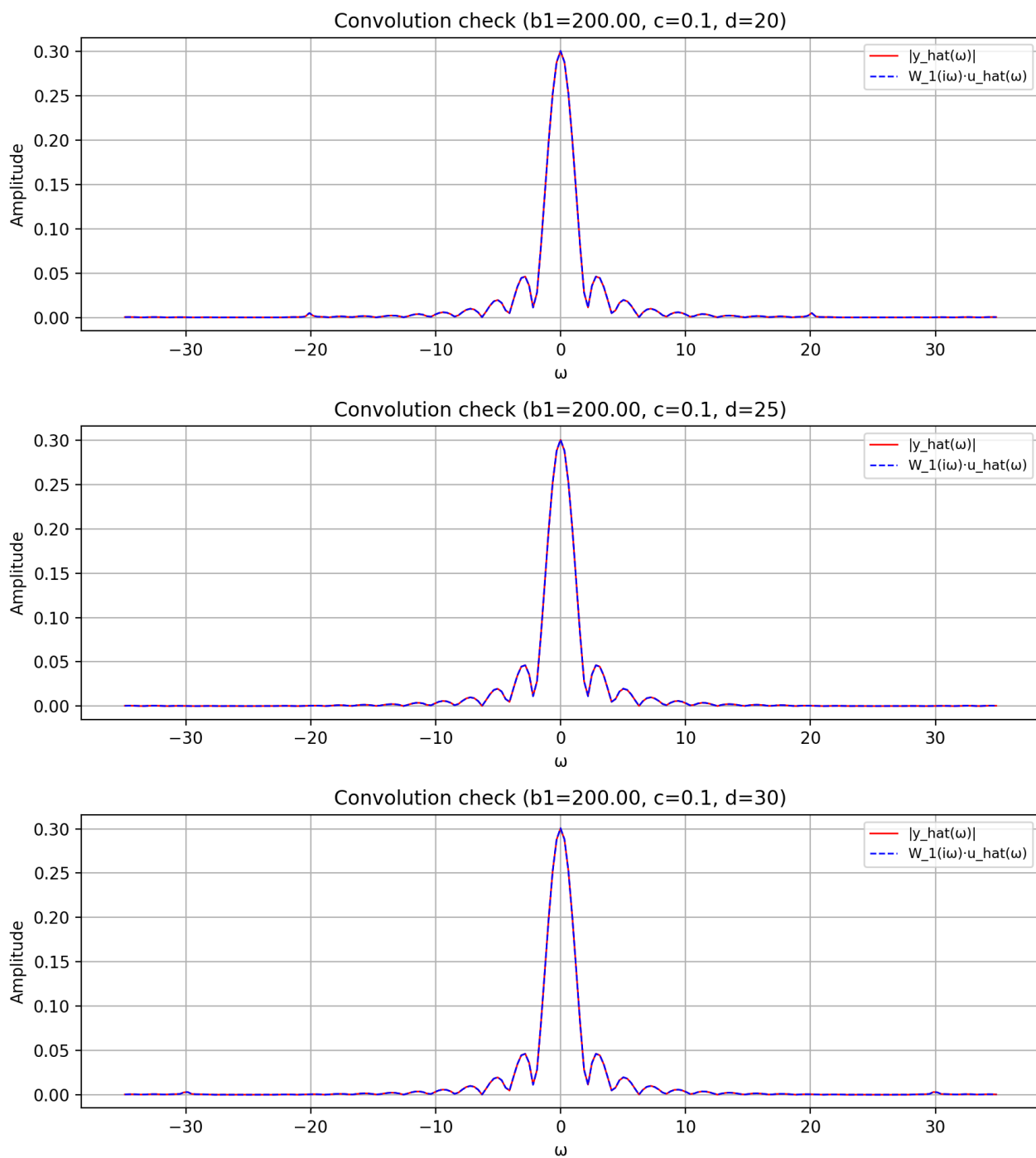


Рисунок 1.42. Частотная область

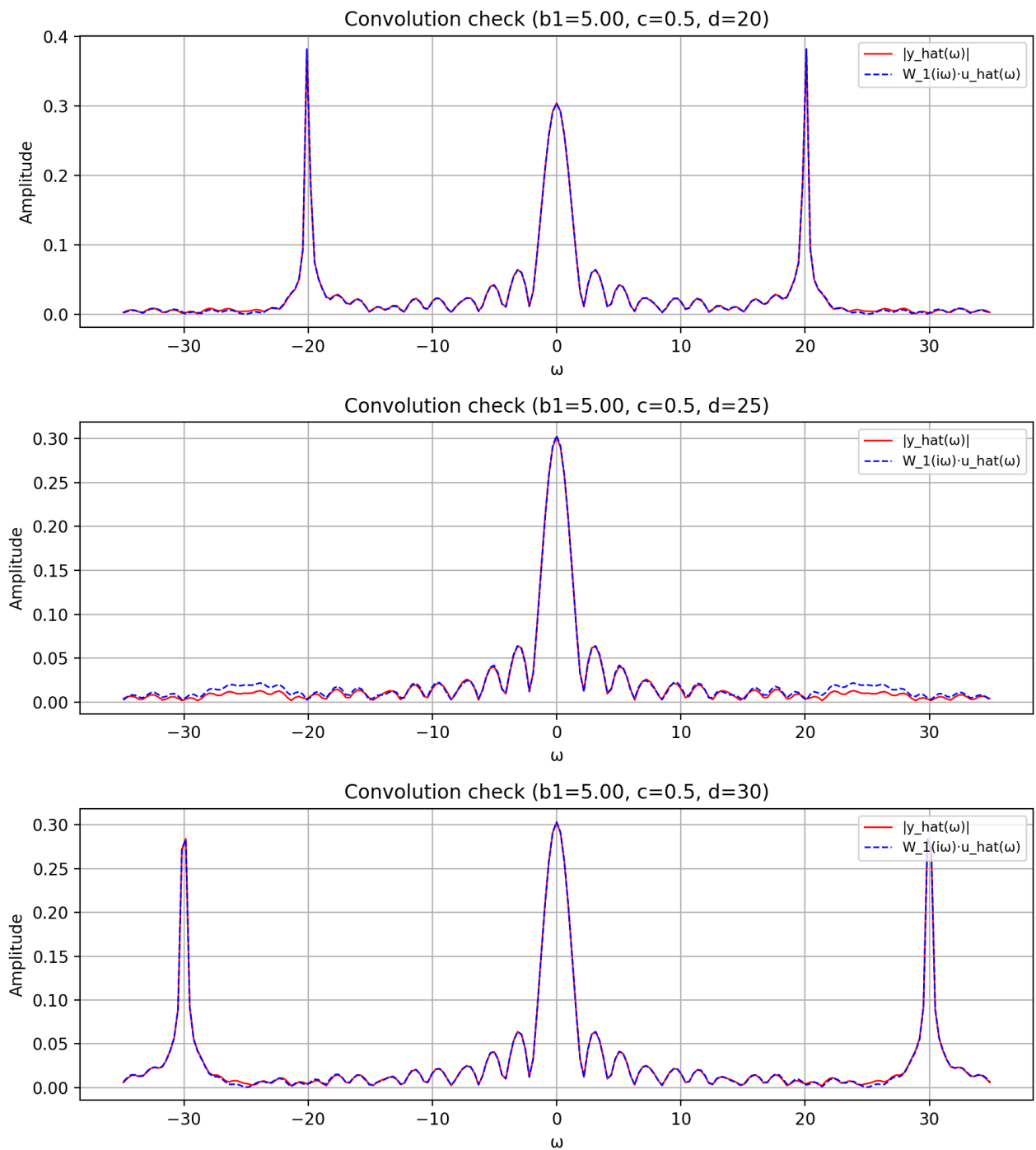


Рисунок 1.43. Частотная область

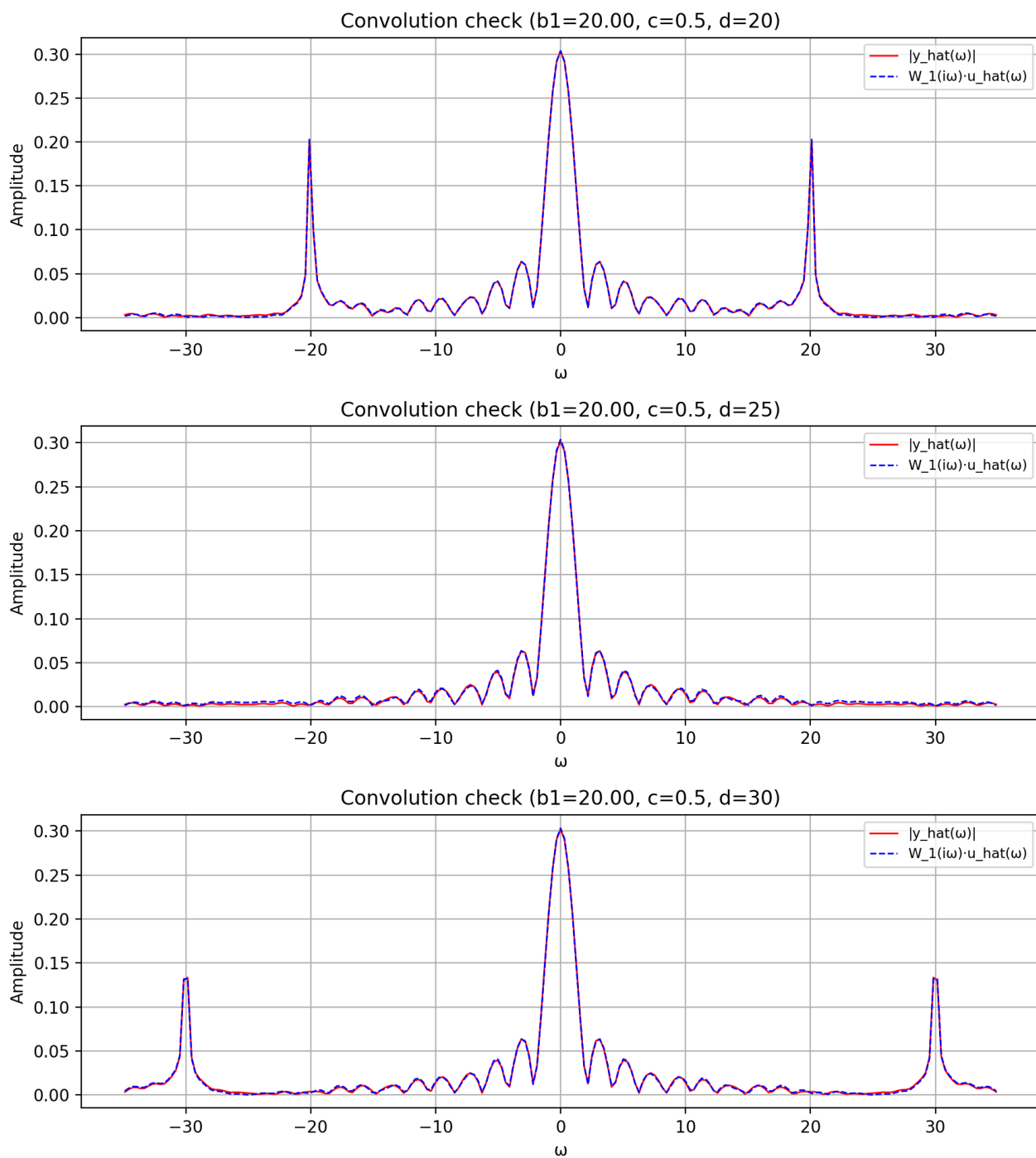


Рисунок 1.44. Частотная область

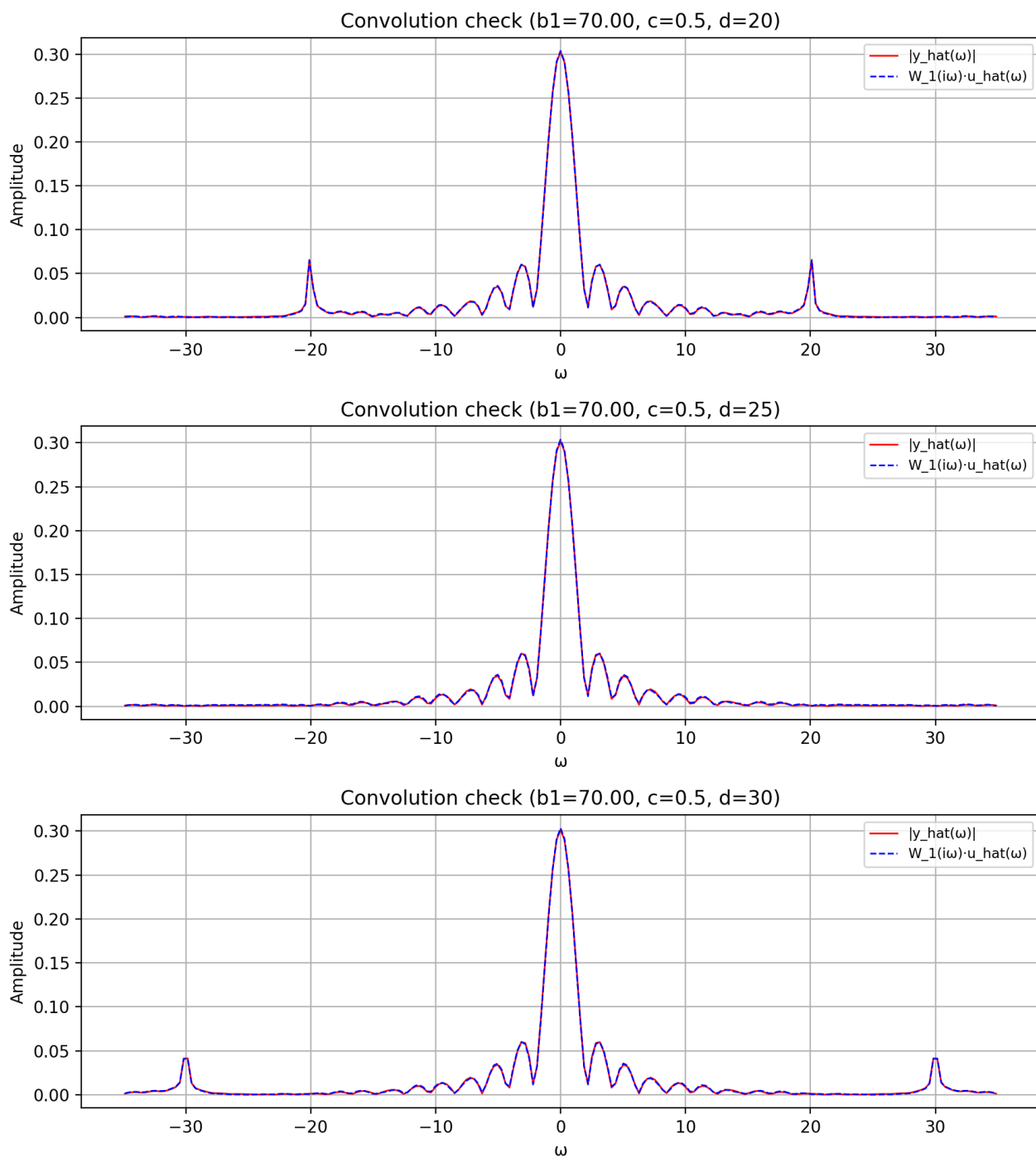


Рисунок 1.45. Частотная область

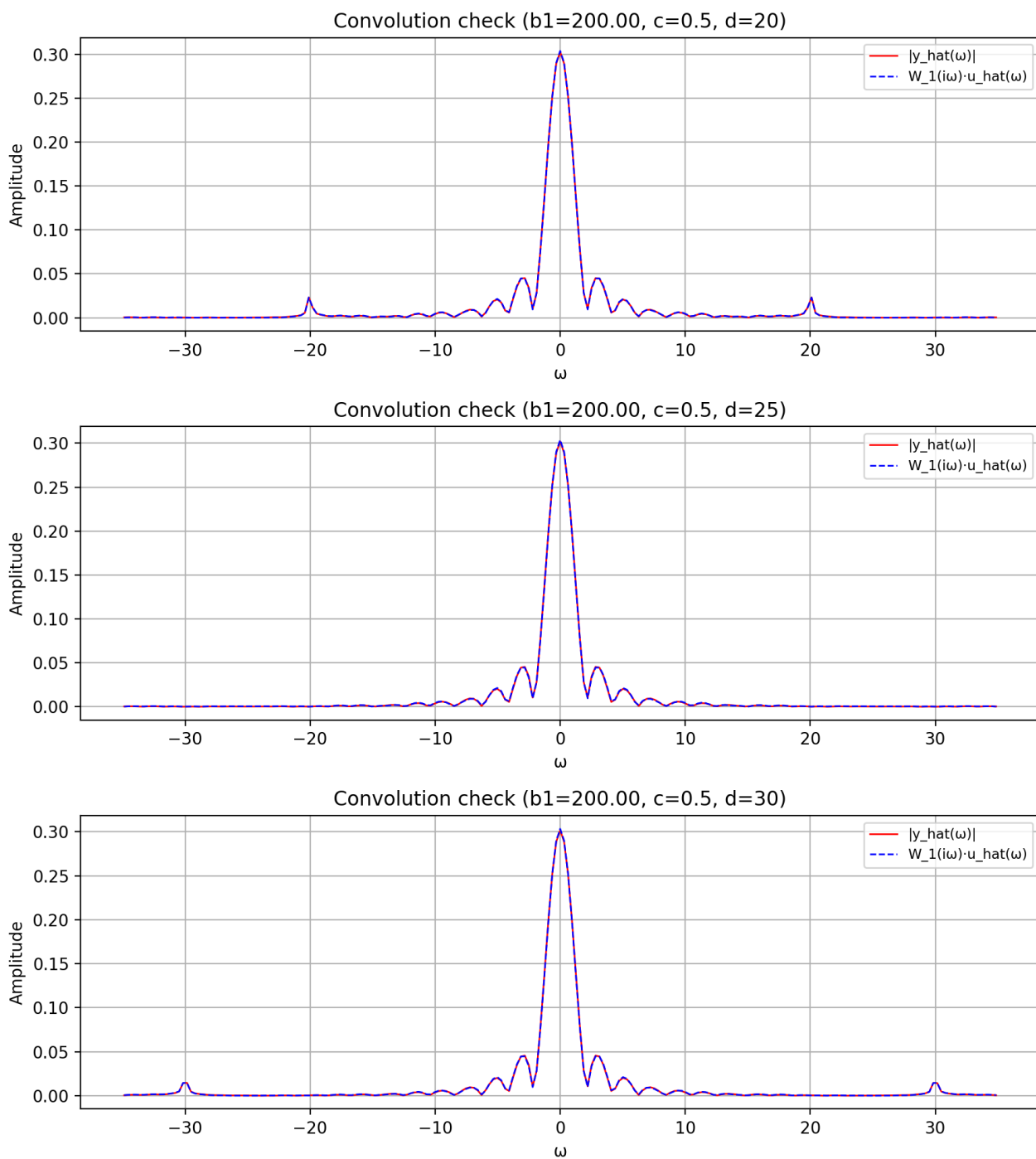


Рисунок 1.46. Частотная область

2. Задание 2. Сглаживание биржевых данных

Вообразим, что мы разрабатываем инвестиционное приложение, в котором должна присутствовать функция представления сглаженных графиков котировок акций. При этом степень сглаживания должна зависеть от рассматриваемого нами временного периода.

Скачаем [отсюда](#) файл с данными о стоимости акций Сбербанка за достаточно продолжительный период. Загрузим данные в Python и применим к ним линейную фильтрацию с помощью фильтра первого порядка. Последовательно возьмём следующие значения постоянной времени T : 1 день, 1 неделя, 1 месяц, 3 месяца, 1 год. Построим сравнительные графики исходного и фильтрованного сигналов, красиво оформим результат.

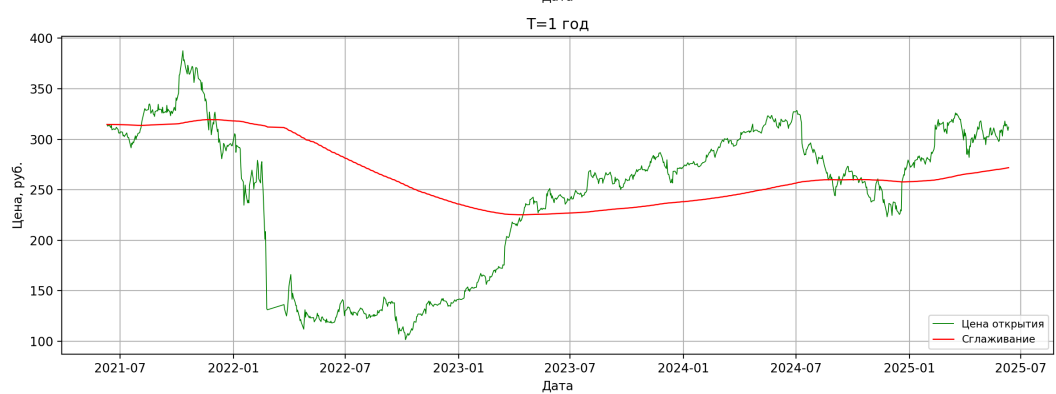
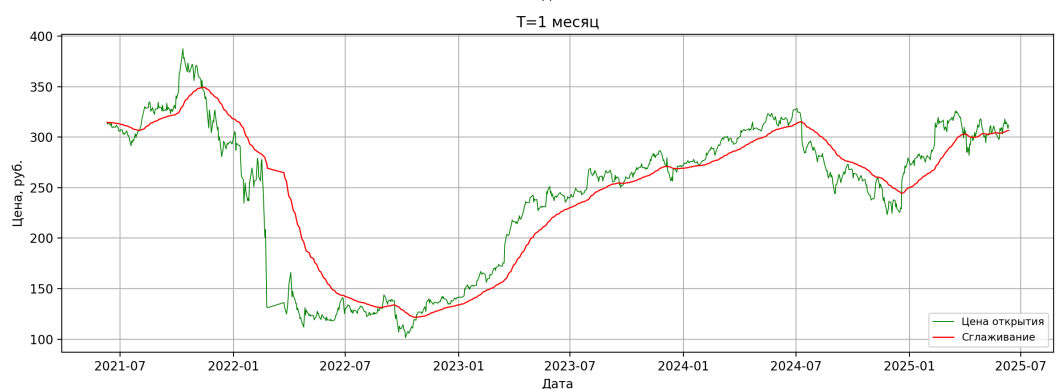
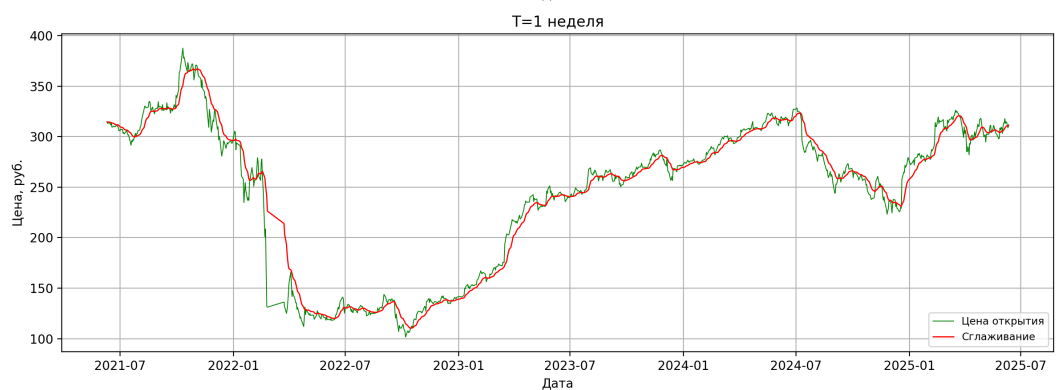
Возьмём данные за период 10.06.2021 – 10.06.2025 и будем анализировать цену закрытия (см. рис. 2.1).

Выводы

Линейная фильтрация может использоваться не только для обработки сигналов, но и быть так же полезна при анализе финансовых инструментов. Однако, на практике стоит применять более сложные фильтры, так как можно заметить, что данный вариант хуже справляется со сглаживанием там, где происходят резкие изменения в цене. В нашем рассмотренном варианте больше всего подходят случаи T : 1 день и 1 неделя, при большем T сглаживание начинает съедать слишком много информации о цене.

Общий вывод

Линейная фильтрация - эффективный инструмент для обработки сигналов и работы со сглаживанием графиков. Преимущество динамической фильтрации заключается в её возможности работать с сигналом непосредственно по мере его получения, что делает её востребованной в реальных практических задачах.



3. Приложение

```
1 # %%
2 import numpy as np
3 import matplotlib.pyplot as plt
4 from scipy.fft import fft, fftfreq, fftshift, ifft, ifftshift
5 from scipy import signal
6 import pandas as pd
7 from datetime import datetime
8 %matplotlib widget
9 np.random.seed(17)
10
11 # %% [markdown]
12 # Util-функции
13
14 # %%
15 def draw_plots(rows, cols, width, height, subplot_data,
16     ↳ legend_loc="best", legend_fontsize="small"):
17     fig, axes = plt.subplots(rows, cols, figsize=(width, height))
18     axes = axes.flatten() if rows * cols > 1 else [axes]
19     flat_data = [item for row in subplot_data for item in row]
20     for idx, data in enumerate(flat_data):
21         if idx ≥ len(axes):
22             raise
23             ↳ ValueError(f"Too many subplots provided in 'subplot_data': expected <
24             ↳ * cols}, got more.")
25         if not data:
26             continue
27         ax = axes[idx]
28         (
29             x_arrays, y_arrays,
30             labels,
31             x_label, y_label,
32             colors, linestyle,
33             linewidth, markers,
34             markersizes, title,
35             markerevery
36         ) = data + [None] * (12 - len(data))
37         num_plots = len(y_arrays)
38         for i in range(num_plots):
39             x = x_arrays[i]
40             y = y_arrays[i]
41             label = labels[i] if labels and i < len(labels) else None
42             color = colors[i] if colors and i < len(colors) else None
43             linestyle = linestyle[i] if linestyle and i <
44                 ↳ len(linestyle) else '-'
45             lw = linewidth[i] if linewidth and i < len(linewidth) else 2
46             marker = markers[i] if markers and i < len(markers) else None
47             markersize = markersizes[i] if markersizes and i <
48                 ↳ len(markersizes) else None
49             mevery = markerevery[i] if markerevery and i <
50                 ↳ len(markerevery) else None
51             ax.plot(x, y,
52                 label=label,
53                 color=color,
54                 linestyle=linestyle,
55                 linewidth=lw,
56                 marker=marker,
57                 markersize=markersize,
58                 markevery=mevery)
59         if labels:
60             ax.legend(loc=legend_loc, fontsize=legend_fontsize)
```

```

64         ax.grid(True)
65         if x_label:
66             ax.set_xlabel(x_label)
67         if y_label:
68             ax.set_ylabel(y_label)
69         if title:
70             ax.set_title(title)
71
72     for idx in range(len(flat_data), len(axes)):
73         fig.delaxes(axes[idx])
74
75     plt.gca().set_axisbelow(True)
76     plt.tight_layout()
77     plt.show()
78
79     # %%
80     g = lambda t, a: a if (1 ≤ t ≤ 4) else 0
81     u = lambda t, a, b, c, d: g(t, a) + b * np.random.uniform(-1, 1) + c *
      ↪ np.sin(d * t)
82
83     # %% [markdown]
84     # # Задание 1.1
85
86     # %%
87     def calc_spectrum(signal, dt):
88         spectrum = fftshift(np.abs(fft(signal)))
89         freq = 2 * np.pi * fftshift(fftfreq(len(signal), dt))
90         return freq, spectrum
91
92     # %%
93     T_range = [0.025, 0.1, 0.4]
94     a_range = [2, 8, 16]
95     b = 0.5
96     c = 0
97     d = 10
98     plots_11 = []
99
100    N = 2 ** 15
101    t = np.linspace(0, 20, N)
102    dt = t[1] - t[0]
103
104    time_subplot_data = []
105    spectrum_subplot_data = []
106    freq_response_subplot_data = []
107    convolution_subplot_data_time = []
108    convolution_subplot_data_spectrum = []
109
110
111    for i, a in enumerate(a_range):
112        time_row = []
113        spectrum_row = []
114        freq_response_row = []
115        conv_time_row = []
116        conv_freq_row = []
117
118        for j, T in enumerate(T_range):
119            g_signal = np.array([g(ti, a) for ti in t])
120            u_signal = np.array([u(ti, a, b, c, d) for ti in t])
121
122            W = signal.TransferFunction([1], [T, 1])
123            _, y_signal, _ = signal.lsim(W, U=u_signal, T=t)
124
125            freq_g, func_g = calc_spectrum(g_signal, dt)
126            freq_u, func_u = calc_spectrum(u_signal, dt)
127            freq_y, func_y = calc_spectrum(y_signal, dt)
128
129            W_mag = 1 / np.sqrt(1 + (T * freq_g)**2)
130
131            afc_mask = (freq_g ≥ 0) & (freq_g < 100)
132            omega_mask = np.abs(freq_g) < 40
133
134            time_plot = [
135                [t[t < 7]] * 3,
136                [u_signal[t < 7], g_signal[t < 7], y_signal[t < 7]],

```

```

137         ['u(t)', 'g(t)', 'y(t), filtered'],
138         't', 'Amplitude',
139         ['g', 'b', 'r'],
140         ['-',] * 3,
141         [0.5, 1.2, 1.0],
142         [None] * 3,
143         [None] * 3,
144         f'a={a}, T={T}',
145     ]
146     time_row.append(time_plot)
147
148     spectrum_plot = [
149         [freq_u[omega_mask], freq_g[omega_mask], freq_y[omega_mask]],
150         [2.0 / N * func_u[omega_mask],
151          2.0 / N * func_g[omega_mask],
152          2.0 / N * func_y[omega_mask]
153         ],
154         ['|u_hat(w)|', '|g_hat(w)|', '|y_hat(w)|'],
155         'w', 'Amplitude',
156         ['b', "#09FF00", 'r'],
157         ['-', ':', '-'],
158         [1.0, 1.0, 1.0],
159         [None] * 3,
160         [None] * 3,
161         f'a={a}, T={T}',
162     ]
163     spectrum_row.append(spectrum_plot)
164
165     target = 1/np.sqrt(2)
166     idx = np.argmin(np.abs(W_mag[afc_mask] - target))
167     cutoff_freq = freq_g[afc_mask][idx]
168     cutoff_amp = W_mag[afc_mask][idx]
169
170     freq_response_plot = [
171         [freq_g[afc_mask], [cutoff_freq, cutoff_freq], [0,
172         ↪ cutoff_freq]],
173         [W_mag[afc_mask], [0, cutoff_amp], [cutoff_amp, cutoff_amp]],
174         ['|W(iw)|', "w_c", None],
175         'w', '|W(iw)|',
176         ['m', 'b', 'b'],
177         ['-', '--', '--'],
178         [1.5, 0.9, 0.9],
179         [None] * 3,
180         [None] * 3,
181         f'T={T}',
182     ]
183     freq_response_row.append(freq_response_plot)
184
185     W_freq = 1 / (1 + 1j * T * freq_u)
186     y_freq = ifft(ifftshift(W_freq) * fft(u_signal)).real
187
188     conv_plot_time = [
189         [t[t<7], t[t<7]],
190         [y_signal[t<7], y_freq[t<7]],
191         ['y(t)', 'F^{-1}(W_1(iw)·u_hat(w))'],
192         't', 'Amplitude',
193         ['r', 'b'],
194         ['-', '--'],
195         [0.75, 0.75],
196         [None]*2,
197         [None]*2,
198         f'Convolution check (a={a}, T={T})'
199     ]
200     conv_time_row.append(conv_plot_time)
201
202     func_y_mult = np.abs(W_freq) * func_u
203
204     conv_plot_freq = [
205         [freq_y[omega_mask], freq_u[omega_mask]],
206         [2.0 / N * func_y[omega_mask], 2.0 / N *
207         ↪ func_y_mult[omega_mask]],

```

```

206         ['|y_hat(w)|', 'W_1(iw)·u_hat(w)'],
207         'w', 'Amplitude',
208         ['r', 'b'],
209         ['-', '--'],
210         [1, 1],
211         [None]*2,
212         [None]*2,
213         f'Convolution check (a={a}, T={T})'
214     ]
215     conv_freq_row.append(conv_plot_freq)
216
217
218     time_subplot_data.append(time_row)
219     spectrum_subplot_data.append(spectrum_row)
220     if i == 0:
221         freq_response_subplot_data.append(freq_response_row)
222         convolution_subplot_data_time.append(conv_time_row)
223         convolution_subplot_data_spectrum.append(conv_freq_row)
224
225
226     for tsd in time_subplot_data:
227         draw_plots(
228             rows=3,
229             cols=1,
230             width=9,
231             height=13,
232             subplot_data=[tsd],
233             legend_loc='upper right'
234         )
235
236     for ssd in spectrum_subplot_data:
237         draw_plots(
238             rows=3,
239             cols=1,
240             width=9,
241             height=13,
242             subplot_data=[ssd],
243             legend_loc='upper right'
244         )
245
246     draw_plots(
247         rows=3,
248         cols=1,
249         width=7,
250         height=10,
251         subplot_data=freq_response_subplot_data,
252         legend_loc='upper right'
253     )
254
255     for csd in convolution_subplot_data_time:
256         draw_plots(
257             rows=3,
258             cols=1,
259             width=9,
260             height=10,
261             subplot_data=[csd],
262             legend_loc='upper right'
263         )
264
265     for fcd in convolution_subplot_data_spectrum:
266         draw_plots(
267             rows=3,
268             cols=1,
269             width=9,
270             height=10,
271             subplot_data=[fcd],
272             legend_loc='upper right'
273         )
274
275     # %% [markdown]
276     # # Задание 1.2
277
278     # %%
279     a = 1

```

```

280 b = 0
281 c_range = [0.1, 0.5]
282 d_range = [20, 25, 30]
283
284 a1 = 0
285 a2 = b2 = 625
286 b1_range = [5, 20, 70, 200]
287
288 N = 2 ** 15
289 t = np.linspace(0, 20, N)
290 dt = t[1] - t[0]
291
292 time_subplot_data = []
293 spectrum_subplot_data = []
294 freq_response_subplot_data = []
295 convolution_subplot_data_time = []
296 convolution_subplot_data_spectrum = []
297
298 for i, c in enumerate(c_range):
299     freq_response_row = []
300     for j, b1 in enumerate(b1_range):
301         time_row_b = []
302         spectrum_row_b = []
303         conv_time_row = []
304         conv_freq_row = []
305
306         for k, d in enumerate(d_range):
307             g_signal = np.array([g(ti, a) for ti in t])
308             u_signal = np.array([u(ti, a, b, c, d) for ti in t])
309
310             W = signal.TransferFunction([1, a1, a2], [1, b1, b2])
311             _, y_signal, _ = signal.lsim(W, U=u_signal, T=t)
312
313             freq_g, func_g = calc_spectrum(g_signal, dt)
314             freq_u, func_u = calc_spectrum(u_signal, dt)
315             freq_y, func_y = calc_spectrum(y_signal, dt)
316
317             W_mag = ((1j*freq_g)**2 + a1*(1j*freq_g) + a2) /
318                     ↪ ((1j*freq_g)**2 + b1*(1j*freq_g) + b2)
319
320             afc_mask = (freq_g ≥ 0) & (freq_g < 120)
321             omega_mask = np.abs(freq_g) < 35
322             t_mask = t < 7
323
324             time_plot = [
325                 [t[t_mask]] * 3,
326                 [u_signal[t_mask], g_signal[t_mask], y_signal[t_mask]],
327                 ['u(t)', 'g(t)', 'y(t), filtered'],
328                 ['t', 'Amplitude',
329                  'g', 'b', 'r'],
330                 ['-'] * 3,
331                 [1.0, 1.0, 1.0],
332                 [None] * 3,
333                 [None] * 3,
334                 f'b1={b1:.2f}, c={c}, d={d}',
335             ]
336             time_row_b.append([time_plot])
337
338             spectrum_plot = [
339                 [freq_u[omega_mask], freq_g[omega_mask],
340                  ↪ freq_y[omega_mask]],
341                 [2.0 / N * func_u[omega_mask],
342                  2.0 / N * func_g[omega_mask],
343                  2.0 / N * func_y[omega_mask]],
344                 ['|u_hat(w)|', '|g_hat(w)|', 'y_hat(w)|'],
345                 ['w', 'Amplitude', 'r'],
346                 ['b', "#039D71", 'r'],
347                 ['-'] * 3,
348                 [1.0, 2.0, 1.0],
349                 [None] * 3,
350                 [None] * 3,
351                 f'b1={b1}, c={c}, d={d}',

```

```

350     ]
351     spectrum_row_b.append([spectrum_plot])
352
353     if c == c_range[0] and d == d_range[0]:
354         freq_response_plot = [
355             [freq_g[afc_mask]],
356             [W_mag[afc_mask]],
357             ['|W(iw)|'],
358             ['ω', '|W(iw)|'],
359             ['m'],
360             ['-'],
361             [1.5],
362             [None],
363             [None],
364             f'b1={b1}',
365         ]
366         freq_response_row.append(freq_response_plot)
367
368     W_freq = ((1j*freq_u)**2 + a1*(1j*freq_u) + a2) /
369     ↪ ((1j*freq_u)**2 + b1*(1j*freq_u) + b2)
370     y_freq = ifft(ifftshift(W_freq) * fft(u_signal)).real
371
372     conv_plot_time = [
373         [t[t<7], t[t<7]],
374         [y_signal[t<7], y_freq[t<7]],
375         ['y(t)', 'F^{-1}{W_1(iw)·u_hat(ω)}'],
376         ['t', 'Amplitude'],
377         ['r', 'b'],
378         ['-', '--'],
379         [0.75, 0.75],
380         [None]*2,
381         [None]*2,
382         f'Convolution check (b1={b1:.2f}, c={c}, d={d})'
383     ]
384     conv_time_row.append([conv_plot_time])
385
386     func_y_mult = np.abs(W_mag) * func_u
387
388     conv_plot_freq = [
389         [freq_y[omega_mask], freq_u[omega_mask]],
390         [2.0 / N * func_y[omega_mask], 2.0 / N *
391         ↪ func_y_mult[omega_mask]],
392         ['|y_hat(ω)|', 'W_1(iw)·u_hat(ω)'],
393         ['ω', 'Amplitude'],
394         ['r', 'b'],
395         ['-', '--'],
396         [1, 1],
397         [None]*2,
398         [None]*2,
399         f'Convolution check (b1={b1:.2f}, c={c}, d={d})'
400     ]
401     conv_freq_row.append([conv_plot_freq])
402
403     time_subplot_data.append([time_row_b])
404     spectrum_subplot_data.append([spectrum_row_b])
405     convolution_subplot_data_time.append([conv_time_row])
406     convolution_subplot_data_spectrum.append([conv_freq_row])
407
408     freq_response_subplot_data.append(freq_response_row)
409
410 for tsd in time_subplot_data:
411     for b in tsd:
412         draw_plots(
413             rows=3,
414             cols=1,
415             width=9,
416             height=13,
417             subplot_data=b,
418             legend_loc='upper right',
419             legend_fontsize='medium',
420         )

```

```

419
420 for ssd in spectrum_subplot_data:
421     for b in ssd:
422         draw_plots(
423             rows=3,
424             cols=1,
425             width=9,
426             height=13,
427             subplot_data=b,
428             legend_loc='upper right',
429             legend_fontsize='medium',
430         )
431
432 draw_plots(
433     rows=2,
434     cols=2,
435     width=10,
436     height=8,
437     subplot_data=freq_response_subplot_data,
438     legend_loc='upper right',
439     legend_fontsize='medium',
440 )
441
442 for csd in convolution_subplot_data_time:
443     for b in csd:
444         draw_plots(
445             rows=3,
446             cols=1,
447             width=9,
448             height=10,
449             subplot_data=b,
450             legend_loc='upper right'
451         )
452
453 for css in convolution_subplot_data_spectrum:
454     for b in css:
455         draw_plots(
456             rows=3,
457             cols=1,
458             width=9,
459             height=10,
460             subplot_data=b,
461             legend_loc='upper right'
462         )
463
464 # %% [markdown]
465 # # Задание 2
466
467 # %%
468 data = pd.read_csv('SBER_DATA.csv', sep=';', parse_dates=['<DATE>'],
469                  date_parser=lambda x: datetime.strptime(x, '%y%m%d'))
470 data = data.sort_values('<DATE>')
471
472 dates = data['<DATE>'].values
473 prices = data['<CLOSE>'].values
474 t_days = np.arange(len(dates))
475
476 time_constants_days = [1, 7, 30, 90, 365]
477
478 filtered_signals = []
479 for tc in time_constants_days:
480     W = signal.TransferFunction([1], [tc, 1])
481     _, y_signal, _ = signal.lsim(W, U=prices, T=t_days, X0=prices[0] *
482                                ↪ tc)
483     filtered_signals.append(y_signal)
484
485 plot_data = []
486
487 for i, tc in enumerate(time_constants_days):
488     if tc == 1:
489         tc_label = "1 день"
490     elif tc == 7:
491         tc_label = "1 неделя"

```

```

492     tc_label = "1 месяц"
493 elif tc == 90:
494     tc_label = "3 месяца"
495 else:
496     tc_label = "1 год"
497
498 plot_data.append([
499     [dates] * 2,
500     [prices, filtered_signals[i]],
501     ['Цена открытия', f'Сглаживание'],
502     'Дата', 'Цена, руб.',
503     ['g', 'r'],
504     ['-'] * 2,
505     [0.7, 1.0],
506     [None] * 2,
507     [None] * 2,
508     f'T={tc_label}'
509 ])
510
511 draw_plots(
512     rows=6,
513     cols=1,
514     width=12,
515     height=26,
516     subplot_data=[plot_data],
517     legend_loc='lower right'
518 )
519
520

```

Листинг 1: Листинг